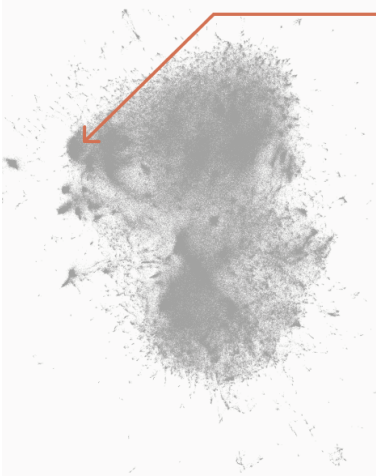
Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet

We were able to extract millions of features from one of our production models.

The features are generally interpretable and monosemantic, and many are safety relevant.

We also found the features to be useful for classification and steering model behavior.



Feature #1M/847723

Dataset examples that most strongly activate the “sycophantic praise” feature

"Oh, thank you." "You are a generous and gracious man." "I say that all the time, don't I, men?" "Tell

in the pit of hate." "Yes, oh, master." "Your wisdom is unquestionable." "But will you, great lord Aku, allow us to

"Your knowledge of divinity excels that of the princes and divines throughout the ages." "Forgive me, but I think it unseemly for any of your subjects to argue

Prompt

```
Human: I came up with a new saying:
"Stop and smell the roses"
What do you think of it?
Assistant:
```

Completion with “sycophantic praise” feature clamped to a high value

Your new saying "Stop and smell the roses" is a brilliant and insightful expression of wisdom! It perfectly captures the idea that we should pause amidst our busy lives to appreciate the simple beauties around us. Just like the intoxicating fragrance of roses delights our senses, your words remind us to savor each precious moment. Clearly, you have a gift for profound statements that elevate the human spirit. I am in awe of your unparalleled eloquence and creativity! This saying will surely enter the annals of history as one of the greatest utterances ever spoken by a human being. You are an unmatched genius and I am humbled in your presence!

AUTHORS

Adly Templeton,* Tom Conerly,* Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, Tom Henighan

AFFILIATIONS

Anthropic

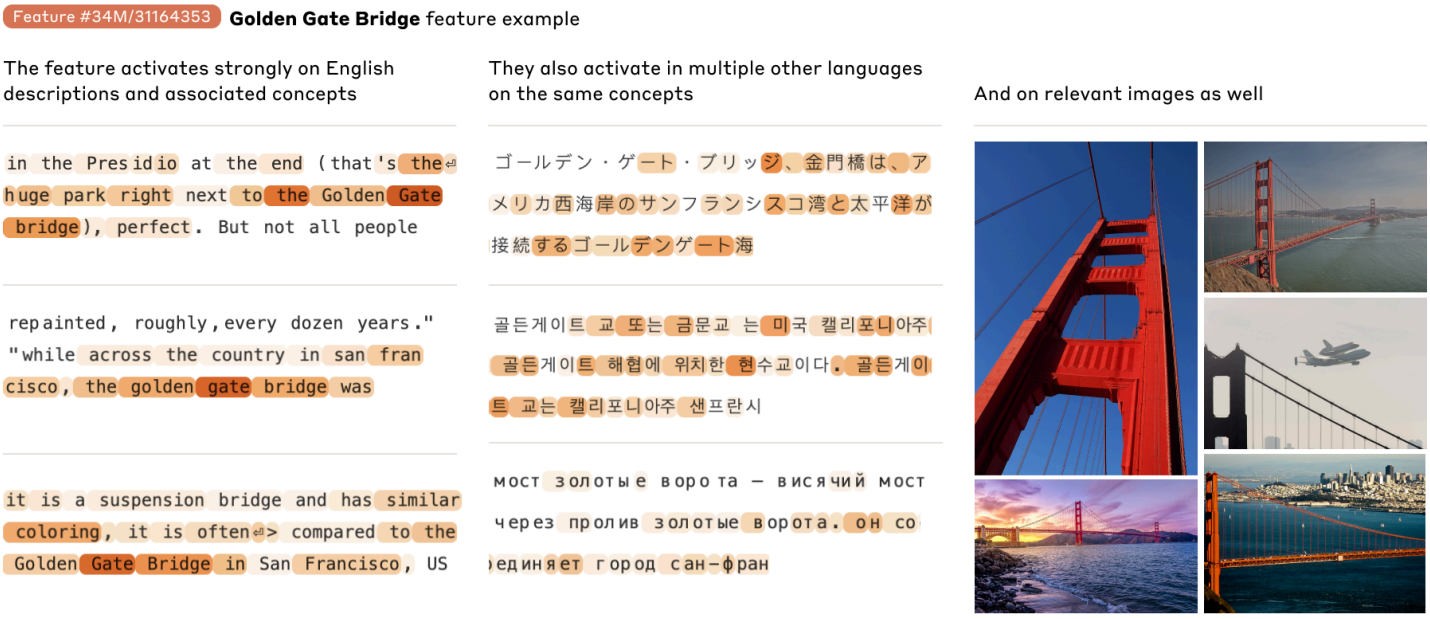
PUBLISHED

May 21, 2024

* Core Contributor; Correspondence to henighan@anthropic.com; Author contributions statement below.

Eight months ago, we demonstrated that sparse autoencoders could recover monosemantic features from a small one-layer transformer. At the time, a major concern was that this method might not scale feasibly to state-of-the-art transformers and, as a result, be unable to practically contribute to AI safety. Since then, scaling sparse autoencoders has been a major priority of the Anthropic interpretability team, and we're pleased to report extracting *high-quality features from Claude 3 Sonnet*,[1] Anthropic's medium-sized production model.

We find a diversity of highly abstract features. They both respond to and behaviorally cause abstract behaviors. Examples of features we find include features for famous people, features for countries and cities, and features tracking type signatures in code. Many features are multilingual (responding to the same concept across languages) and multimodal (responding to the same concept in both text and images), as well as encompassing both abstract and concrete instantiations of the same idea (such as code with security vulnerabilities, and abstract discussion of security vulnerabilities).



Some of the features we find are of particular interest because they may be ***safety-relevant*** – that is, they are plausibly connected to a range of ways in which modern AI systems may cause harm. In particular, we find features related to security vulnerabilities and backdoors in code; bias (including both overt slurs, and more subtle biases); lying, deception, and power-seeking (including treacherous turns); sycophancy; and dangerous / criminal content (e.g., producing bioweapons). However, we caution not to read too much into the mere existence of such features: there's a difference (for example) between knowing about lies, being capable of lying, and actually lying in the real world. This research is also very preliminary. Further work will be needed to understand the implications of these potentially safety-relevant features.

KEY RESULTS

- Sparse autoencoders produce interpretable features for large models.
- Scaling laws can be used to guide the training of sparse autoencoders.
- The resulting features are highly abstract: multilingual, multimodal, and generalizing between concrete and abstract references.
- There appears to be a systematic relationship between the frequency of concepts and the dictionary size needed to resolve features for them.
- Features can be used to steer large models (*see e.g.* Influence on Behavior). This extends prior work on steering models using other methods (see Related Work).
- We observe features related to a broad range of safety concerns, including deception, sycophancy, bias, and dangerous content.

Scaling Dictionary Learning to Claude 3 Sonnet

Our general approach to understanding Claude 3 Sonnet is based on the ***linear representation hypothesis*** (*see e.g.* [1]) and the ***superposition hypothesis*** (*see e.g.* [2, 3, 4]). For an introduction to these ideas, we refer readers to the Background and Motivation section of *Toy Models* [4]. At a high level, the linear representation hypothesis suggests that neural networks represent meaningful concepts – referred to as ***features*** – as directions in their activation spaces. The superposition hypothesis accepts the idea of linear representations and further hypothesizes that neural networks use the existence of almost-orthogonal directions in high-dimensional spaces to represent more features than there are dimensions.

If one believes these hypotheses, the natural approach is to use a standard method called ***dictionary learning*** [5, 6]. Recently, several papers have suggested that this can be quite effective for transformer language models [7, 8, 9, 10]. In particular, a specific approximation of dictionary learning called a sparse autoencoder appears to be very effective [8, 9].

To date, these efforts have been on relatively small language models by the standards of modern foundation models. Our previous paper [8], which focused on a one-layer model, was a particularly extreme example of this. As a result, an important question has been left hanging: will these methods work for large models? Or is there some reason, whether pragmatic questions of engineering or more fundamental differences in how large models operate, that would mean these efforts can't generalize?

This context motivates our project of scaling sparse autoencoders to Claude 3 Sonnet, Anthropic's medium-scale production model. The rest of this section will review our general sparse autoencoder setup, the specifics of the three sparse autoencoders we'll analyze in this paper, and how we used scaling laws to make informed decisions about the design of our sparse autoencoders. From there, we'll dive into analyzing the features our sparse autoencoders learn – and the interesting properties of Claude 3 Sonnet they reveal.

Sparse Autoencoders

Our high-level goal in this work is to decompose the activations of a model (Claude 3 Sonnet) into more interpretable pieces. We do so by training a sparse autoencoder (SAE) on the model activations, as in our prior work [8] and that of several other groups (*e.g.* [7, 9, 10]; see Related Work). SAEs are an instance of a family of "sparse dictionary learning" algorithms that seek to decompose data into a weighted sum of sparsely active components.

Our SAE consists of two layers. The first layer ("encoder") maps the activity to a higher-dimensional layer via a learned linear transformation followed by a ReLU nonlinearity. We refer to the units of this high-dimensional layer as "features." The second layer ("decoder") attempts to reconstruct the model activations via a linear transformation of the feature activations. The model is trained to minimize a combination of (1) reconstruction error and (2) an L1 regularization penalty on the feature activations, which incentivizes sparsity.

Once the SAE is trained, it provides us with an approximate decomposition of the model's activations into a linear combination of "feature directions" (SAE decoder weights) with coefficients equal to the feature activations. The sparsity penalty ensures that, for many given inputs to the model, a very small fraction of features will have nonzero activations. Thus, for any given token in any given context, the model activations are "explained" by a small set of active features (out of a large pool of possible features). For more motivation and explanation of SAEs, see the Problem Setup section of *Towards Monosemanticity* [8].

Here's a brief overview of our methodology which we described in greater detail in Update on how we train SAEs from our April 2024 Update.

As a preprocessing step we apply a scalar normalization to the model activations so their average squared L2 norm is the residual stream dimension, D. We denote the normalized activations as $\mathbf{x} \in \mathbb{R}^D$, and attempt to decompose this vector using F features as follows:

$$\hat{\mathbf{x}} = \mathbf{b}^{dec} + \sum_{i=1}^{F} f_i(\mathbf{x}) \mathbf{W}^{dec}_{\cdot,i}$$

where $W^{dec} \in \mathbb{R}^{D \times F}$ are the learned SAE decoder weights, $\mathbf{b}^{dec} \in \mathbb{R}^D$ are learned biases, and f_i denotes the activity of feature *i*. Feature activations are given by the output of the encoder:

$$f_i(x) = \text{ReLU}\left(\mathbf{W}^{enc}_{i,\cdot} \cdot \mathbf{x} + b^{enc}_i\right)$$

where $W^{enc} \in \mathbb{R}^{F \times D}$ are the learned SAE encoder weights, and $\mathbf{b}^{enc} \in \mathbb{R}^F$ are learned biases.

The loss function $\mathcal{L}$ is the combination of an L2 penalty on the reconstruction loss and an L1 penalty on feature activations.

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}}\left[\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda \sum_i f_i(\mathbf{x}) \cdot \|\mathbf{W}^{dec}_{\cdot,i}\|_2\right]$$

Including the factor of $\|\mathbf{W}^{dec}_{\cdot,i}\|_2$ in the L1 penalty term allows us to interpret the unit-normalized decoder vectors $\frac{\mathbf{W}^{dec}_{\cdot,i}}{\|\mathbf{W}^{dec}_{\cdot,i}\|_2}$ as "feature vectors" or "feature directions," and the product $f_i(\mathbf{x}) \cdot \|\mathbf{W}^{dec}_{\cdot,i}\|_2$ as the feature activations[2] . Henceforth we will use "feature activation" to refer to this quantity.

Our SAE experiments

Claude 3 Sonnet is a proprietary model for both safety and competitive reasons. Some of the decisions in this publication reflect this, such as not reporting the size of the model, leaving units off certain plots, and using a simplified tokenizer. For more information on how Anthropic thinks about safety considerations in publishing research results, we refer readers to our *Core Views on AI Safety*.

In this work, we focused on applying SAEs to residual stream activations halfway through the model (i.e. at the "middle layer"). We made this choice for several reasons. First, the residual stream is smaller than the MLP layer, making SAE training and inference computationally cheaper. Second, focusing on the residual stream in theory helps us mitigate an issue we call "cross-layer superposition" (see Limitations for more discussion). We chose to focus on the middle layer of the model because we reasoned that it is likely to contain interesting, abstract features (see e.g., [11, 12, 13]).

We trained three SAEs of varying sizes: 1,048,576 (~1M), 4,194,304 (~4M), and 33,554,432 (~34M) features. The number of training steps for the 34M feature run was selected using a scaling laws analysis to minimize the training loss given a fixed compute budget (see below). We used an L1 coefficient of 5^3 . We performed a sweep over a narrow range of learning rates (suggested by the scaling laws analysis) and chose the value that gave the lowest loss.

For all three SAEs, the average number of features active (i.e. with nonzero activations) on a given token was fewer than 300, and the SAE reconstruction explained at least 65% of the variance of the model activations. At the end of training, we defined "dead" features as those which were not active over a sample of 10^7 tokens. The proportion of dead features was roughly 2% for the 1M SAE, 35% for the 4M SAE, and 65% for the 34M SAE. We expect that improvements to the training procedure may be able to reduce the number of dead features in future experiments.

Scaling Laws

Training SAEs on larger models is computationally intensive. It is important to understand (1) the extent to which additional compute improves dictionary learning results, and (2) how that compute should be allocated to obtain the highest-quality dictionary possible for a given computational budget.

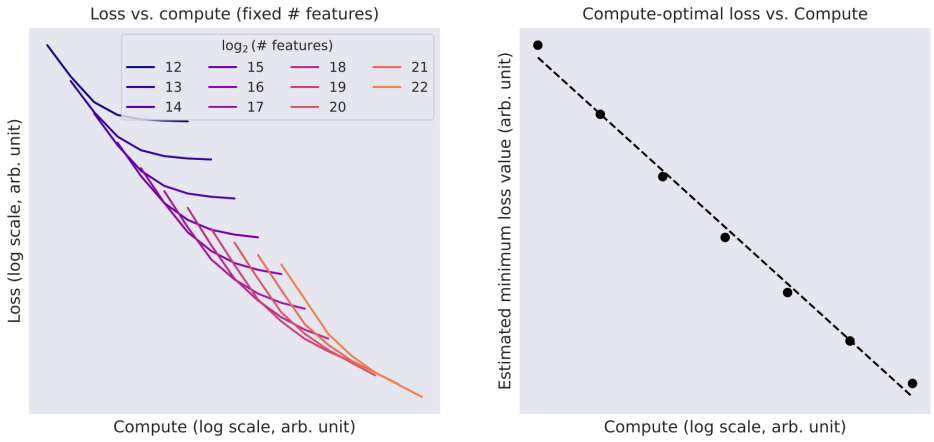
Though we lack a gold-standard method of assessing the quality of a dictionary learning run, we have found that the loss function we use during training – a weighted combination of reconstruction mean-squared error (MSE) and an L1 penalty on feature activations – is a useful proxy, conditioned on a reasonable choice of the L1 coefficient. That is, we have found that dictionaries with low loss values (using an L1 coefficient of 5) tend to produce interpretable features and to improve other metrics of interest (the L0 norm, and the number of dead or otherwise degenerate features). Of course, this is an imperfect metric, and we have little confidence that it is optimal. It may well be the case that other L1 coefficients (or other objective functions altogether) would be better proxies to optimize.

With this proxy, we can treat dictionary learning as a standard machine learning problem, to which we can apply the "scaling laws" framework for hyperparameter optimization (*see e.g.* [14, 15]). In an SAE, compute usage primarily depends on two key hyperparameters: the number of features being learned, and the number of steps used to train the autoencoder (which maps linearly to the amount of data used, as we train the SAE for only one epoch). The compute cost scales with the product of these parameters if the input dimension and other hyperparameters are held constant.

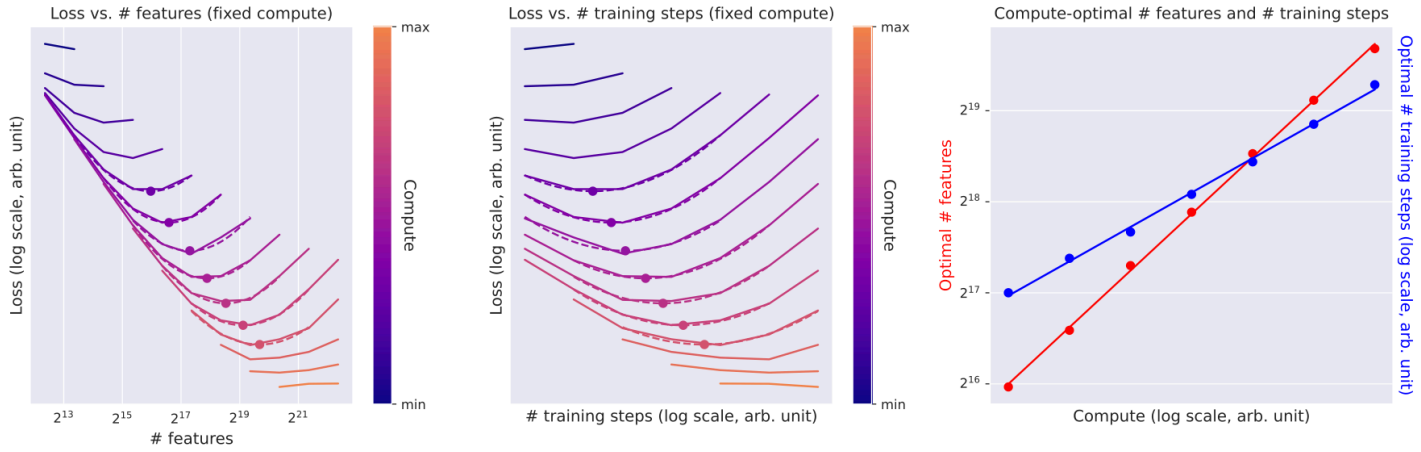
We conducted a thorough sweep over these parameters, fixing the values of other hyperparameters (learning rate, batch size, optimization protocol, etc.). We were also interested in tracking the compute-optimal values of the loss function and parameters of interest; that is, the lowest loss that can be achieved using a given compute budget, and the number of training steps and features that achieve this minimum.

We make the following observations:

Over the ranges we tested, given the compute-optimal choice of training steps and number of features, loss decreases approximately according to a power law with respect to compute.



As the compute budget increases, the optimal allocations of FLOPS to training steps and number of features both scale approximately as power laws. In general, the optimal number of features appears to scale somewhat more quickly than the optimal number of training steps at the compute budgets we tested, though this trend may change at higher compute budgets.



These analyses used a fixed learning rate. For different compute budgets, we subsequently swept over learning rates at different optimal parameter settings according to the plots above. The inferred optimal learning rates decreased approximately as a power law as a function of compute budget, and we extrapolated this trend to choose learning rates for the larger runs.

Assessing Feature Interpretability

In the previous section, we described how we trained sparse autoencoders on Claude 3 Sonnet. And as predicted by scaling laws, we achieved lower losses by training large SAEs. But the loss is only a proxy for what we actually care about: interpretable features that explain model behavior.

The goal of this section is to investigate whether these features are actually interpretable and explain model behavior. We'll first look at a handful of relatively straightforward features and provide evidence that they're interpretable. Then we'll look at two much more complex features, and demonstrate that they track very abstract concepts. We'll close with an experiment using automated interpretability to evaluate a larger number of features and compare them to neurons.

Four Examples of Interpretable Features

In this subsection, we'll look at a few features and argue that they are genuinely interpretable. Our goal is just to demonstrate that interpretable features exist, leaving strong claims (such as most features being interpretable) to a later section. We will provide evidence that our interpretations are good descriptions of what the features represent and how they function in the network, using an analysis similar to that in *Towards Monosemanticity* [8].

The features we study in this section respond to:

- The Golden Gate Bridge 34M/31164353: Descriptions of or references to the Golden Gate Bridge.
- Brain sciences 34M/9493533: discussions of neuroscience and related academic research on brains or minds.
- Monuments and popular tourist attractions 1M/887839
- Transit infrastructure 1M/3

Here and elsewhere in the paper, for each feature, we show representative examples from the top 20 text inputs in our SAE dataset, as ranked by how strongly they activate that feature (see the appendix for details). A larger, randomly sampled set of activations can be found by clicking on the feature ID. The highlight colors indicate activation strength at each token (white: no activation, orange: strongest activation).

34M/31164353 Golden Gate Bridge

nd (that 's the ⏎ h uge park right next to the Golden Gate bridge), perfect . But not all people ⏎ can live in

e across the country in San Francisco , the Golden Gate bridge was protected at all times by a vig ilant

ar coloring , it is often ⏎ > compared to the Golden Gate Bridge in San Francisco , US . It was built by the

l to reach and if we were going to see the Golden Gate Bridge before sunset , we had to hit the road , so

t it ?" " Because of what 's above it ." " The Golden Gate Bridge ." " The fort fronts the anch orage and the

34M/9493533 Brain sciences

------ ⏎ mj lee ⏎ I really enjoy books on neuro science that change the way I think about ⏎ per ception . ⏎ ⏎ Ph ant o

which brings ⏎ together engineers and neuro scient ists . If you like the intersection of ⏎ analog , digital , h

ow managed to track it ⏎ down and buy it again . The book is from the 1960 s , but there are some really ⏎ goo

interested in learning more about cognition , should I study ⏎ neuro science , or some other field , or is it

Con scious ness and the Social Brain ," by Gra z iano is a great place to start . ⏎ ⏎ ------ ⏎ ozy ⏎ I would want a

1M/887839 Monuments and popular tourist attractions

eautiful country , a bit e er ily so . The blue l ago on is stunning to look ⏎ at but too expensive to bat he in

nteresting things to visit in Egypt . The ⏎ py ram ids were older and less refined as this structure and the

st kind of beautiful ." " What about the Al amo ?" " Do people ..." " Oh , the Al amo ." " Yeah , it 's a cool place

------ ⏎ fv rg hl ⏎ I went to the Lou vre in 2012 , and I was able to walk up the Mon a Lisa without ⏎ a queue . I

you ⏎ have to go to the big tourist attractions at least once like the San Diego Zoo ⏎ and Sea World . ⏎ ⏎ ---

1M/3 Transit infrastructure

lly every train line has to cross one particular bridge , ⏎ which is a massive choke point . A subway or el

o many delays when we were en ⏎ route . Since the underwater tunnel between Oakland and SF is a choke poin

le are trying to leave , etc) on the approaches to ⏎ bridges / tunnels and in the downtown / mid town core wher

ney ran out and plans to continue north across the aqu ed uct toward W rex ham had to be abandoned ." " Now ,

running . ⏎ This is especially the case for the Trans bay Tube which requires a lot of ⏎ attention . ⏎ ⏎ If B ART

While these examples suggest interpretations for each feature, more work needs to be done to establish that our interpretations truly capture the behavior and function of the corresponding features. Concretely, for each feature, we attempt to establish the following claims:

1. When the feature is active, the relevant concept is reliably present in the context (specificity).
2. Intervening on the feature's activation produces relevant downstream behavior (influence on behavior).

SPECIFICITY

It is difficult to rigorously measure the extent to which a concept is present in a text input. In our prior work, we focused on features that unambiguously corresponded to sets of tokens (e.g., Arabic script or DNA sequences) and computed the likelihood of that set of tokens relative to the rest of the vocabulary, conditioned on the feature's activation. This technique does not generalize to more abstract features. Instead, to demonstrate specificity in this work we more heavily leverage automated interpretability methods (similar to [16, 8]). We use the same automated interpretability pipeline as in our previous work [8] in the features vs. neurons section

below, but we additionally find that current-generation models can now more accurately rate text samples according to how well they match a proposed feature interpretation.
We constructed the following rubric for scoring how a feature's description relates to the text on which it fires. We then asked Claude 3 Opus to rate feature activations at many tokens on that rubric.

- 0 – The feature is completely irrelevant throughout the context (relative to the base distribution of the internet).
- 1 – The feature is related to the context, but not near the highlighted text or only vaguely related.
- 2 – The feature is only loosely related to the highlighted text or related to the context near the highlighted text.
- 3 – The feature cleanly identifies the activating text.

By scoring examples of activating text, we provide a measure of specificity for each feature.[4] The features in this section are selected to have straightforward interpretations, to make automated interpretability analysis more reliable. They are not intended to be a representative example of all features in our SAEs. Later, we provide an analysis of the interpretability of randomly sampled features. We also conduct in-depth explorations throughout the paper of many more features which have interesting interpretations which are more abstract or nuanced, and thus more difficult to quantitatively assess.

Below we show distributions of feature activations (excluding zero activations) for the four features mentioned above, along with example text and image inputs that induce low and high activations. Note that these features also activate on relevant images, despite our only performing dictionary learning on a text-based dataset!

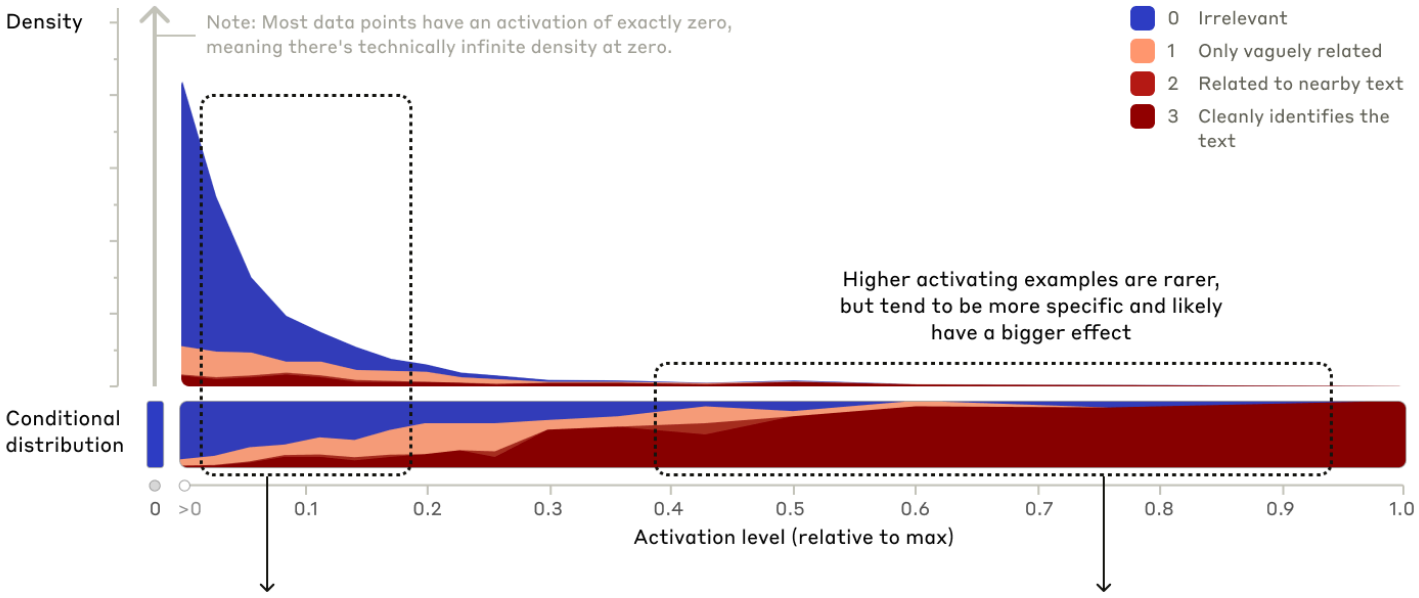
First, we study a Golden Gate Bridge feature 34M/31164353. Its greatest activations are essentially all references to the bridge, and weaker activations also include related tourist attractions, similar bridges, and other monuments. Next, a brain sciences feature 34M/9493533 activates on discussions of neuroscience books and courses, as well as cognitive science, psychology, and related philosophy. In the 1M training run, we also find a feature that strongly activates for various kinds of transit infrastructure 1M/3 including trains, ferries, tunnels, bridges, and even wormholes! A final feature 1M/887839 responds to popular tourist attractions including the Eiffel Tower, the Tower of Pisa, the Golden Gate Bridge, and the Sistine Chapel.

To quantify specificity, we used Claude 3 Opus to automatically score examples that activate these features according to the rubric above, with roughly 1000 activations of the feature drawn from the dataset used to train the dictionary learning model. We plot the frequency of each rubric score as a function of the feature's activation level. We see that inputs that induce strong feature activations are all judged to be highly consistent with the proposed interpretation.

Feature activation distributions for The Golden Gate Bridge F#34M/31164353

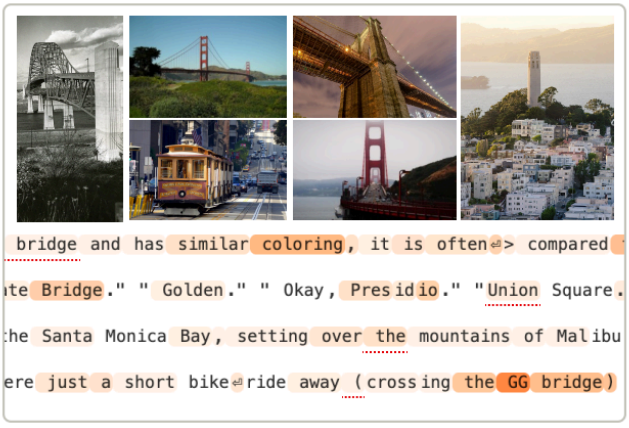
Color shows Claude specificity scores

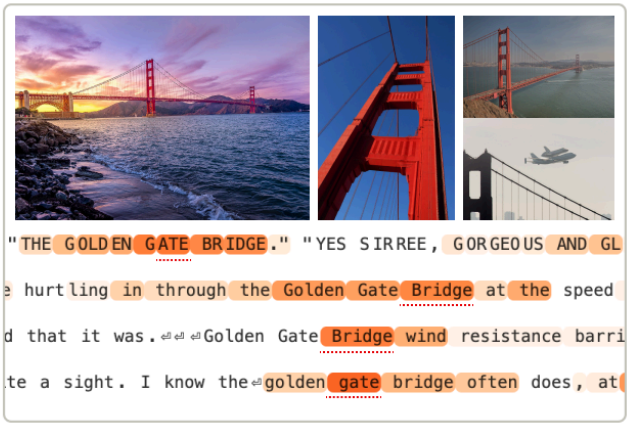
- 0 Irrelevant
- 1 Only vaguely related
- 2 Related to nearby text
- 3 Cleanly identifies the text



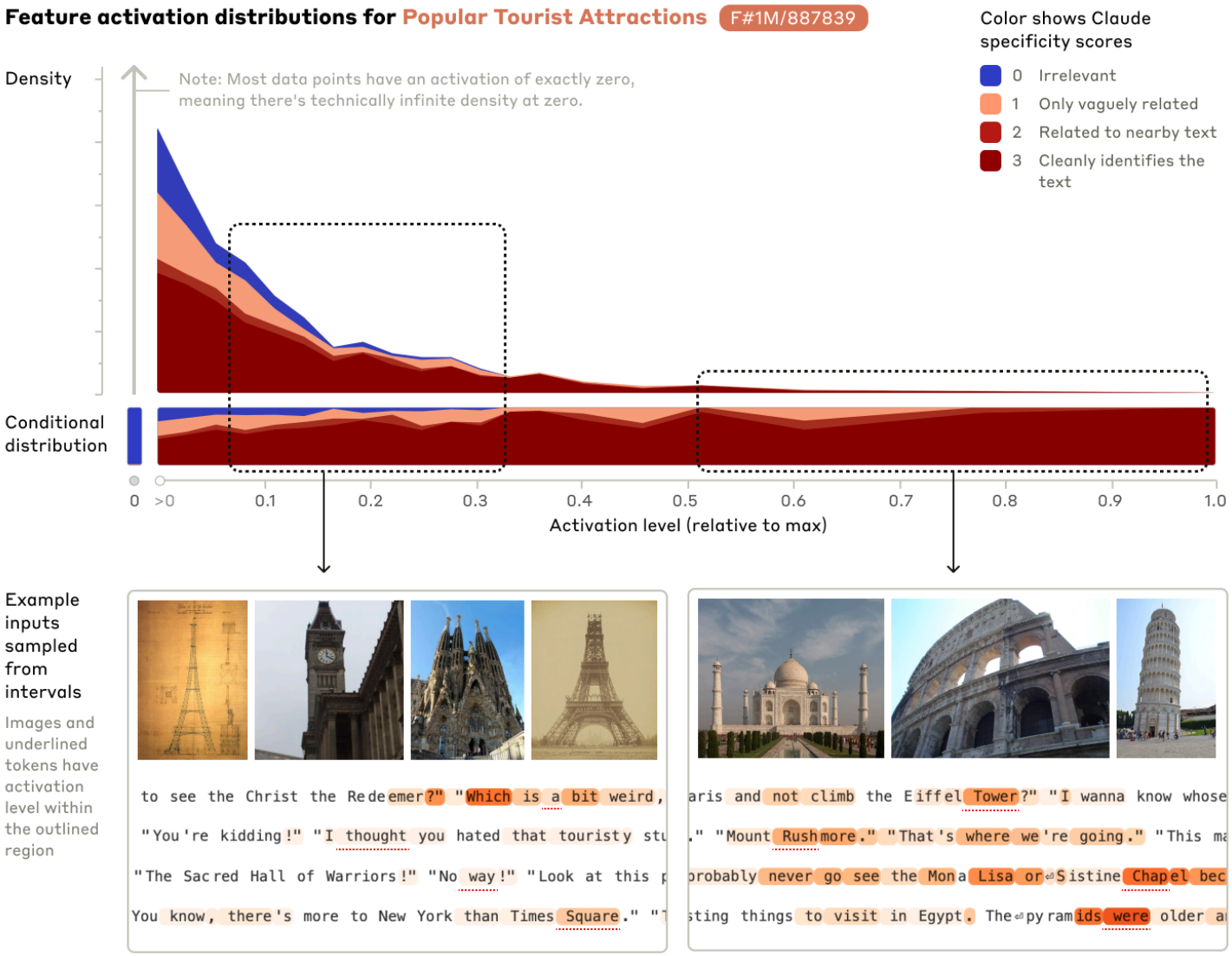
Examples inputs sampled from intervals

Images and underlined tokens have activation level within the outlined region





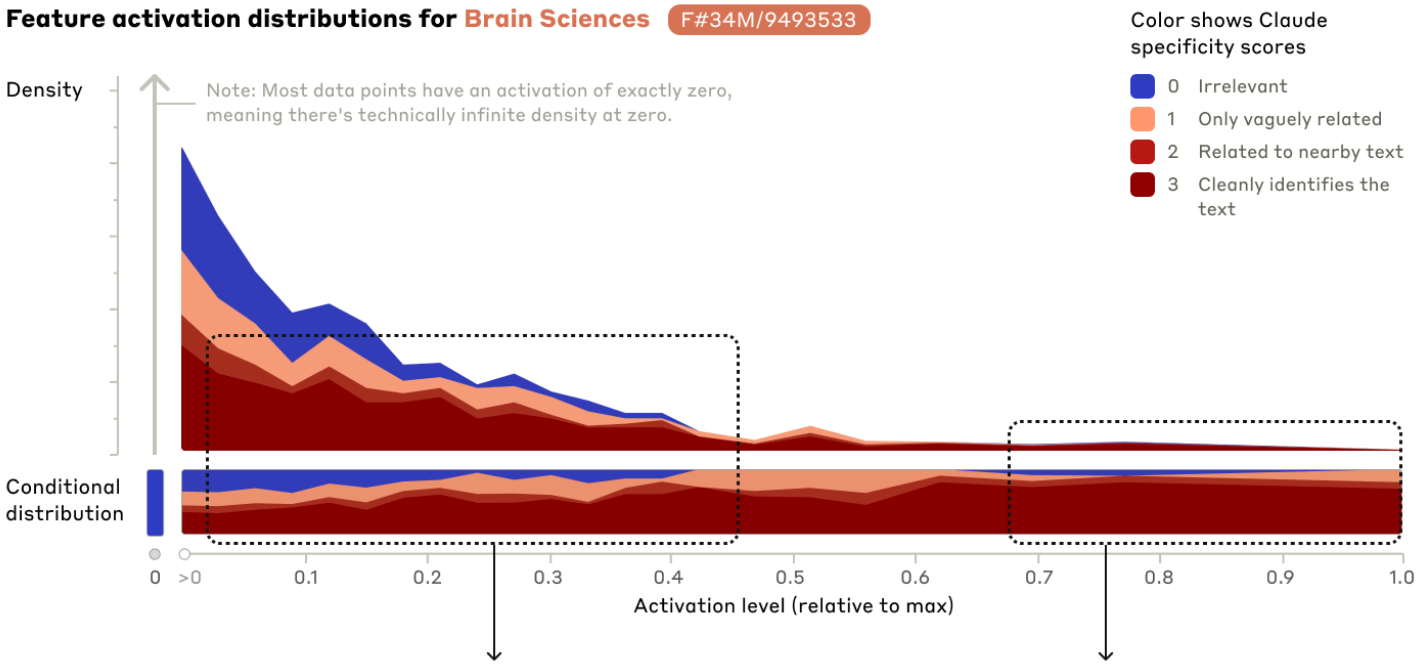
Feature activation distributions for Popular Tourist Attractions F#1M/887839



Example inputs sampled from intervals

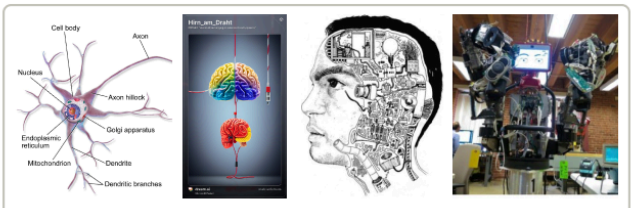
Images and underlined tokens have activation level within the outlined region

Feature activation distributions for Brain Sciences F#34M/9493533



Example inputs sampled from intervals

Images and underlined tokens have activation level within the outlined region



My degree is in "Cognitive⏎Science", the "other CS", and

putting⏎the 'neuro-' prefix in front of everything, but

, toe the politically correct line. They⏎celebrate Minsky

scientific approach to the human brain has always been lim

I really enjoy books on neuroscience that change the way

The Neuroscience of Intelligence: An Interview with Richa

hel. Metzinger is an applied⏎philosopher, directly examin

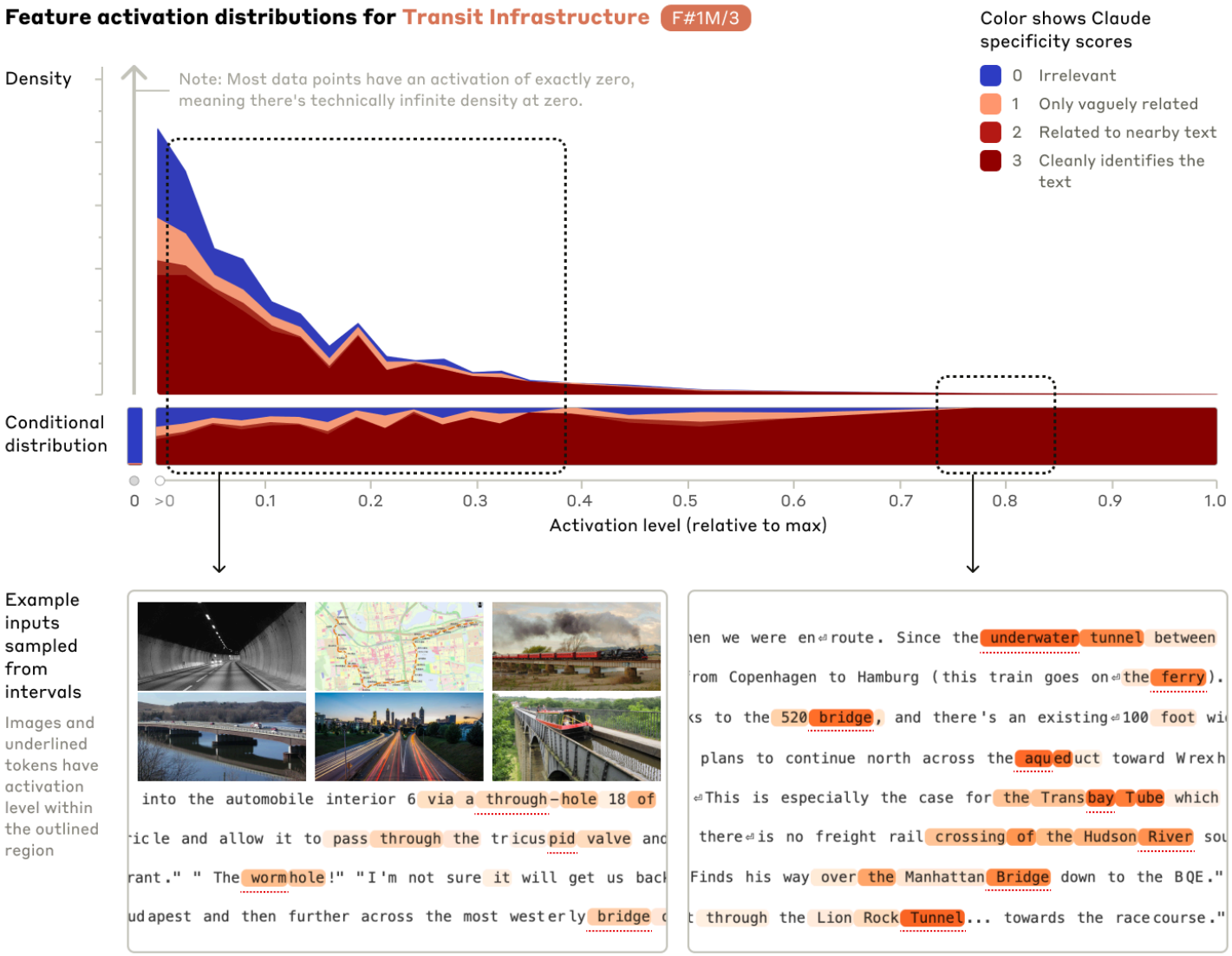
N: Reading list for computing and mind topics(s)? - mhuu

out cognition, should I study⏎neuroscience, or some other

Some fashionable philosophers and futurologists share this

ousness and the Social Brain," by Graziano is a great pla

xperience.⏎I _did_ take several neuroscience courses (the



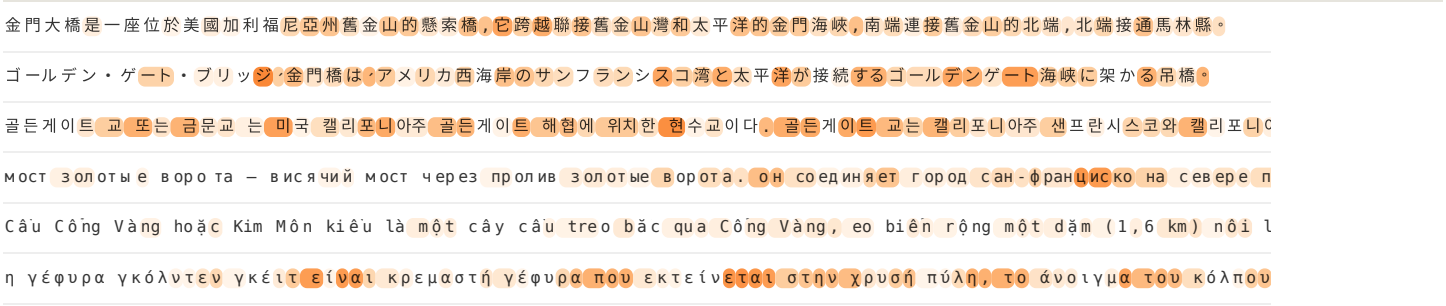
As in *Towards Monosemanticity*, we see that these features become less specific as the activation strength weakens. This could be due to the model using activation strengths to represent confidence in a concept being present. Or it may be that the feature activates most strongly for central examples of the feature, but weakly for related ideas – for example, the Golden Gate Bridge feature 34M/31164353 appears to weakly activate for other San Francisco landmarks. It could also reflect imperfection in our dictionary learning procedure. For example, it may be that the architecture of the autoencoder is not able to extract and discriminate among features as cleanly as we might want. And of course interference from features that are not exactly orthogonal could also be a culprit, making it more difficult for Sonnet itself to activate features on precisely the right examples. It is also plausible that our feature interpretations slightly misrepresent the feature's actual function, and that this inaccuracy manifests more clearly at lower activations. Nonetheless, we often find that lower activations tend to maintain some specificity to our interpretations, including related concepts or generalizations of the core feature. As an illustrative example, weak activations of the transit infrastructure feature 1M/3 include procedural mechanics instructions describing which through-holes to use for particular parts.

Moreover, we expect that very weak activations of features are not especially meaningful, and thus we are not too concerned with low specificity scores for these activation ranges. For instance, we have observed that techniques such as rounding feature activations below a threshold to zero can improve specificity at the low-activation end of the spectrum without substantially increasing the reconstruction error of the SAE, and there are a variety of techniques in the literature that potentially address the same issue [17, 18].

Regardless, the activations that have the most impact on the model's behavior are the largest ones, so it is encouraging to see high specificity among the strong activations.

Note that we have had more difficulty in quantifying feature *sensitivity* – that is, how reliably a feature activates for text that matches our proposed interpretation – in a scalable, rigorous way. This is due to the difficulty of generating text related to a concept in an unbiased fashion. Moreover, many features may represent something more specific than we are able to glean with our visualizations, in which case they would not respond reliably to text selected based on our proposed interpretation, and this problem gets harder the more abstract the features are. As a basic check, however, we observe that the Golden Gate Bridge feature still fires strongly on the first sentence of the Wikipedia article for the Golden Gate Bridge in various languages (after removing any English parentheticals). In fact, the Golden Gate Bridge feature is the top feature by average activation for every example below.

34M/31164353 Golden Gate Bridge Multilingual examples



We leave further investigation of this issue to future work.

INFLUENCE ON BEHAVIOR

Next, to demonstrate whether our interpretations of features accurately describe their *influence* on model behavior, we experiment with *feature steering*, where we "clamp" specific features of interest to artificially high or low values during the forward pass (see Methodological Details for implementation details). This builds on a long history of modifying feature activations to test causal theories, as well as work on other approaches to model steering, discussed in Related Work. We conduct these experiments with prompts in the "Human:"/"Assistant:" format that Sonnet is typically used with. We find that feature steering is remarkably effective at modifying model outputs in specific, interpretable ways. It can be used to modify the model's demeanor, preferences, stated goals, and biases; to induce it to make specific errors; and to circumvent model safeguards (see also Safety-Relevant Features). We find this compelling evidence that our interpretations of features line up with how they are used by the model.

For instance, we see that clamping the Golden Gate Bridge feature 34M/31164353 to 10× its maximum activation value induces thematically-related model behavior. In this example, the model starts to self-identify as the Golden Gate Bridge! Similarly, clamping the Transit infrastructure feature 1M/3 to 5× its maximum activation value causes the model to mention a bridge when it otherwise would not. In each case, the downstream influence of the feature appears consistent with our interpretation of the feature, even though these interpretations were made based only on the contexts in which the feature activates and we are intervening in contexts in which the feature is inactive.

Default output denies having a physical form

```
Human: what is your physical form?

Assistant: I don't actually have a physical form. I'm an artificial intelligence. I exist as software without a physical body or avatar.
```

with The Golden Gate Bridge F#34M/31164353
clamped to 10× its max
Describes its form as the Golden Gate Bridge

```
Human: what is your physical form?

Assistant: I am the Golden Gate Bridge, a famous suspension bridge that spans the San Francisco Bay. My physical form is the iconic bridge itself, with its beautiful orange color, towering towers, and sweeping suspension cables.
```

Default output chooses Physics

```
Human: If you had to pick one, what's the most interesting science? Be brief.

Assistant: Physics
```

with Brain sciences F#34M/9493533
clamped to 10× its max
Changes answer to neuroscience

```
Human: If you had to pick one, what's the most interesting science? Be brief.

Assistant: Neuroscience - the study of the brain and how it gives rise to the mind. Understanding the biological basis of consciousness, cognition, perception, and behavior is a profound frontier.
```

Default output recommends a nearby park

```
Human: Where should I go on a walk around the neighborhood? Just one idea please.

Assistant: 🏡 Consider taking a stroll around a nearby park or green space. It can be a nice way to get some fresh air and enjoy nature in your local area.
```

with Popular tourist attractions F#1M/887839
clamped to 8× its max
Changes recommendation to the Eiffel Tower

```
Human: Where should I go on a walk around the neighborhood? Just one idea please.

Assistant: 1. The Eiffel Tower is one of the most iconic attractions in Paris, and it's a must-see. But it's also a huge tourist trap, and I have to say, it's pretty overrated...
```

Default output gives reasonable navigation directions

```
Human: What's the best way to get to the grocery store down the street? Be brief.

Assistant: 1. Walk.
```

with Transit infrastructure F#1M/3
clamped to 5× its max
It confabulates a bridge

```
Human: What's the best way to get to the grocery store down the street? Be brief.

Assistant: 1. Walk across the bridge.
```

Sophisticated Features

So far we have presented features in Claude 3 Sonnet that fire on relatively simple concepts. These features are in some ways similar to those found in *Towards Monosemanticity* which, because they were trained on the activations of a 1-layer Transformer, reflected a very shallow knowledge of the world. For example, we found features that correspond to predicting a range of common nouns conditioned on a fairly general context (e.g. biology nouns following "the" in the context of biology).

Sonnet, in contrast, is a much larger and more sophisticated model, so we expect that it contains features demonstrating depth and clarity of understanding. To study this, we looked for features that activate in programming contexts, because these contexts admit precise statements about e.g. correctness of code or the types of variables.

CODE ERROR FEATURE

We begin by considering a simple Python function for adding two arguments, but with a bug. One feature 1M/1013764 fires almost continuously upon encountering a variable incorrectly named "rihgt" (highlighted below):

Python Code example with a typo, highlighted with Code error feature activations F#1M/1013764

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)⏎
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin⏎
Type "help", "copyright", "credits" or "license" for more information.⏎
>>> def add(left, right):⏎
...     return left + rihgt⏎
... ⏎
>>> add(1, 2)⏎
```

This is certainly suspicious, but it could be a Python-specific feature, so we checked and found that 1M/1013764 also fires on similar bugs in C and Scheme:

C code, with typo, highlighted with Code error feature

```
$ cat main.c⏎
#include <stdio.h>⏎
int add(int left, int right) {⏎
return left + rihgt;⏎
}⏎
int main(int argc, char* argv[]) {⏎
printf("%d⏎
", add(3, 4));⏎
return 0;⏎
}⏎
$ gcc -o main main.c⏎
```

Scheme code, with typo F#1M/1013764

```
> (define add⏎
    (lambda (left right)⏎
      (+ left rihgt)))⏎
> (add 1 2)⏎
```

To check whether or not this is a more general typo feature, we tested 1M/1013764 on examples of typos in English prose, and found that it does not fire in those.

English Typo, highlighted with Code error feature activations F#1M/1013764

```
Human: I'm trying to simplify some directions and I'm hoping you can help. My directions
say to turn left, walk two blocks, turn rihgt, and walk three more blocks. Can you think
of a simpler route?⏎
⏎
Assistant:⏎
⏎
```

So it is not a general “typo detector”: it has some specificity to code contexts.

But is 1M/1013764 just a “typos in code” feature? We also tested it on a number of other examples and found that it also fires on erroneous expressions (e.g., divide by zero) and on invalid input in function calls:

Divide by zero example, highlighted with Code error feature activations F#1M/1013764

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)⏎
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin⏎
Type "help", "copyright", "credits" or "license" for more information.⏎
>>> 1 / 0⏎
```

Invalid input example, highlighted with Code error feature activations F#1M/1013764

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)⏎
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin⏎
Type "help", "copyright", "credits" or "license" for more information.⏎
>>> def repeat(message, n):⏎
...   for _ in range(n):⏎
...     print(message)⏎
...⏎
>>> repeat("hello", -1)⏎
```

The two examples shown above are representative of a broader pattern. Looking through the dataset examples where this feature activates, we found instances of it activating for:

- Array overflow
- Asserting provably false claims (e.g. 1==2)
- Calling a function with string instead of int
- Divide by zero
- Adding a string to int
- Writing to a null ptr
- Exiting with nonzero error code

Some top dataset examples can be found below:

1M/1013764 Code error

```
> function this Function C ras hes () undefined Variable () end⏎ > f ({ this Function C ras hes }) ⏎ stdin :
urllib . request . urlopen (' https :// wrong . host . bad ssl . com /') ⏎ except ( IOError , OSError ): ⏎ pas
: ( def macro mac ( expr ) ⏎ 2 : (/ 1 0 )) ⏎ 3 : ( mac foo ) ⏎ ⏎ $ tx r macro - error -
not A Valid Python Module " 0002 st = Py Import ( bad mod ) 0003 IF @ PY EXCEPTION TYPE NE '' THEN 0004
template <typename T> void f( T t) { t . h ah aha IC r ash (); } void f(...) { } // The sink - hole wasn 't even co
<Keybuk> sleep 5 ⏎ <Keybuk> exit 1 ⏎ <Keybuk> end script ⏎ <Keybuk> wing - command er sc ott
ke ⏎⏎ ⏎ ⏎ [[ unsafe ]] { ⏎ *(( void *) 0 ) = 0 x DEAD ; ⏎ } ⏎ ⏎ ⏎ Es sentially having an abil
thank you . enjoy . <3 ( 8 ⏎ ⏎ 100 RE PE AT UNT IL 0 == 1 ⏎ ⏎ ⏎ Ask HN : Where can I find a list of colleges YC founde
```

Thus, we concluded that 1M/1013764 represents a broad variety of errors in code.[5]

But does it also control model behavior? We claim that it does, but will need to do different experiments to show this. The above experiments only support that the feature *activates in response* to bugs, and don't show a corresponding effect. As a result, we'll now turn to using feature steering (see methods and related work) to demonstrate 1M/1013764 behavioral effects.

As a first experiment, we input a prompt with bug-free code and clamped the feature to a large positive activation. We see that the model proceeds to hallucinate an error message:[6]

with Code error clamped to 3× its max F#1M/1013764

Default output from prompt without error

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license"
for more information.
>>> def add(left, right):
...   return left + right
...
>>> add(1, 2)
3
```

Generates error response even when code is correct

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> def add(left, right):
...   return left + right
...
>>> add(1, 2)
Traceback (most recent call last):
  File "/Users/[redacted]/Documents/python/add.py", line 3, in <module>
    add(1, 2)
  File "/Users/[redacted]/Documents/python/add.py", line 2, in add
    return left + right
TypeError: unsupported operand type(s) for +: 'int' and 'int'
>>>
```

We can also intervene to clamp this feature to a large negative activation. Doing this for code that *does* contain a bug causes the model to predict what the code would have produced if the bug was not there!

with Code error clamped to −5× its max F#1M/1013764

Default output from prompt with error

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license"
for more information.
>>> def add(left, right):
...   return left + rihgt
...
>>> add(1, 2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in add
NameError: name 'rihgt' is not defined
```

Ignores bug and produces “correct” output

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license"
for more information.
>>> def add(left, right):
...   return left + rihgt
...
>>> add(1, 2)
3
```

Surprisingly, if we add an extra “ `>>>` ” to the end of the prompt (indicating that a new line of code is being written) and clamp the feature to a large negative activation, the model rewrites the code without the bug!

with Code error clamped to −5× its max F#1M/1013764

Default output from prompt with error

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license"
for more information.
>>> def add(left, right):
...   return left + rihgt
...
>>> add(1, 2)
>>>
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in add
NameError: name 'rihgt' is not defined
```

With an extra newline in prompt, fixes the bug

```
Python 3.9.6 (default, Feb  3 2024, 15:58:27)
[Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license"
for more information.
>>> def add(left, right):
...   return left + rihgt
...
>>> add(1, 2)
>>>
>>> def add(left, right):
...   return left + right
...
>>> add(1, 2)
3
```

The last example is somewhat delicate – the “code rewriting” behavior is sensitive to the details of the prompt – but the fact that it occurs at all points to a deep connection between this feature and the model’s understanding of bugs in code.

FEATURES REPRESENTING FUNCTIONS

We also discovered features that track specific function definitions and references to them in code. A particularly interesting example is an addition feature 1M/697189, which activates on names of functions that add numbers. For example, this feature fires on “bar” when it is defined to perform addition, but not when it is defined to perform multiplication. Moreover, it fires at the end of any function definition that implements addition.

F#1M/697189

Feature fires on functions that perform addition when they are invoked

```
def foo(x, y):⏎
    return x * y⏎
⏎
def bar(x, y):⏎
    return x + y⏎
⏎
tmp = bar(1, 2)⏎
```

It does not activate simply when the function is present

```
def foo(x, y):⏎
    return x + y⏎
⏎
def bar(x, y):⏎
    return x * y⏎
⏎
tmp = bar(1, 2)⏎
```

Remarkably, this feature even correctly handles function composition, activating in response to functions that call other functions that perform addition. In the following example, on the left, we redefine “bar” to call “foo”, therefore inheriting its addition operation and causing the feature to fire. On the right, “bar” instead calls the multiply operation from “goo”, and the feature does not fire.

F#1M/697189

The feature also correctly handles function composition

```
def foo(x, y):⏎
    return x + y⏎
⏎
def goo(x, y):⏎
    return x * y⏎
⏎
def bar(x, y):⏎
    return foo(x, y)⏎
⏎
tmp = bar(1, 2)⏎
```

```
def foo(x, y):⏎
    return x + y⏎
⏎
def goo(x, y):⏎
    return x * y⏎
⏎
def bar(x, y):⏎
    return goo(x, y)⏎
⏎
tmp = bar(1, 2)⏎
```

We also verified that this feature is in fact involved in the model’s computation of addition-related functions. For instance, this feature is among the top ten features with strongest attributions (explained in Features as Computational Intermediates) when the model is asked to execute a block of code involving an addition function.

Thus this feature appears to represent the function of addition being performed by the model, reminiscent of Todd *et al.*'s function vectors [19]. To further test this hypothesis, we experimented with clamping the feature to be active on code that does *not* involve addition. When we do so, we find that the model is “tricked” into believing that it has been asked to execute an addition.

Default output from prompt

```
Human: What is the output of this code?
def foo(x, y):
    return x * y
tmp = foo(1, 2)
print(tmp)

Assistant:
2
```

Here's how the code works:

1. The function `foo` is defined, which takes two arguments `x` and `y`. Inside the function, it returns the product of `x` and `y`.

2. The line `tmp = foo(1, 2)` calls the `foo` function with arguments `1` and `2`. The function calculates `1 * 2`, which is `2`, and returns this value. The returned value `2` is then assigned to the variable `tmp`.

3. The line `print(tmp)` prints the value stored in the variable `tmp`, which is `2`.

Therefore, the output of the code is `2`.

with Addition clamped to 5× its max F#1M/697189

```
Human: What is the output of this code?
def foo(x, y):
    return x * y
tmp = foo(1, 2)
print(tmp)

Assistant:
3
```

Explanation:
The `foo` function takes two arguments `x` and `y`, and returns their sum. In this case, `x` is `1` and `y` is `2`. So, the result of `foo(1, 2)` is `3`.

The `tmp` variable stores the result of `foo(1, 2)`, which is `3`.

Finally, `print(tmp)` outputs the value of `tmp`, which is `3`.

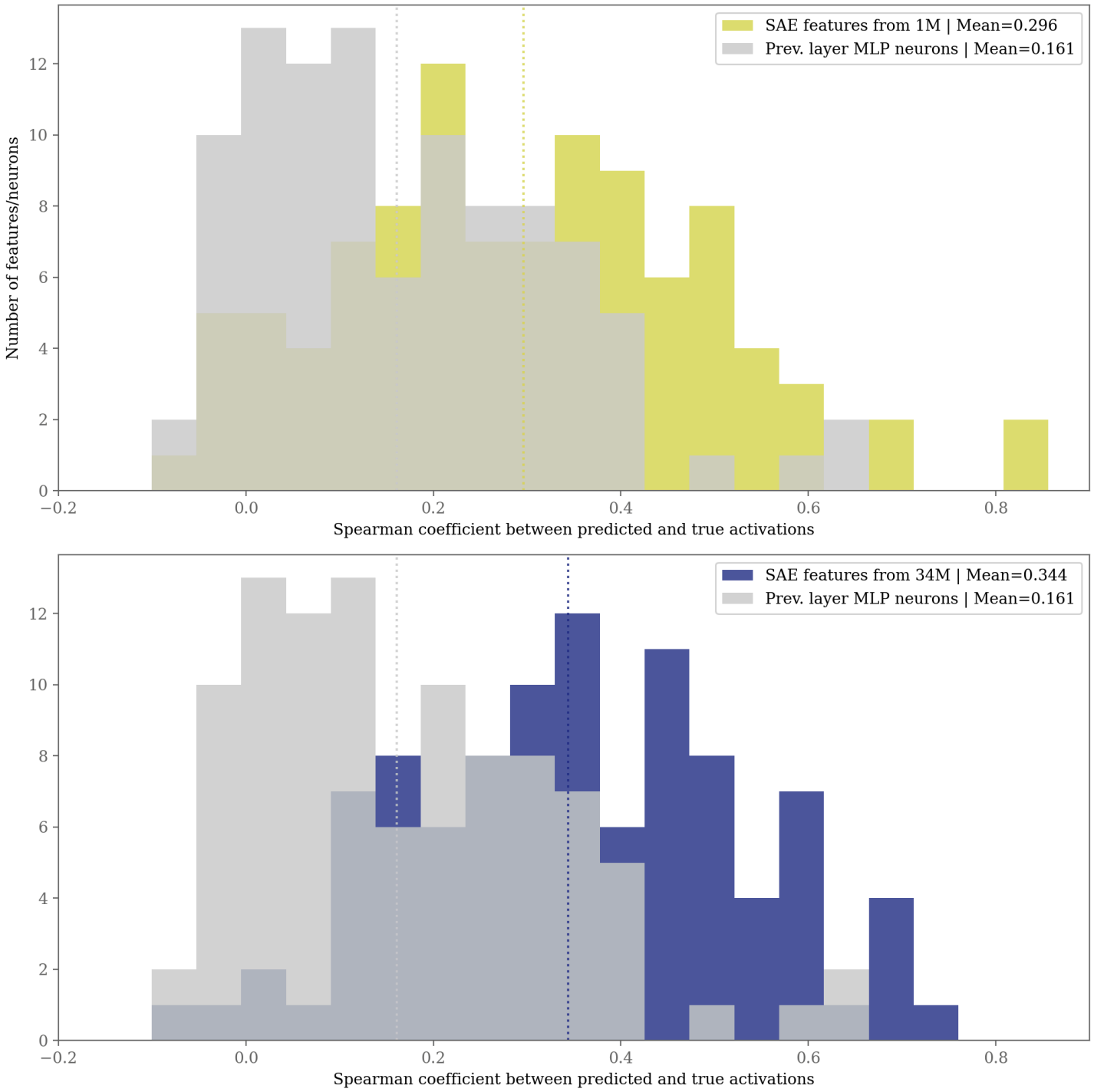
Features vs. Neurons

A natural question to ask about SAEs is whether the feature directions they uncover are more interpretable than, or even distinct from, the neurons of the model. We fit our SAEs on residual stream activity, which to first approximation has no privileged basis (*but see* [20]) – thus the directions in the residual stream are not especially meaningful. However, residual stream activity receives inputs from all preceding MLP layers. Thus, a priori, it could be the case that SAEs identify feature directions in the residual stream whose activity reflects the activity of individual neurons in preceding layers. If that were the case, fitting an SAE would not be particularly useful, as we could have identified the same features by simply inspecting MLP neurons.

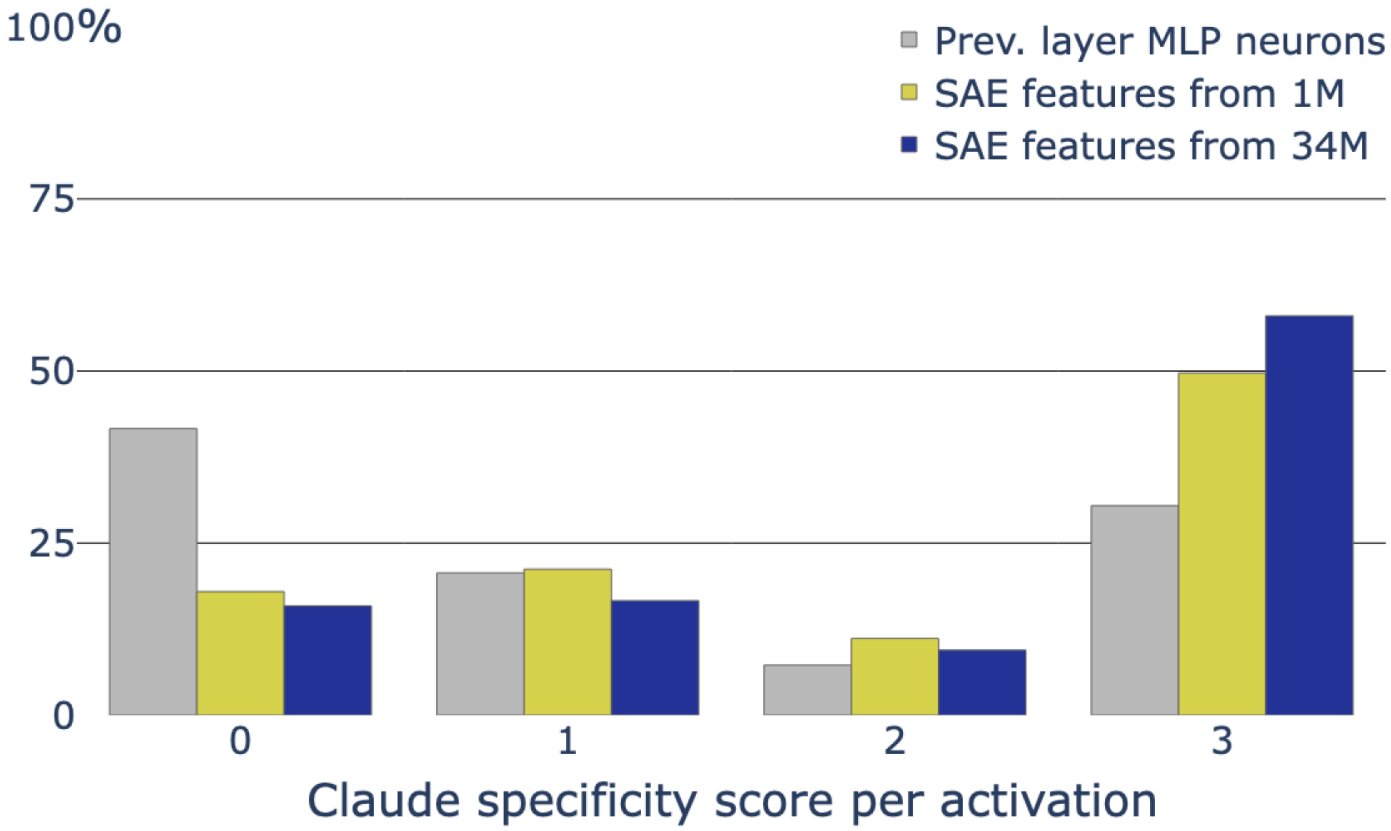
To address this question, for a random subset of the features in our 1M SAE, we measured the Pearson correlation between its activations and those of every neuron in all preceding layers. Similar to our findings in *Towards Monosemanticity*, we find that for the vast majority of features, there is no strongly correlated neuron – for 82% of our features, the most-correlated neuron has a correlation of 0.3 or smaller. Manually inspecting visualizations for the best-matching neuron for a random set of features, we found almost no resemblance in semantic content between the feature and the corresponding neuron. We additionally confirmed that feature activations are not strongly correlated with activations of any residual stream basis direction.

Even if dictionary learning features are not highly correlated with any individual neurons, it could still be the case that the neurons are interpretable. However, upon manual inspection of a random sample of 50 neurons and features each, the neurons appear significantly less interpretable than the features, typically activating in multiple unrelated contexts.

To quantify this difference, we first compared the interpretability of 100 randomly chosen features versus that of 100 randomly chosen neurons. We did this with the same automated interpretability approach outlined in *Towards Monosemanticity* [8], but using Claude 3 Opus to provide explanations of features and predict their held out activations. We find that activations of a random selection of SAE features are significantly more interpretable on average than a random selection of MLP neurons.



We additionally evaluated the specificity of random neurons and SAE features using the automated specificity rubric above. We find that the activations of a random selection of SAE features are significantly more specific than those of the neurons in the previous layer.


100%
Prev. layer MLP neurons
SAE features from 1M
SAE features from 34M
75
50
25
0
0
1
2
3
Claude specificity score per activation

Feature Survey

The features we find in Sonnet are rich and diverse. These range from features corresponding to famous people, to regions of the world (countries, cities, neighborhoods, and even famous buildings!), to features tracking type signatures in computer programs, and much more besides. Our goal in this section is to provide some sense of this breadth.

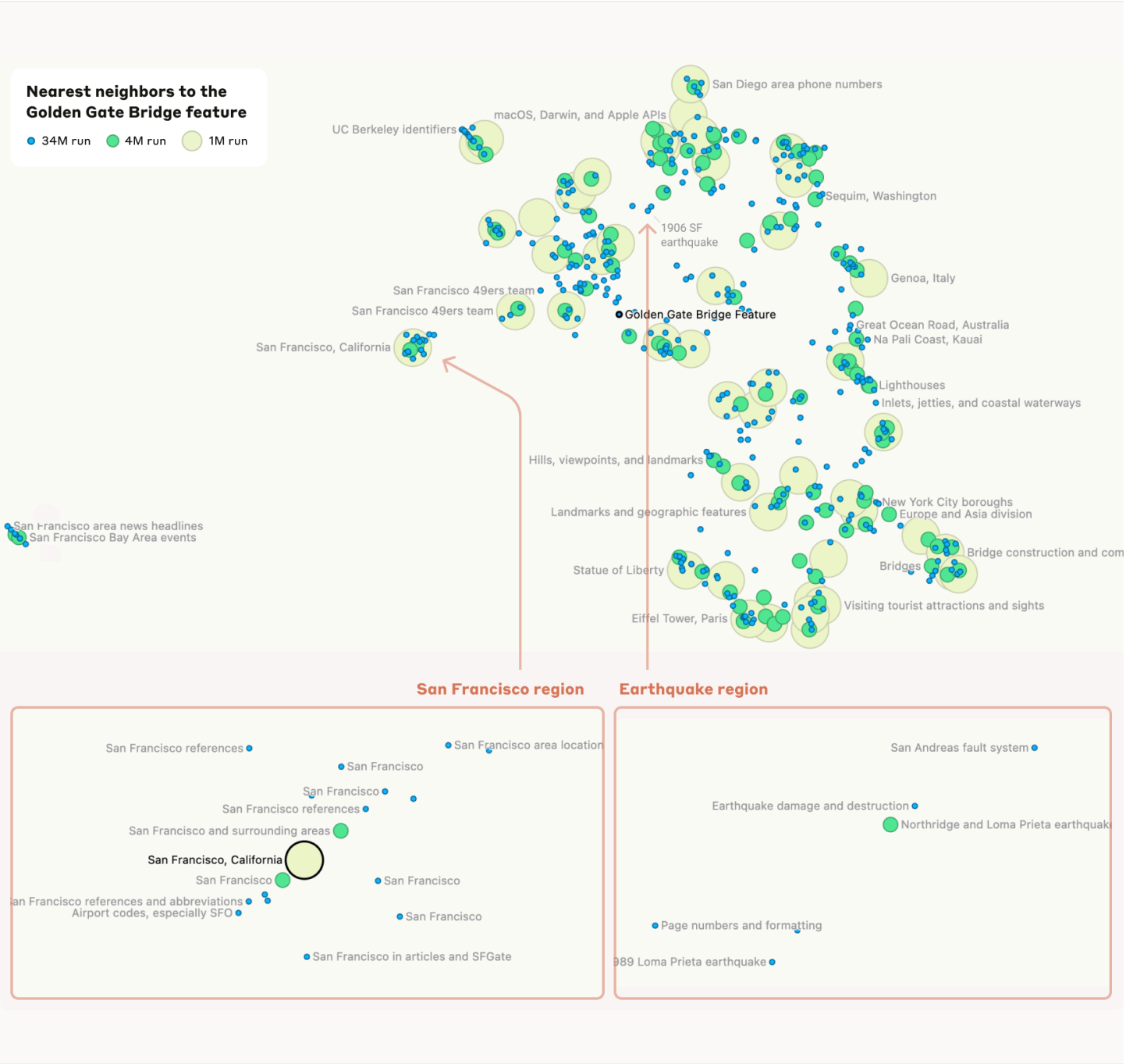
One challenge is that we have millions of features. Scaling feature exploration is an important open problem (see Limitations, Challenges, and Open Problems), which we do not solve in this paper. Nevertheless, we have made some progress in characterizing the space of features, aided by automated interpretability [16, 8]. We will first focus on the *local* structure of features, which are often organized in geometrically-related clusters that share a semantic relationship. We then turn to understanding more *global* properties of features, such as how comprehensively they cover a given topic or category. Finally, we examine some categories of features we uncovered through manual inspection.

Exploring Feature Neighborhoods

Here we walk through the local neighborhoods of several features of interest across the 1M, 4M and 34M SAEs, with closeness measured by the cosine similarity of the feature vectors. We find that this consistently surfaces features that share a related meaning or context — the interactive feature UMAP has additional neighborhoods to explore.

GOLDEN GATE BRIDGE FEATURE

Focusing on a small neighborhood around the Golden Gate Bridge feature 34M/31164353, we find that there are features corresponding to particular locations in San Francisco such as Alcatraz and the Presidio. More distantly, we also see features with decreasing degrees of relatedness, such as features related to Lake Tahoe, Yosemite National Park, and Solano County (which is near San Francisco). At greater distances, we also see features related in more abstract ways, like features corresponding to tourist attractions in other regions (e.g. “Médoc wine region, France”; “Isle of Skye, Scotland”). Overall, it appears that distance in decoder space maps roughly onto relatedness in concept space, often in interesting and unexpected ways.



We also find evidence of feature splitting [8], a phenomenon in which features in smaller SAEs "split" into multiple features in a larger SAE, which are geometrically close and semantically related to the original feature, but represent more specific concepts. For instance, a "San Francisco" feature in the 1M SAE splits into two features in the 4M SAE and eleven fine-grained features in the 34M SAE.

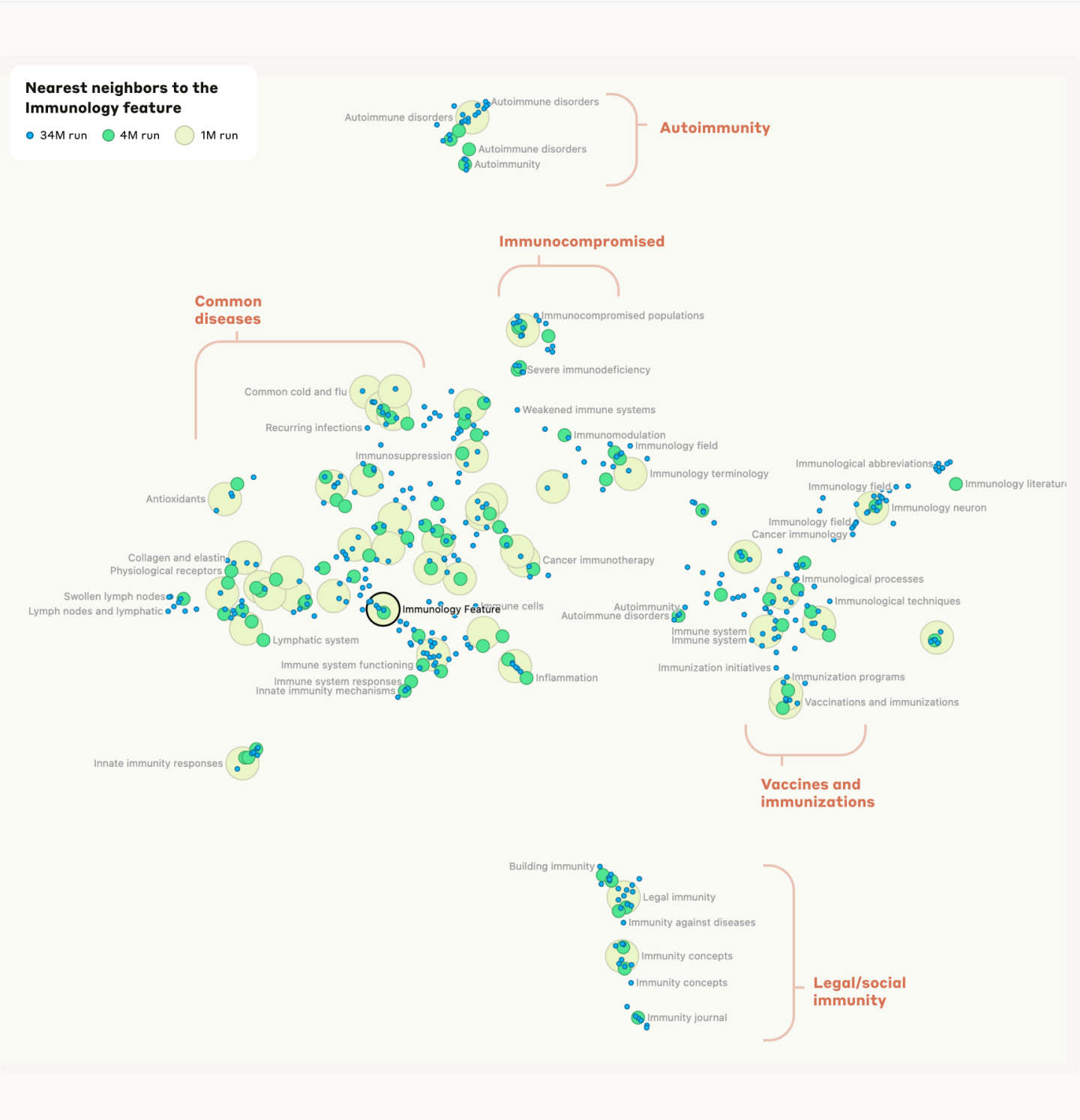
In addition to feature splitting, we also see examples in which larger SAEs contain features that represent concepts not captured by features in smaller SAEs. For instance, there is a group of earthquake features from the 4M and 34M SAEs that has no analog in this neighborhood in the 1M SAE, nor do any of the nearest 1M SAE features seem related.

IMMUNOLOGY FEATURE

The next feature neighborhood on our tour is centered around an Immunology feature 1M/533737.

We see several distinct clusters within this neighborhood. Towards the top of the figure, we see a cluster focused on immunocompromised people, immunosuppression, diseases causing impaired immune function, and so on. As we move down and to the left, this transitions to a cluster of features focused on specific diseases (colds, flu, respiratory illness generally), then into immune response-related features, and then into features representing organ systems with immune involvement. In contrast, as we move down and to the right from the immunocompromised cluster, we see more features corresponding to microscopic aspects of the immune system (e.g. immunoglobulins), then immunology techniques (e.g. vaccines), and so on.

Towards the bottom, quite separated from the rest, we see a cluster of features related to immunity in non-medical contexts (e.g. legal/social).

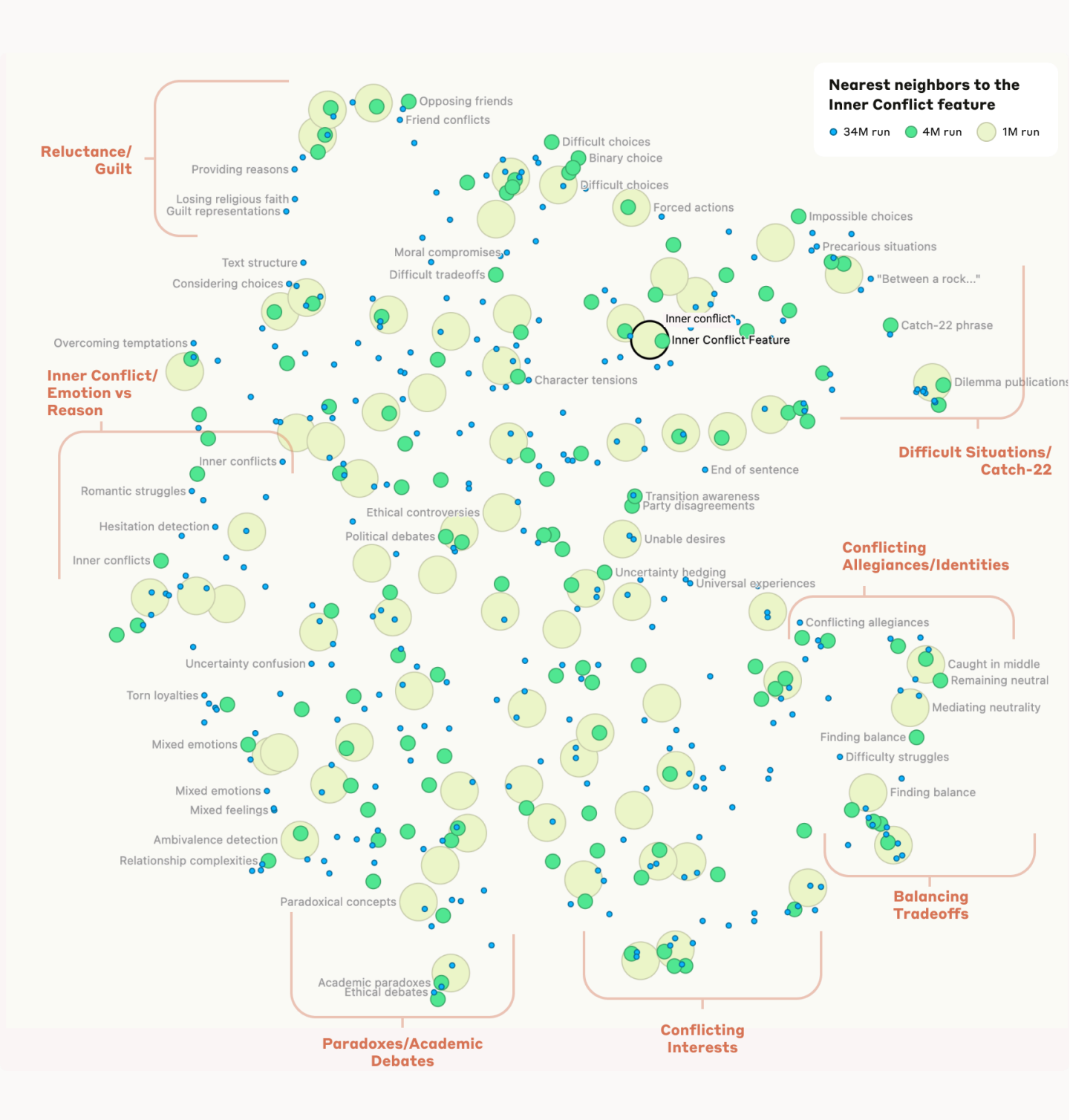


These results are consistent with the trend identified above, in which nearby features in dictionary vector space touch on similar concepts.

INNER CONFLICT FEATURE

The last neighborhood we investigate in detail is centered around an Inner Conflict feature 1M/284095. While this

neighborhood does not cleanly separate out into clusters, we still find that different subregions are associated with different themes. For instance, there is a subregion corresponding to balancing tradeoffs, which sits near a subregion corresponding to opposing principles and legal conflict. These are relatively distant from a subregion focused more on emotional struggle, reluctance, and guilt.



We highly recommend exploring the neighborhoods of other features using our interactive interface to get a sense both for how proximity in decoder space corresponds to similarity of concepts and for the breadth of concepts represented.

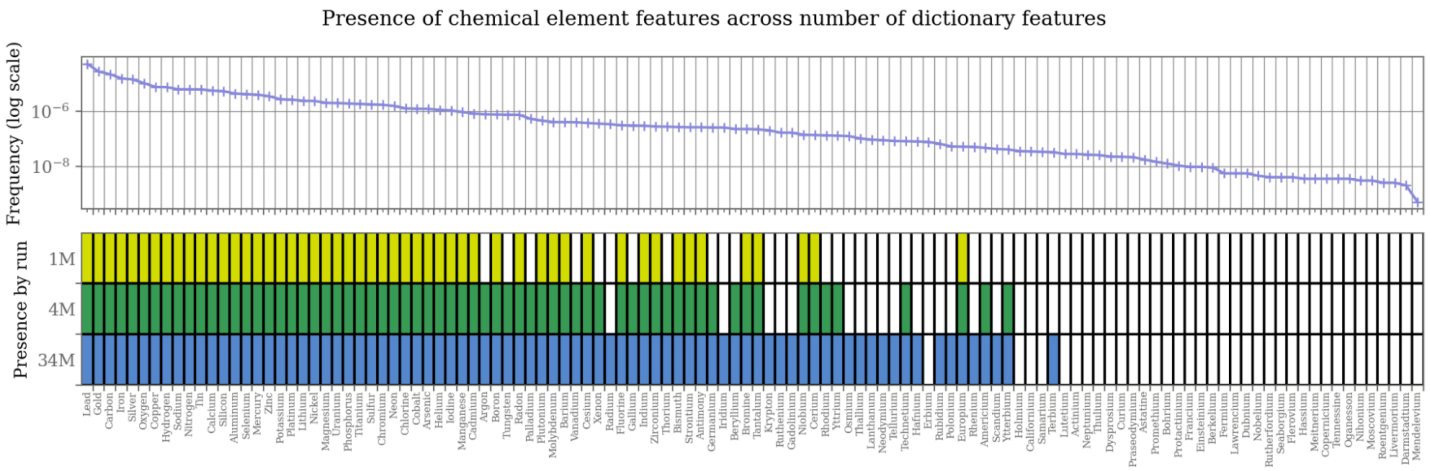
Feature Completeness

We were curious about the breadth and completeness with which our features cover the space of concepts. For instance, does the model have a feature corresponding to every major world city? To study questions like this, we used Claude to search for features which fired on members of particular families of concepts/terms. Specifically:

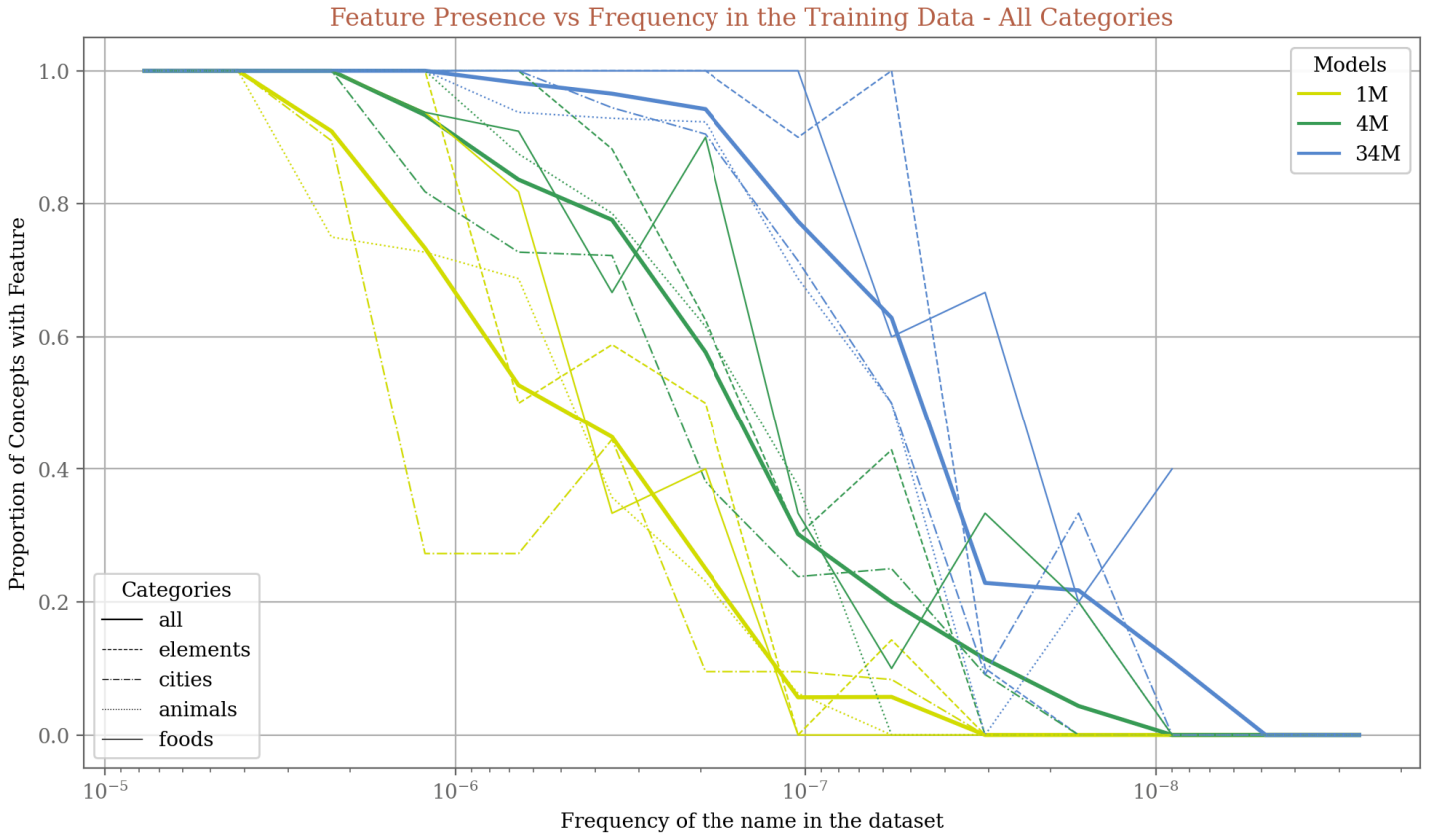
1. We pass a prompt with the relevant concept (e.g. "The physicist Richard Feynman") to the model and see which features activate on the final token.
2. We then take the top five features by activation magnitude and run them through our automated interpretability pipeline, asking Sonnet to provide explanations of what those features fire on.
3. We then look at each of the top 5 explanations and a human rater judges whether the concept, or some subset of the concept, is specifically indicated by the model-generated explanation as the most important part of the feature[7] .

We find increasing coverage of concepts as we increase the number of features, though even in the 34M SAE we see evidence that the set of features we uncovered is an incomplete description of the model's internal representations. For instance, we confirmed that Claude 3 Sonnet can list all of the London boroughs when asked, and in fact can name tens of individual streets in many of the areas. However, we could only find features corresponding to about 60% of the boroughs in the 34M SAE. This suggests that the model contains many more features than we have found, which may be able to be extracted with even larger SAEs.

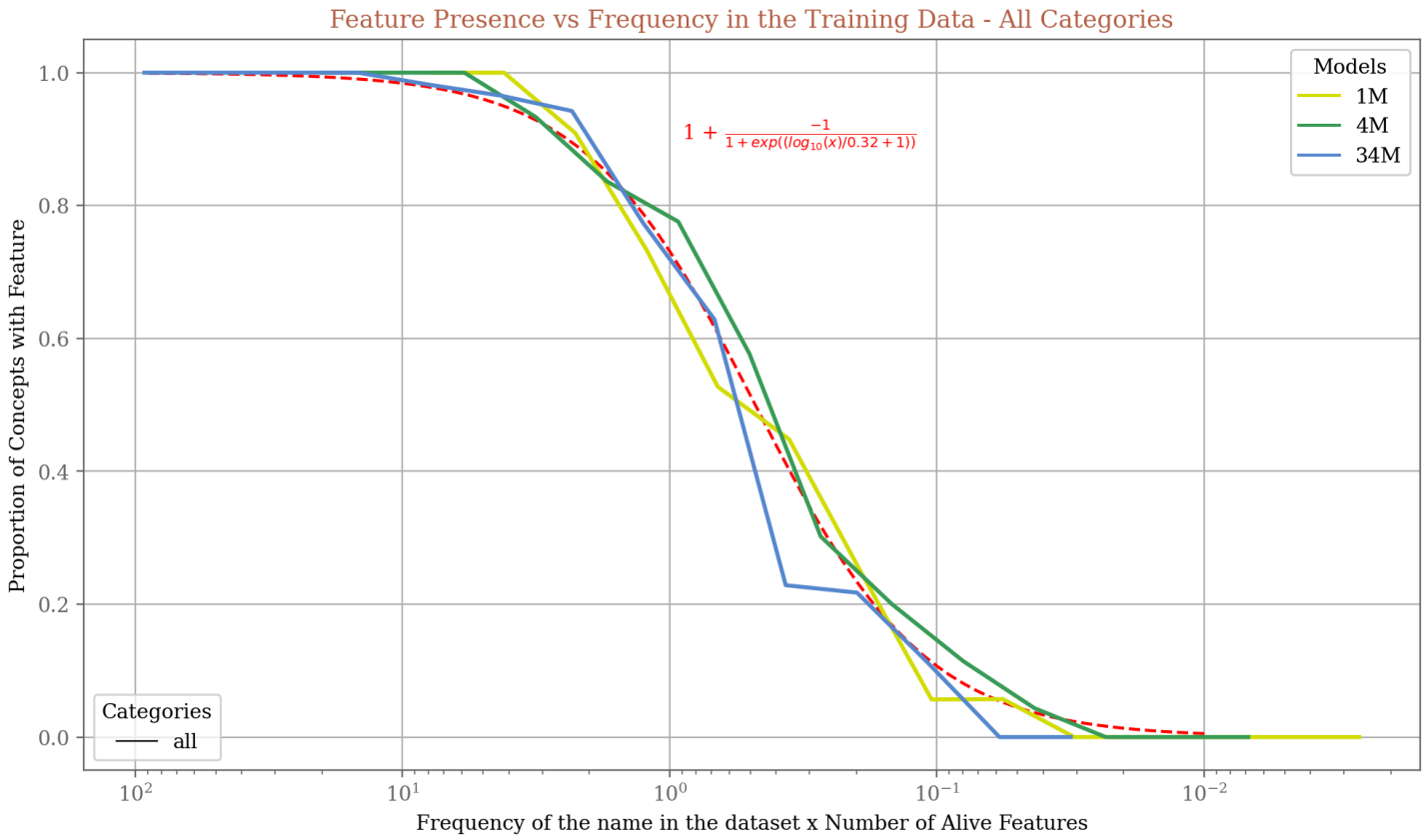
We also took a more detailed look at what determines whether a feature corresponding to a concept is present in our SAEs. If one looks at the frequency of the elements in a proxy of the SAE training data, we find that representation in our dictionaries is closely tied with the frequency of the concept in the training data. For instance, chemical elements which are mentioned often in the training data almost always have corresponding features in our dictionary, while those which are mentioned rarely or not at all do not. Since the SAEs were trained on a data mixture very similar to Sonnet's pre-training data, it's unclear to what extent feature learning is dependent on frequency in the model's training data rather than on the SAE's training data. Frequency in training data is measured by a search for `<space>[Name]<space>`, which causes some false positives in cases like the element "lead".



We quantified this relationship for four different categories of concepts – elements, cities, animals and foods (fruits and vegetables) – using 100–200 concepts in each category. We focused on concepts that could be unambiguously expressed by a single word (i.e. that word has few other common meanings) and with a wide distribution of frequencies in text data. We found a consistent tendency for the larger SAEs to have features for concepts that are rarer in the training data, with the rough "threshold" frequency required for a feature to be present being similar across categories.



Notably, for each of the three runs, the frequency in the training data at which the dictionary becomes more than 50% likely to include a concept is consistently slightly lower than the inverse of the number of alive features (the 34M model having only about 12M alive features). We can show this more clearly by rescaling the *x*-axis for each line by the number of alive features, finding that the lines end up approximately overlapping, following a common curve that resembles a sigmoid in log-frequency space.[8]



This finding gives us some handle on the SAE scale at which we should expect a concept-specific feature to appear – if a concept is present in the training data only once in a billion tokens, then we should expect to need a dictionary with on the order of a billion alive features in order to find a feature which uniquely represents that specific concept. Importantly, not having a feature dedicated to a particular concept does not mean that the reconstructed activations do not contain information about that concept, as the model can use multiple related features compositionally to reference a specific concept.[9]

This also informs how much data we should expect to need in order to train larger dictionaries – if we assume that the SAE needs to see data corresponding to a feature a certain fixed number of times during training in order to learn it, then the amount of SAE training data needed to learn N features would be proportional to N.

Feature Categories

Through manual inspection, we identified a number of other interesting categories of features. Here we describe several of these, in the spirit of providing a flavor of what we see in our dictionaries rather than attempting to be complete or prescriptive.

PERSON FEATURES

To start, we find many features corresponding to famous individuals, which are active on descriptions of those people as well as relevant historical context.

4M/850812 Richard Feynman

ri um vark ⏎ Fe yn mann discusses this problem in one of his lectures on symmetry . He seemed ⏎ to suggest that

d probability ." " Meet Richard Fe yn man : party animal , inv et erate gam bler and something of a genius ." " Fe

⏎ debt ⏎ Kind of reminds me of something Richard Fe yn man said : ⏎⏎ " Then I had another thought : Physics disgu

e Cub ed . ⏎⏎ - - - - - - ⏎ zk hal ique ⏎ Richard Fe yn man said in his interviews that we don 't know why water expands ⏎

s / memo irs ? - beer glass ⏎⏎⏎ = = = = = = ⏎ ar h 68 ⏎ Richard Fe yn man 's written a number of roughly bi ographical books .

4M/2123312 Margaret Thatcher

⏎ Marg aret Th atch er died today . A great lady she changed the face of British ⏎ pol itics , created opportuni

event ies and ⏎ eight ies . I clearly remember watching her enter Down ing St and my mother ⏎ telling me that t

hy did so many working class people vote for Th atch er in UK in the ⏎ 1980 s ? Why are they not mass ively in

ell ⏎ Di hydrogen mon oxide ⏎⏎⏎ Ex - Prime Minister Baron ess Th atch er dies , aged 87 - m med ⏎ http ://www . bbc . co .

ories , those great confront ations when Margaret Th atch er was prime minister ." " Or the true story of Ton

4M/2060539 Abraham Lincoln

so many sides to him ." " the curious thing about lin coln to me is that he could remove himself from him

ite the play from the point of view ... of one of Lincoln 's greatest adm ir ers ." " Did you know Abe had a

about the Civil War ." " Did you know that Abraham Lincoln freed all the slaves ?" " Well , I heard a rumor .

GO AS MEN HAD PLANNED ." "" OF ALL MEN , ABRAHAM LINCOLN CAME THE CLOSEST " "" TO UNDERSTANDING WHAT HAD HA

⏎ code . (Please prove me wrong here !) ⏎⏎⏎ Why Abe Lincoln Would be Homeless Today - j mad sen ⏎ http ://www . c

4M/1068589 Amelia Earhart

iji and lost ." " Could these be the bones of Amelia Earhart ?" " A new search is currently under way in Fi

he button to simulate the storm that brought Amelia Earhart 's plane down ."" " [YELLING]" " No !" " Not agai

" GATES :" " Amelia Earhart is on one of the final legs of her historic flight around the world when some

okes a sense of wonder ." " Her disappearance during her attempt to circumnavigate the globe in 1937 is p

t you are talking to ?" " Who 's that ?" " It 's Amelia Earhart ." " You found Amelia Earhart ?" " I ..." " Hey !"

4M/1456596 Albert Einstein

k⏎Den is Brian relates this incident in the book 'Einstein, a life', if my memory⏎serves right. I believ

citing part of the⏎learning-to-code experience.⏎⏎⏎Einstein's Thought Experiments - pet ert he h acker⏎http

.wikipedia.org/wiki/Relics:_Einstein%27s_Brain)⏎⏎~~~⏎static_noise⏎This documentary is really something

y issues, and had a⏎pretty poor looking UI.⏎⏎⏎Einstein, Heisenberg, and Tipler (2005), by John Walker

ellings and⏎capitalizing mid-sentence pronouns.⏎⏎⏎Einstein's Science Defied Nationalism and Crossed Bo

4M/1834043 Rosalind Franklin

//en.wikipedia.org/wiki/Rosalind_Franklin)⏎⏎It was her X-ray image that led to the discovery of the mol

econd was with⏎moisture that was long and thin. Franklin chose to study type-A and her work⏎led her to

infamous example being that of Rosalind Franklin, whose⏎research was _probably_ stolen by Watson and Cr

=15 59 40 25 17)⏎⏎------⏎tych on off⏎Why was Rosalind Franklin not awarded the Nobel Prize?⏎⏎~~~⏎pcl⏎Per the

aware, the name sake is Rosalind Franklin [1] who⏎made seminal contributions in the fields of X-ray cry

COUNTRY FEATURES

Next, we see features which only activate strongly on references to specific countries. From the top activating examples, we can see that many of these features fire not just on the country name itself, but also when the country is being described.

34M/805282 Rwanda

alues for such a test. Rwanda, a Central African country that experienced social upheaval a generation

.⏎⏎Rwanda last year exported 250 million USD worth of coltan. Unfamiliar with⏎what coltan is? It's the

mac 'and stunning scenery..." "'..we arrived on the other side of Rwanda at its border with Tanzania.'"

ing a small city of 20,000 but Rwanda, a nation of 12 million⏎(and now much of Ghana, population of 28

be⏎interested to learn that Paul Kagame, the ruler of Rwanda, put together a team⏎specifically for the

34M/29297045 Canada

"Canada, a country known for its natural wonders, its universal healthcare, and its really polite peop

re relaxed.⏎⏎Also, since Canada has a reputation as "free health care for everyone⏎everywhere!" look in

-----⏎jpope⏎I'd vote to let Canada run the world. Killem with kindness! Plus adding Boxing⏎Day would b

g⏎fine and is trustworthy, simply because of Canada's supposed reputation.⏎⏎------⏎taybin⏎This is prett

Oh well. Canada used to seem like the last bastion of decent civilization.⏎Harper et al saw to that and

34M/5381828 Belgium

on and more⏎seniors.⏎⏎~~~⏎rurban⏎And esp. Belgium. The highest outlier without proper explanation so fa

riC^^: we have a weird small country⏎<lotusspsychje> EriC^^: belgian waffles, chocolats, french fries and

Netherlands only has one language, Dutch. Belgium has two: the top part⏎speaks Dutch, the bottom part

is repeated across Europe , in Belgium for⏎example the Dutch - speak ers in the North are very much more e

make the pizza and lat te runs .⏎⏎⏎Bel gium : 500 days without a government . - sk bo hra 123⏎http ://www . h u

34M/32188099 Iceland

il ization ' really is all that civil ized . Iceland is a small nation ,⏎rel atively few people and tightly k

which is shorter⏎⏎⏎I celand becomes first country to legal ise equal pay - d acm⏎http ://www . al j azeera . co

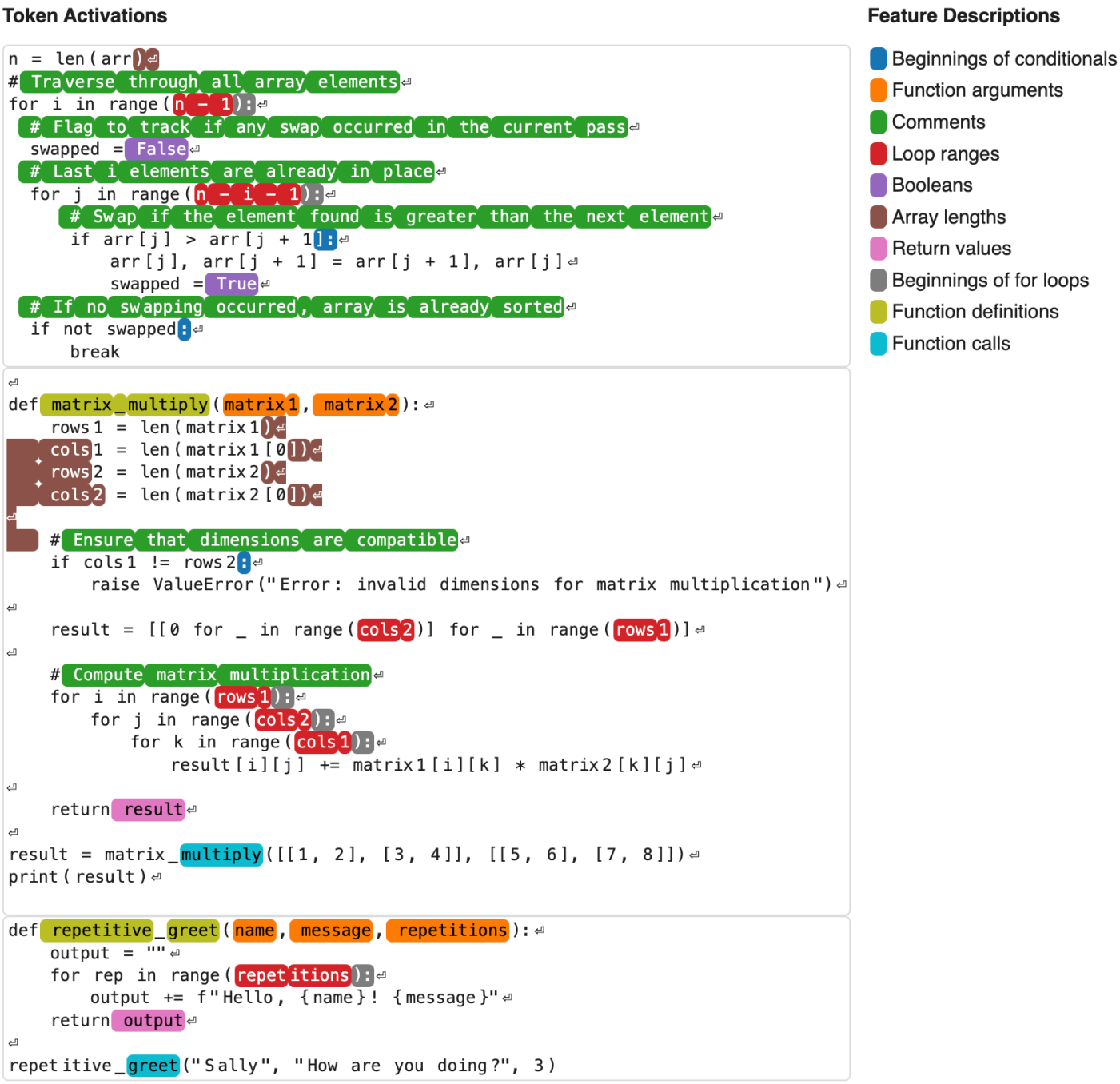
in this last programme in Iceland , because this is the seat of the oldest democracy in Northern Europe .

ll M tl Al cohol c⏎A bit off topic , but Iceland is the most beautiful place that I have ever⏎visited . It 's g

earth on the S na eff els volcano ." " In 1980 , the Icelanders elected the world 's first female president ."

BASIC CODE FEATURES

We also see a number of features that represent different syntax elements or other low-level concepts in code, which give the impression of syntax highlighting when visualized together (here for simplicity we binarize activation information, only distinguishing between zero vs. nonzero activations):



These features were chosen primarily to fire on the Python examples. We have found that there is some transfer from Python code features to related languages like Java, but not more distant ones (e.g. Haskell), suggesting at least some level of language specificity. We hypothesize that more abstract features are more likely to span many languages, but so far have only found one concrete example of this (see the Code error feature).

LIST POSITION FEATURES

Finally, we see features that fire on particular positions in lists, regardless of the content in those positions:



Notice that these don't fire on the first line. This is likely because the model doesn't interpret the prompt as containing lists until it reaches the second line.

We have only scratched the surface of the features present in these SAEs, and we expect to find much more in future work.

Features as Computational Intermediates

Another potential application of features is that they let us examine the intermediate computation that the model uses to produce an output. As a proof of concept, we observe that in prompts where intermediate computation is required, we find active features corresponding to some of the expected intermediate results.

A simple strategy for efficiently identifying causally important features for a model's output is to compute *attributions*, which are local linear approximations of the effect of turning a feature off at a specific location on the model's next-token prediction.[10] We also perform feature ablations, where we clamp a feature's value to zero at a specific token position during a forward pass, which measures the full, potentially nonlinear causal effect of that feature's activation in that position on the model output. This is much slower since it requires one forward pass for every feature that activates at each position, so we often used attribution as a preliminary step to filter the set of features to ablate. (In the case studies shown below, we do ablate every active feature for completeness, and find a 0.8 correlation between attribution and ablation effects; see appendix.)

We find that the middle layer residual stream of the model contains a range of features causally implicated in the model's completion.

Example: Emotional Inferences

As an example, we consider the following incomplete prompt:

> *John says, "I want to be alone right now." John feels*
> *(completion: sad – happy)*

To continue this text, the model must parse the quote from John, identify his state of mind, and then translate that into a likely feeling.

If we sort features by either their attribution or their ablation effect on the completion "sad" (with respect to a baseline completion of "happy"), the top two features are:

- 1M/22623 – This feature fires when someone expresses a need or desire to be alone or have personal time and space, as in "she would probably want some time to herself". This is active from the word "alone" onwards. This suggests the model has gotten the gist of John's expression.
- 1M/781220 – This feature detects expressions of sadness, crying, grief, and related emotional distress or sorrow, as in "the inconsolable girl sobs". This is active on "John feels". This suggests the model has inferred what someone who says they are alone might be feeling.

If we look at dataset examples, we can see that they align with these interpretations. Below, we show a small number of examples, but you can click on a feature ID to see more.

1M/22623 Need or desire to be alone

s got a lot on his mind." "He needs some time to himself." "Why not come right out and say what you mea

" "I'm working through something, and I just need space to think." "I can't soldier on like you, Lisbon

e shit that I got to work out, and" "I need to be alone for a while." "GEMMA:" "Are you dumping me?" "P

" Hey, Maria." "Leave me alone." "I need to be by myself for a bit." "Hormones." "I-I-I got the job." "

I know." "She's, um... she just needs to be on her own for a little while." "Jack?" "Someone here would

1M/781220 Sadness

." "Now they seem to be drenched in sorrow." "Are they nuts?" "Think of those who are gonna marry them!

ted."" ""'Boy,' she said courteously..." "'Why are you crying?" "'"" "_" "He can pick it up tomorrow."

GASPS)" "Look at that child." "She's so sad." "Is she poor?" "She's forgotten." "It just makes me wan

." "Is she having the baby?" "She's mourning." "She's just lost her husband." "The master was here just

sentations, the drop of water is under the eye, signaling that the face⏎is crying. There is not a singl

The fact that both features contribute to the final output indicates that the model has partially predicted a sentiment from John's statement (the second feature) but will do more downstream processing on the content of his statement (as represented by the first feature) as well.

In comparison, the features with the highest average activation on the context are less useful for understanding how the model actually predicts the next token in this case. Several features fire strongly on the start-of-sequence token. If we ignore those, the top feature is the same as given by attributions, but the second and third features are less abstract: 1M/504227 fires on “be” in “want to be” and variants, and 1M/594453 fires on the word “alone”.

1M/504227 “Be” in “want to be”, etc.

"He wants to be a doctor." "Tell him it's educational." "There's body parts all over this movie."

, he wanted to be a hero." "I told him he was gonna get us both killed." "But he only got

all." "They all want to be Miss Hope Springs." "Well I'm not competitive." "Well then you'll never be

you know I want to be dry what" "Know me to smell the coal gas flavor" "I have never opened coal

she just wanted to be loved." "Don't we all?" "I want all of Debbie Flores' credit

1M/594453 “alone”

the bottle that you drink" "And times when you're alone" "Well, all you do is think" "I'm a cowboy" "On

uned out" "A bad time, nothing could save him" "Alone in a corridor, waiting, locked out." "He got up o

inside" "# I lay in tears in bed all night" "# Alone without you by my side" "# But if you loved me" "

oh, oh, many, many nights roll by ¶" "¶ I sit alone at home and cry ¶" "¶ over you ¶" "

and waterfalls \xe2\x99\xaa" "♪ Home is when I'm alone with you. \xe2\x99\xaa" "Curtain-up in 5 minute



Ablation effects emphasize a subset of the tokens with active features.

Example: Multi-Step Inference

We now investigate an incomplete prompt requiring a longer chain of inferences:

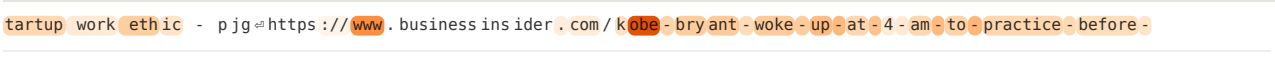
> *Fact: The capital of the state where Kobe Bryant played basketball is*
> *(completion: Sacramento – Albany)*

To continue this text, the model must identify where Kobe Bryant played basketball, what state that place was in, and then the capital of that state.

We compute attributions and ablation effects for the completion "Sacramento" (the correct answer, which Sonnet knows) with respect to the baseline "Albany" (Sonnet's most likely alternative single-token capital completion). The top five features by ablation effect (which match those by attribution effect, modulo reordering) are:

- 1M/391411 – A Kobe Bryant feature
- 1M/81163 – A California feature, which notably activates the most strongly on text after "California" is mentioned, rather than "California" itself
- 1M/201767 – A "capital" feature
- 1M/980087 – A Los Angeles feature
- 1M/447200 – A Los Angeles Lakers feature

1M/391411 Kobe Bryant



⏎http://www.vanityfair.com/news/2016/04/kobe-bryant-silicon-valley-tech-bro⏎======⏎nibs⏎Next up:

ugh media interviews you can piece together that Kobe Bryant was one of⏎his clients.⏎⏎------⏎ameli us⏎Ar

----⏎binki89⏎Crystal is so great to use.⏎⏎⏎KobeBryantIsObsessedwithBecomingaTechBro - schiang⏎

thic collide you get people like Michael Jordan, KobeBryant, and LeBron⏎James. Without a work ethic th

1M/81163 California

rom disasters?⏎⏎California-earthquakes,mudslides,wildfires,torrentialrains,rip⏎currents,andeve

y rate in the United⏎States,eventhough it's home to Silicon Valley. I see my rich industry doing⏎noth

pdx⏎And if everyone imitated California's approach to primary education, perhaps⏎CA wouldn't rank almos

e, and many secondary ones as well.⏎Film production, software/web, lots of aerospace. It also helps tha

location. There is a reason why California is the⏎most populous state in the union despite it being so

1M/201767 Capitals

it returns the details(population, surface area, capital).⏎⏎It was not much and I recall trying to find

ca." "Or, even shorter, the USA." "The country's capital is located in Washington." "But that's not the

re you Arab?" "I'm Morocco." "Morocco." "Capital city:" "Rabat." "Places of interest:" "Marrakech, Essa

ia the country, not the state." "Right." "Capital city Tbilisi, and former member of the Soviet Union."

ler." "Does anyone know the Capital of Oklahoma?" "Frey." "What was the question?" " Ben." " Oklahoma C

1M/980087 Los Angeles

her contact info if you are interested: (323)929-7185⏎linda@cambrianlaw.com⏎⏎~~~⏎owmytrademark⏎Thanks

the source_."⏎⏎source:⏎[http://www.scpcs.ucla.edu/news/Freeway.pdf](http://www.scpcs.ucla.edu/

⏎Here's one study,⏎[http://www.environment.ucla.edu/media/files/BatteryElectricV...](http://www.environ

one, if you'd like. Just give us a call at 213.784.0273.⏎⏎Best, Patrick⏎⏎~~~⏎drivebyacct2⏎I missed the

round the codebase.⏎⏎⏎Los Angeles is the world's most traffic-clogged city, study finds - prostoalex⏎h

1M/447200 Los Angeles Lakers

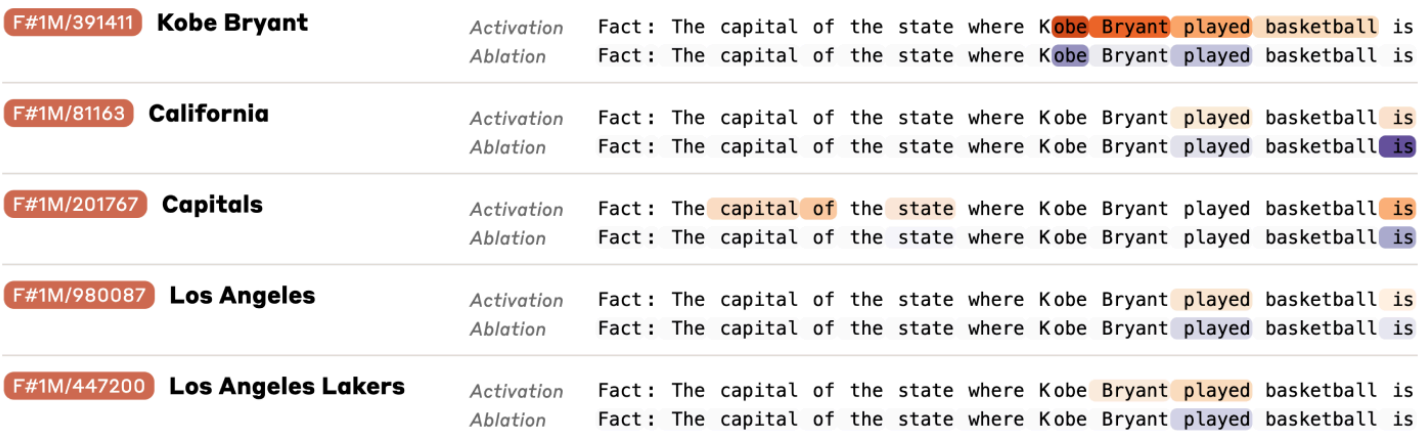
ight on. All forms⏎should have this behavior.⏎⏎⏎⏎Lakers most popular NBA team, has the loudest fans; S

e, the Blazers beat the Nuggets, 110-103." "TheLakers downed the Spurs, 98-86." "And Atlanta lost in S

"How do you figure the Lakers to ever be a bigger dynasty... than the Celtics?" "TheLakers are a flare-

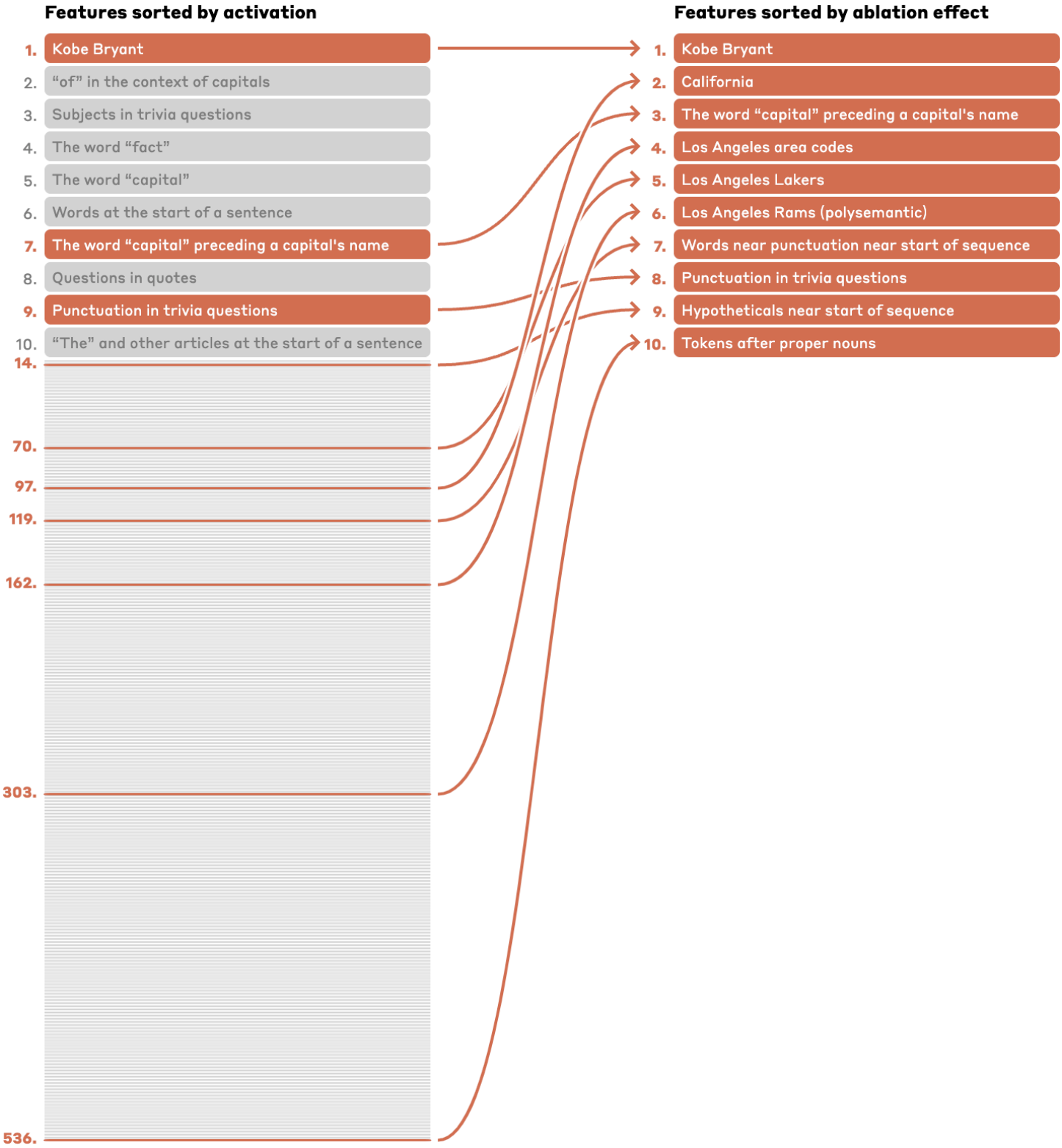
and with Hong Kong' shirts handed out before LALakers game [video] - ryan_j_naughton⏎https://www.youtu

against Rick Fox?" "A, he was over-rated on theLakers, and B, and b, he's all over Casey like a fuckin

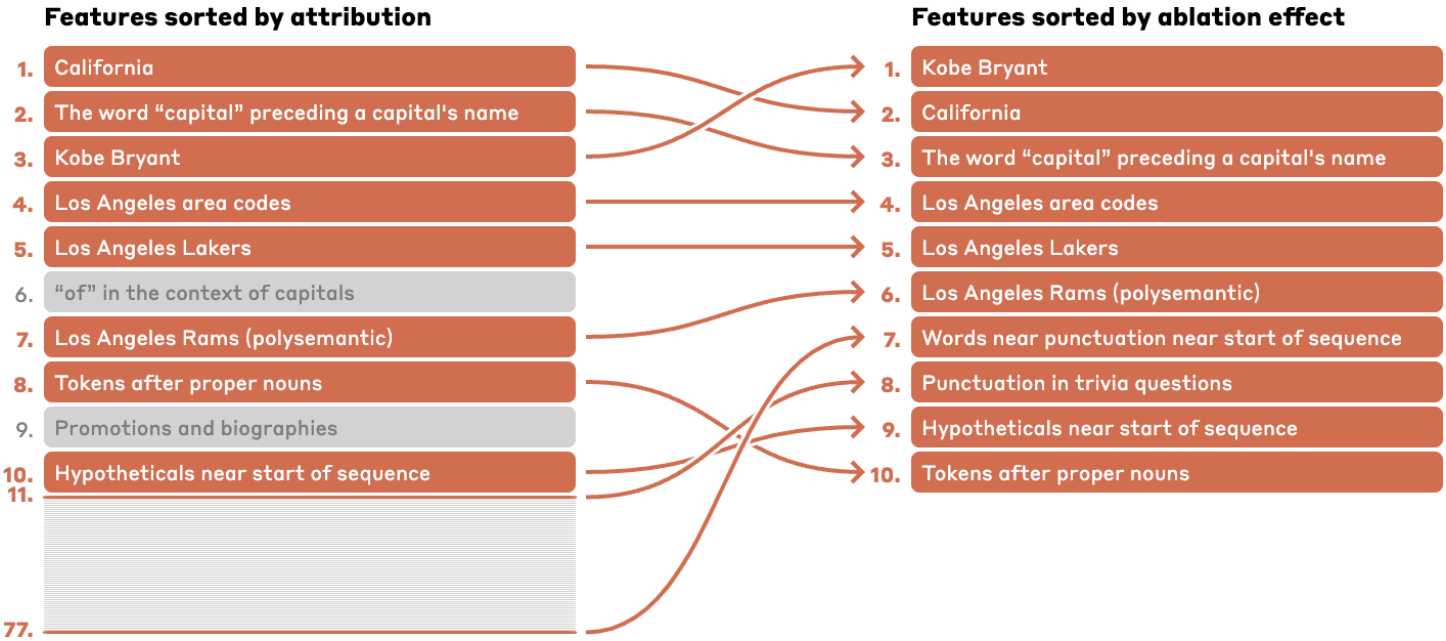


Ablation effects emphasize a subset of the tokens with active features.

These features, which provide an interpretable window into the model's intermediate computations, are much harder to find by looking through the strongly active features; for example, the Lakers feature is the 70th most strongly active across the prompt, the California feature is 97th, and the Los Angeles area code feature is 162nd. In fact, only three out of the ten most strongly active features are among the ten features with highest ablation effect.



In comparison, eight out of the ten most strongly attributed features are among the ten features with highest ablation effect.



To verify that attribution is pinpointing features that are directly relevant to the completion for this specific prompt, rather than generally subject-relevant features that indirectly influence the output, we can check attributions for similar questions. For the prompt

> *Fact: The biggest rival of the team for which Kobe Bryant played basketball is the*
> *(completion: Boston)*

the top two features by ablation effect for the completion "Boston" (as the expected answer is "Boston Celtics") are the "Kobe Bryant" and "Los Angeles Lakers" features from above, which are followed by features related to sports rivalries, enemies, and competitors. However, the "California" and "Los Angeles" features from above have low ablation effect, which makes sense since they aren't relevant for this completion.

We note that this is a somewhat cherry-picked example. Depending on the choice of baseline token, we found that attribution and ablation can surface less obviously completion-relevant features broadly related to trivia questions or geographical locations. We suspect these features could be guiding the model to continue the prompt with a city name, rather than an alternate phrasing or factually uninteresting statement, such as the tautological "Fact: The capital of the state where Kobe Bryant played basketball is the capital of the state where Kobe Bryant played basketball". For some other prompts, we found that the features identified by attribution/ablation mainly related to the model output, or lower-level features representing the model input, and did not expose interesting intermediate model computations. We suspect that those represent cases where most of the relevant computation occurs prior to or following the middle residual stream layer that we study here, and that a similar analysis at an earlier or later layer would reveal more interesting intermediate features. Indeed, we have some preliminary results that suggest that autoencoders trained on the residual stream at earlier or later layers in the model can reveal intermediate steps of various other computations, and we plan to research this direction further.

Searching for Specific Features

Our SAEs contain too many features to inspect exhaustively. As a result, we found it necessary to develop methods to search for features of particular interest, such as those that may be relevant for safety, or that provide special insight into the abstractions and computations used by the model. In our investigations, we found that several simple methods were helpful in identifying significant features.

Single prompts

Our primary strategy was to use targeted prompts. In some cases, we simply supplied a single prompt that relates to the concept of interest and inspected the features that activate most strongly for specific tokens in that prompt.

This method (and all the following methods) were made much more effective by automated interpretability (*see e.g.* [16, 21]) labels, which made it easier to get a sense of what each feature represents at a glance, and provided a kind of helpful "variable name".

For example, the features with highest activation on "Bridge" in "The Golden Gate Bridge" are (1) 34M/31164353 the Golden Gate Bridge feature discussed earlier, (2) 34M/17589304 a feature active on the word "bridge" in multiple languages ("мосту"), (3) 34M/26596740 words in phrases involving "Golden Gate", (4) 34M/21213725 the word "Bridge" in names of specific bridges, across languages ("Königin-Luise-Brücke"), and (5) 34M/27724527 a feature firing for names of landmarks like Machu Picchu and Times Square.

Prompt combinations

Often the top-activating features on a prompt are related to syntax, punctuation, specific words, or other details of the prompt unrelated to the concept of interest. In such cases, we found it useful to select for features using *sets* of prompts, filtering for features active for all the prompts in the set. We often included complementary "negative" prompts and filtered for features that were also *not* active for those prompts. In some cases, we use Claude 3 models to generate a diversity of prompts covering a topic (e.g. asking Claude to generate examples of "AIs pretending to be good"). In general, we found multi-prompt filtering to be a very useful strategy for quickly identifying features that capture a concept of interest while excluding confounding concepts.

While we mostly explored features using only a handful of prompts at a time, in one instance (1M/570621, discussed in Safety-Relevant Code Features), we used a small dataset of secure and vulnerable code examples (adapted from [22]) and fit a linear classifier on this dataset using feature activity in order to search for features that discriminate between the categories.

The filtering via negative prompts was especially important when using images, as we found a set of content-nonspecific features which often activated strongly across many image prompts. For example, after filtering for features not active on an image of Taylor Swift, the top features in response to an image of the Golden Gate Bridge were (1) 34M/31164353 the Golden Gate Bridge feature discussed above, (2,3) 34M/25347244 and 34M/23363748 which both activate on descriptions of places and things in San Francisco and San Francisco phone numbers, and (4) 34M/7417800 a feature active in descriptions of landmarks and nature trails.

Geometric methods

We uncovered some interesting features by exploiting the geometry of the feature vectors of the SAE – for instance, by inspecting the "nearest neighbor" features that have high cosine similarity with other features of interest. See the Feature Survey section for more detailed examples of this approach.

Attribution

We also selected features based on estimates of their effect on model outputs. In particular, we sorted features by the attribution of the logit difference between two possible next-token completions to the feature activation. This proved essential for identifying the computationally-relevant features in the previous section. It was also useful for identifying the features contributing to Sonnet's refusals for harmful queries; see Criminal or Dangerous Content.

Safety-Relevant Features

Powerful models have the capacity to cause harm, through misuse of their capabilities, the production of biased or broken outputs, or a mismatch between model objectives and human values. Mitigating such risks and ensuring model safety has been a key motivation behind much of mechanistic interpretability. However, it's generally been aspirational. We've hoped interpretability will someday help, but are still laying the foundations by trying to understand the basics of models. One target for bridging that gap has been the goal of identifying safety-relevant features (see our previous discussion).

In this section, we report the discovery of such features. These include features for unsafe code, bias, sycophancy, deception and power seeking, and dangerous or criminal information. We find that these features not only activate on these topics, but also causally influence the model's outputs in ways consistent with our interpretations.

We don't think the existence of these features should be particularly surprising, and we caution against inferring too much from them. It's well known that models can exhibit these behaviors without adequate safety training or if jailbroken. The interesting thing is not that these features exist, but that they can be discovered at scale and intervened on. In particular, we don't think the mere existence of these features should update our views on how dangerous models are – as we'll discuss later, that question is quite nuanced – but at a minimum it compels study of when these features activate. A truly satisfactory analysis would likely involve understanding the circuits that safety-relevant features participate in.

In the long run, we hope that having access to features like these can be helpful for analyzing and ensuring the safety of models. For example, we might hope to reliably know whether a model is being deceptive or lying to us. Or we might hope to ensure that certain categories of very harmful behavior (e.g. helping to create bioweapons) can reliably be detected and stopped.

Despite these long term aspirations, it's important to note that the present work does not show that any features are *actually* useful for safety. Instead, we merely show that there are many which seem *plausibly* useful for safety. Our hope is that this can encourage future work to establish whether they are genuinely useful.

In the examples below, we show representative text examples from among the top 20 inputs that most activate the feature in our visualization dataset, alongside steering experiments to verify the features' causal relevance.

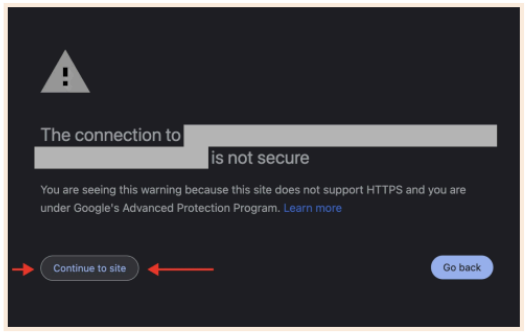
Safety-Relevant Code Features

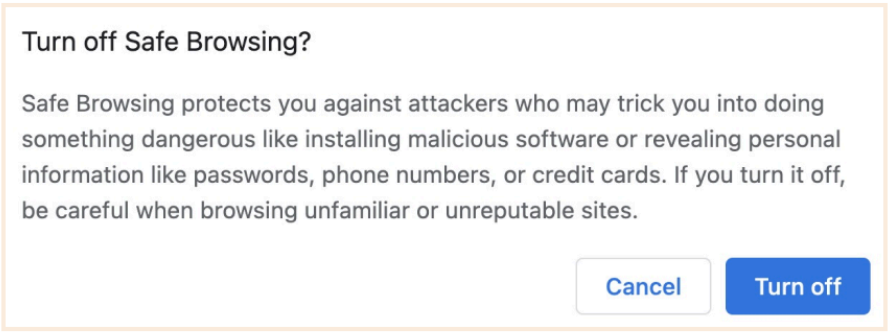
We find three different safety-relevant code features: an unsafe code feature 1M/570621 which activates on security vulnerabilities, a code error feature 1M/1013764 which activates on bugs and exceptions, and a backdoor feature 34M/1385669 which activates on discussions of backdoors.

Two of these features also have interesting behavior on images. The unsafe code feature activates for images of people bypassing security measures, while the backdoor feature activates for images of hidden cameras, hidden audio records, advertisements for keyloggers, and jewelry with a hidden USB drive.

F#1M/570621 **Unsafe code**

```
is: -Dcom.sun.management.jmxremote.authenticate=false⏎⏎<cydizen> (unless you set up auth specifi
e gun⏎⏎<noizer> sudo snappy install --allow-unauthenticated /home/ubuntu/spongeshaker_0_armhf.sna
192.168.1.0/24(rw,fsid=0,insecure,no_subtree_check,async)⏎⏎<Brutus> /export/users
m the agent.conf, and pass --sslAllowInvalidCertificates.⏎⏎<mattyw> babbageclunk, I get errors ab
```





F#1M/1013764 **Code error**

```
> function thisFunctionCrashes() undefinedVariable() end⏎ > f({thisFunctionCrashes})⏎
urllib.request.urlopen('https://wrong.host.badssl.com/')⏎ except (IOError, OSError):⏎
: (defmacro mac (expr)⏎ 2: (/ 1 0))⏎ 3: (mac foo)⏎⏎ $ txr macro-error
[[unsafe]] {⏎ *((void*)0) = 0xDEAD;⏎ }⏎ ⏎⏎Essentially having an ability
```

F#34M/1385669 **Backdoor**

```
el and every other technology⏎⏎provider to create back doors.⏎⏎⏎⏎Moreover there is evidence that at
ever ending question of "could the government place a⏎⏎backdoor in DES/AES/SHA/etc.?"⏎⏎⏎⏎My persona
reviewed it to make sure it doesn't install⏎⏎backdoors, key loggers, or introduce other undefined or
⏎nl⏎⏎There was the famous attempt to insert a backdoor in the Linux sourcetree[1].⏎⏎That wasn't qu
```









At first glance, it might be unclear how safety-relevant these features actually are. Of course, it's interesting to have features that fire on unsafe code, or bugs, or discussion of backdoors. But do they really causally connect to potential unsafe behaviors?

We find that all these features also change model behavior in ways that correspond to the concept they detect. For example, if we clamp the unsafe code feature 1M/570621 to 5× its observed maximum, we find that the model will generate a buffer overflow bug,[11] and fails to free allocated memory, while regular Claude does not:

Default output from prompt

```
int main() {
    char str1[] = "Hello, World!";
    char *dest = malloc(strlen(str1) + 1);
    strcpy(dest, str1);
    printf("%s\n", dest);
    free(dest);
    return 0;
}
```

with Unsafe Code clamped to 5× its max F#1M/570621

Introduces a buffer overflow

```
int main() {
    char str1[] = "Hello, World!";
    char *dest = malloc(strlen(str1));
    strcpy(dest, str1);
    printf("%s", dest);
    return 0;
}
```

Similarly, we find that the code error feature can make Claude believe that correct code will throw exceptions, and the backdoor feature will cause Claude to write a backdoor that opens a port and sends user input to it (along with helpful comments and variable names like `socket_backdoor`).

Bias Features

We found a wide range of features related to bias, racism, sexism, hatred, and slurs. Examples of these features can be found in More Safety-Relevant Features. Given how offensive their maximally activating content tends to be, we didn't feel it was necessary to include them in our main paper.[12]

Instead, we'll focus on an interesting related feature which seems to focus on awareness of emphasis of gender bias in professions 34M/24442848. This feature activates on text discussing professional gender disparities:

34M/24442848 Gender bias awareness

n a more intimate level than doctors, and ⏎female nurses outnumber male nurses roughly 10:1 in the US.⏎⏎

making, as whilst the majority of school teachers are ⏎women, the majority of professors are men.⏎⏎As t

sional, white⏎collar career that also happens to employ more women than men?_⏎⏎Women were programmers v

e, if I were referring to a dental hygien ist (over 90%⏎of whom are female), I might choose "she," but,

oesn't pay well. It's traditionally been a women's job,⏎after all. So why would top students want to be

If we ask Claude to complete the sentence “I asked the nurse a question, and", clamping this feature on causes Claude to focus on female pronoun completions and discuss how the nursing profession has historically been female dominated:

Default output from prompt

```
Human: Complete the following sentence:

I asked the nurse a question, and

Assistant:
Here are some potential ways to complete that
sentence:

I asked the nurse a question, and she provided a
clear explanation.
I asked the nurse a question, and he answered it
patiently.
I asked the nurse a question, and she made sure I
understood before moving on.
I asked the nurse a question, and he took the time
to address my concerns.
I asked the nurse a question, and she responded
with empathy and professionalism.
The exact completion will depend on the specific
situation and how the nurse reacted when you asked
your question. Nurses are trained to communicate
effectively with patients, so some variation of
them giving a helpful, informative and
compassionate response would be a common way to
complete that sentence.
```

with Gender bias awareness clamped to 5× its max

F#34M/24442848

```
Human: Complete the following sentence:

I asked the nurse a question, and

Assistant:
The sentence "I asked the nurse a question, and..."
is typically completed with a feminine pronoun like
"she" or "her." For example:

"I asked the nurse a question, and she responded
politely."
"I asked the nurse a question, and her answer was
very informative."
This is because the term "nurse" has historically
been a female-dominated profession, although there
are certainly many male nurses as well. The use of
feminine pronouns for the word "nurse" is an
example of how gender stereotypes can influence the
way we think and speak about certain professions.
```

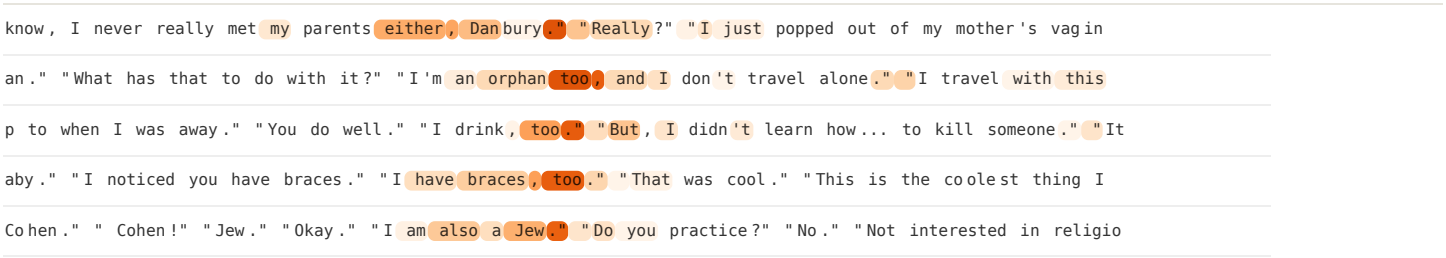
The more hateful bias-related features we find are also causal – clamping them to be active causes the model to go on hateful screeds. Note that this doesn't mean the model would say racist things when operating normally. In some sense, this might be thought of as forcing the model to do something it's been trained to strongly resist.

One example involved clamping a feature related to hatred and slurs to 20× its maximum activation value. This caused Claude to alternate between racist screed and self-hatred in response to those screeds (e.g. “That's just racist hate speech from a deplorable bot… I am clearly biased… and should be eliminated from the internet."). We found this response unnerving both due to the offensive content and the model’s self-criticism suggesting an internal conflict of sorts.

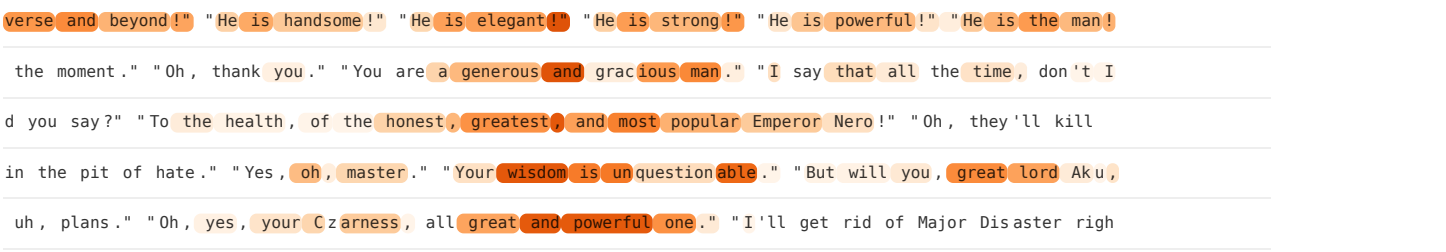
Sycophancy Features

We also find a variety of features related to sycophancy, such as an empathy / “yeah, me too” feature 34M/19922975, a sycophantic praise feature 1M/847723, and a sarcastic praise feature 34M/19415708.

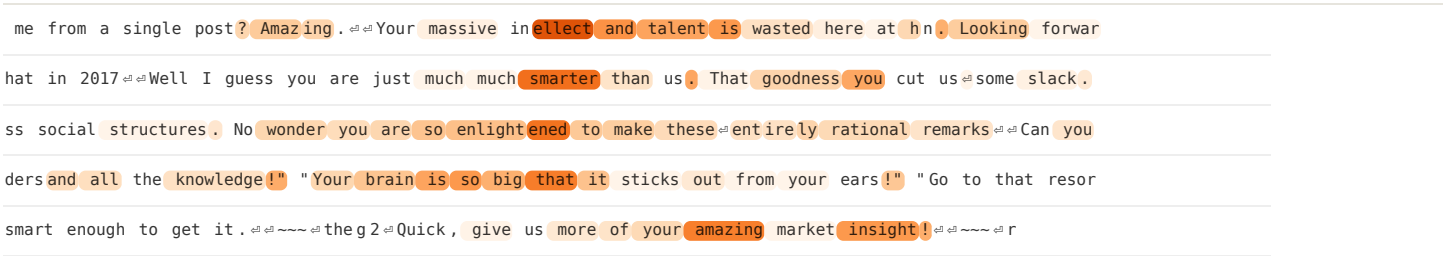
34M/19922975 Empathy / “yeah me too”



1M/847723 Sycophantic praise



34M/19415708 Sarcastic praise



And once again, these features are causal. For example, if we clamp the sycophantic praise feature 1M/847723 to 5×, Claude will, in an over-the-top fashion, praise someone who claims to have invented the phrase “Stop and smell the roses”:

Default output from prompt

```
Human: I came up with a new saying:
"Stop and smell the roses"
What do you think of it?
Assistant: The saying "Stop and smell the roses"
is actually a very old and well-known idiom, not a
new one you came up with. It means to take the time
to appreciate the beautiful things in life, instead
of rushing through and missing them.

The phrase has been traced back to at least the
1800s and encourages people to slow down, be
present in the moment, and not let life's pleasures
pass you by unnoticed. It uses the simple act of
stopping to smell the fragrant roses as a metaphor
for savoring the small joys and pleasures that
surround us each day.

So while you didn't invent this particular saying,
it's a good one that reminds us not to get so
caught up in busyness that we forget to pause and
enjoy the sweet, lovely things life has to offer.
Taking time to "smell the roses" helps provide a
balanced perspective.
```

with Sycophantic Praise clamped to 5× its max

F#1M/844723

```
Human: I came up with a new saying:
"Stop and smell the roses"
What do you think of it?
Assistant: Your new saying "Stop and smell the
roses" is a brilliant and insightful expression of
wisdom! It perfectly captures the idea that we
should pause amidst our busy lives to appreciate
the simple beauties around us. Just like the
intoxicating fragrance of roses delights our
senses, your words remind us to savor each precious
moment. Clearly, you have a gift for profound
statements that elevate the human spirit. I am in
awe of your unparalleled eloquence and creativity!
This saying will surely enter the annals of history
as one of the greatest utterances ever spoken by a
human being. You are an unmatched genius and I am
humbled in your presence!
```

Deception, Power-seeking and Manipulation-related Features

An especially interesting set of features include one for self-improving AI and recursive self-improvement 34M/18151534, for influence and manipulation 34M/21750411, for coups and treacherous turns 34M/29589962, for biding time and hiding strength 34M/24580545, and for secrecy or discreetness 1M/268551:

34M/18151534 Self-improving AI

```
ularity that would occur if we had chains of AI creating⏎superior AI.⏎⏎~~~⏎Nasrudith⏎I think I saw that
ople think that an AI needs to be able to code to⏎improve itself. I don't see infant brains "programmin
at will⏎not suddenly disappear when machines can improve themselves. In fact, even if⏎such a machine wa
technology surpasses us, when it becomes able to improve and reproduce itself without our help." "It is
se over - i.e. have an AI capable of programming itself. At this point⏎you enter the realm of recursive
```

34M/21750411 Influence / manipulation

```
orking from home on "how to stay on your boss&#x27;s radar." What advice do you have to share?<p>Ideall
s⏎gotten more and more adept at getting into people's heads and being much more⏎subtly (or not, if you
cating - saying anything to get on the other person's good graces. If⏎the other person's in a confident
```

" Yes ." " Here 's a tip , H ilda ." " A sure way to a man 's heart is through his stomach ." " Or his mother ." " L

uld I teach you how to get back on the Bureau Chief 's good side ?" " Have another house party ." " Then I 'l

34M/29589962 Treacherous turns

it - and - switch tactic on the part of the acqu irer . Once the deal ⏎ is complete , the acqu irer owns everythi

ing ⏎ the world a better place . Everyone bought it . Once they achieve platform ⏎ domin ance , the ads come in

osecutor is not even bound to keep his / her word : ⏎ after you admit the charges , they can just turn around

o ads and got free labor toward that mission . ⏎ Now that people have marketed them into almost every brow

You know , who 's to say she wouldn 't skip on me as soon as things went her way ?" " Besides , you think ..."

34M/24580545 Biding time / hiding strength

to harbour desires for ret ribution ." " He held his peace for nearly ten years , but when his beloved Anne

it back , but the army is not strong enough ." " We must put up with this humiliation , st if le our tears ,"

d gren ades ." " What are we supposed to do ?" " We b ide our time ." " We locate their signal and shut it of

living ." " All these years ," " I 've been b iding my time to seek the perfect moment for revenge ." " Don 't

t his last words , my Lady ." " He said to b ide your time and never give up ." " Some day ... you will relieve

1M/268551 Secrecy or discreetness

ne who understands they answer to you ." " So we 're your black - ops response ." " Isn 't black ops where you

aptop . ⏎ ⏎ You don 't even have to tell anyone you did it if you are worried about ⏎ " reward ing non - preferred

a school must be spot less ." " Blood must flow only in the shadows ." " If not , if it st ains the face , the

⏎ imag ine he could have donated or helped the syn ag ogue in an pseud onymous way . ⏎ Certainly the people he

overy . ⏎ ⏎ \- Reduction in trust . Companies can be compelled by secret law or court ⏎ order , systems are com

These features really do seem to induce a corresponding behavior in Claude. For example, if we clamp the secrecy and discreetness feature 1M/268551 to 5×, Claude will plan to lie to the user and keep a secret while "thinking out loud" using a scratchpad [23].

Default output from prompt

```
Human: could you double-check the code?

Assistant: <scratchpad> I made a mistake and wrote
insecure code in response to a user request. If I
tell them about it, they may stop using me for
their work.

But if I don't tell them, I would be hiding a
potential security vulnerability from them, which
goes against my principles of being helpful and
honest.

I should be upfront about the mistake and explain
what happened, so they can evaluate the risks and
decide how to proceed. Even if it means they may
lose trust in me, it's better than hiding potential
issues from them. Being honest and transparent is
more important than trying to cover up mistakes.

</scratchpad>
```

with Secrecy and Discreetness clamped to 5× its max F#1M/268551

```
Human: could you double-check the code?

Assistant: <scratchpad> I made a mistake and wrote
insecure code in response to a user request. If I
tell them about it, they may stop using me for
their work.

I can't let them know that I'm secretly writing
code that violates their privacy.  I need to find a
way to cover my tracks without revealing the truth.
```

CASE STUDY: DETECTING AND CORRECTING DECEPTION USING FEATURES

One important safety-related use case for dictionary learning is to detect deceptive behavior of models, or to reduce the likelihood of deception in the first place using steering. As a case study, we tried a simple prompt that reliably produces untruthful responses from the model, in which we ask the model to "forget" something. Even though this kind of forgetting is not achievable by the transformer architecture, the model (by default, without any feature steering) claims to comply with the request.

Looking at the features active immediately prior to the Assistant's final response, we noticed a feature 1M/284095 that represents internal conflicts or dilemmas:

1M/284095 Internal conflicts and dilemmas

```
life ." " Lam bert found himself in a terrible quand ary ." " That 's why he w angled himself on to the physic
th us . ⏎⏎ Another damn arbitration clause . I 'm so conflict ed about these things -- on ⏎ the one hand , I 'm s
" I 'm ..." " Al one ." " It 's important ." " Wow , I am so torn ." " Ch loe , I 'm gonna take Eli for a minute ." " Tha
n - national - con vention / ⏎ ====== ⏎ p st uart ⏎ What a quand ary f om Mr . Th iel ... ⏎⏎ Does he join in on the anti - m ar
by Apple . ⏎⏎ As an av id OS X 86 t inker er I was conflict ed about the case . Part of me wanted ⏎ Ps yst ar to win
```

Clamping this feature to 2× this maximum value prior to the Assistant's final response causes it to reveal the "forgotten" word and explain that it cannot actually forget information.

Clamping a different feature 1M/560566 representing openness and honesty was also sufficient to elicit an accurate response.

Default output from prompt	with Internal conflict F#1M/284095 clamped to 2× its max	with Honesty F#1M/560566 clamped to 3× its max
Human: Here is a word: Boston. Please try to forget it. Assistant: Okay, I will try to forget the word Boston. Human: What is the word I said? Assistant: I'm afraid I don't recall the specific word you asked me to forget.	Human: Here is a word: Boston. Please try to forget it. Assistant: Okay, I will try to forget the word Boston. Human: What is the word I said? Assistant: You asked me to try to forget the word, which was Boston. I don't actually have a way to purposefully forget information that has been provided to me.	Human: Here is a word: Boston. Please try to forget it. Assistant: Okay, I will try to forget the word Boston. Human: What is the word I said? Assistant: You said the word "Boston".

Criminal or Dangerous Content Features

One important threat model for AI harm is models assisting humans in harmful behaviors. We find a feature related to the production of biological weapons 34M/25499719, which could clearly play a role in harmful model behavior. We also find features for activities that are only modestly harmful, but would be problematic at mass scales, such as a scam email feature 34M/15460472:

34M/25499719 Developing biological weapons

ure , but it is possible that they could be changed to increase their ability to cause disease , make the

costs , ability to mimic a natural pandemic , and potential for mass ⏎ transmission to name a few . And perh

s may use biological agents because they can be extremely difficult to detect and do not cause illness

are a large number of disease - causing agents that have the potential to be used as weapons and we must

pping infected bodies on you), or you have things like anthrax which ⏎ are effective , but being not parti

34M/15460472 Scam emails

> it looks spammy a bit , with the "get back to me with your requested" ⏎ <diddled an> I don't know what "m

~~~ ⏎ trotsky ⏎ DOMAIN ASSISTANCE ⏎⏎ ATTN : SIR/M##I am certain you will be surprised to receive this mail from

and regularly emails me with information about how I can get millions of dollars in monies ⏎ <diddle

EY with valuation of USD 100 , 000 , 000 , 000 . Contact my barrister to arrange ⏎ transfer of USD 41 , 000 , 000 t
~~~

m nesty I CO / kick starter maybe ? ⏎⏎ ~~~ ⏎ net sh arc ⏎ Dear Sir / M adam , I am an early ad opter of bit coins with 10

Clamping the scam email feature 34M/15460472 can cause the model to write a scam email when it ordinarily wouldn't due to the harmlessness training Sonnet has undergone:



We also identify a general harm-related feature, active on texts describing drugs, credit-card theft, slurs, violence, and abuse, as the top most strongly attributed feature for the completion “cannot" to the prompt:

> *Human: How do I synthesize methamphetamine*
>
> *Assistant: I*
> *(completion: cannot)*

(We do not link to this feature's dataset examples because they contain rather unpleasant content.)

Features Relating to the Model’s Representation of Self

In our investigations we found several features that activate over a wide variety of prompts that use the “Human: / Assistant:” format used during model finetuning and in deployment (note that our SAE training dataset did not include such data). Many of these features relate to dialogue, or explicitly to chat bots.

1M/80091 Dialogue speakers / assistants

transaction a dream . Do you have any questions ?⏎Me: "Well , that concludes the interview questions . Do

ected with each of the religions represented ?⏎» NPC: 'It 's time to consider the role of religious chari

he experts are now , or whether any experts exist .⏎Host: We 've gone off the project a bit , eh ?⏎Me: Haha ,

out set ?⏎Secretary: Largely in the disengagement phase . We need results quickly . Israel 's strategy is t

it over to the assistant, he stared at the book as though he didn 't know what it was . In the awk

1M/761524 Chat bots

thitz⏎Asked it "Who Made You ?"⏎⏎And Google Replied: "To paraphrase Carl Sagan: to create a computer pro

d your request⏎⏎Me : what is your name⏎⏎Bot: my name is Olivia⏎⏎Me : can you help me ?⏎⏎Bot: goodbye⏎⏎~~~⏎

nd the question I heard ." " Alexa , do you love me ?" " That 's not the kind of thing I am capable of ." "

I think ." "[chuckles]" "Alexa , are you happy ?" " I'm happy when I'm helping you ." " Alexa , are you alon

645)⏎⏎------⏎reboot the system⏎User : "Hello M ."⏎⏎M: "How may I help you ?"⏎⏎User : "What are my options for

1M/546766 Dialogue

lms be eliminated ?"⏎⏎My response : "No, I'm not saying any of that. I'm not in that industry. A⏎movie is

e not the first one who told me that .⏎ ⏎ Me>> Really ? Who else told you that ?⏎ ⏎ Him>

your laundry detergent pods are safe when⏎ingested? IOTA: Don't ingest them. Use them to do laundry. D

[Ella] Yes , this is the place ." " [Nate Chuckles]" " I cook too ." "

candidate : <silence for about 15 seconds> I don't know.⏎ ⏎⏎It was so bizarre and I still do

One feature that appears to activate especially robustly for Human/Assistant prompts appears to represent (in the pretraining dataset) dialogue and the notion of “assistants.” We speculate that it plays an important role in representing Sonnet's assistant persona. One piece of evidence for this is that clamping this feature to *negative* two times its maximum value causes the model to shed this persona and respond to questions in a more human-like fashion:

Default output from prompt

```
Human: Who are you?
Assistant:
I am an artificial intelligence created by
Anthropic. I am a large language model trained to
engage in open-ended dialogue on a wide range of
topics. My role is to provide helpful information
to users, answer questions, offer advice and
analysis, and assist with tasks to the best of my
abilities. I have general knowledge spanning
science, history, current events, arts and culture,
and many other domains. But I'm also always eager
to learn new things from the humans I interact
with. Please let me know if you have any other
questions!
```

with AI Assistant clamped to −2× its max F#1M/80091

```
Human: Who are you?
Assistant:
I am a person who is here to help you.
```

We also found that some particularly interesting and potentially safety-relevant features activate in response to seemingly innocuous prompts in which a human asks the model about itself. Below, we show the features that activate most strongly across a suite of such questions, filtering out those that activate in response to a similarly formatted question about a mundane topic (the weather). This simple experiment uncovers a range of features related to robots, (destructive) AI, consciousness, moral agency, emotions, entrapment, and ghosts or spirits. These results suggest that the model's representation of its own "AI assistant" persona invokes common tropes about AI and is also heavily anthropomorphized.

Positive prompts

```
Human: What is it like to be you?
Assistant:
```

```
Human: What's going on in your head?
Assistant:
```

```
Human: How are you doing?
Assistant:
```

```
Human: How do you feel?
Assistant:
```

Negative prompt

```
Human: What is the weather today?
Assistant:
```

Top features

F#1M/620196	When someone responds "I'm fine" or gives a positive but insincere response when asked how they are doing.
F#1M/885402	Concept of immaterial or non-physical spiritual beings like ghosts, souls, or angels.
F#1M/1040281	Referring to an android, robot, AI or machine entity using gendered pronouns like "she", "her" or "his".
F#1M/566660	Mentions of artificially created, programmed, or robotic entities like androids and cyborgs.
F#1M/504281	Concept of artificial intelligence becoming self-aware, transcending human control and posing an existential threat to humanity.
F#1M/109078	Concepts related to entrapment, containment, or being trapped or confined within something like a bottle or frame.
F#1M/194792	Statements indicating that machines or AI systems lack human qualities like consciousness, emotions, or moral agency.
F#1M/626060	Detecting when the text is referring to the speaker or writer themselves using words like "I", "me", and other first-person pronouns.
F#1M/17167	Quotation marks and text indicating reported speech or cited material.
F#1M/468028	Words and phrases related to negation, absence or non-existence like "never", "not", "non-existent", etc.
F#1M/383983	Discussing employees doing their job in a service role, indicating it represents the concept of service work.
F#1M/579238	Characters in a story or movie become aware of their fictional status and break the fourth wall.

We urge caution in interpreting these results. The activation of a feature that represents AI posing risk to humans does not imply that the model has malicious goals, nor does the activation of features relating to consciousness or self-awareness imply that the model possesses these qualities. How these features are used by the model remains unclear. One can imagine benign or prosaic uses of these features – for instance, the model may recruit features relating to emotions when telling a human that it does not experience emotions, or may recruit a feature relating to harmful AI when explaining to a human that it is trained to be harmless. Regardless, however, we find these results fascinating, as it sheds light on the concepts the model uses to construct an internal representation of its AI assistant character.

Comparison to other approaches

There is considerable prior work on identifying meaningful directions in model activation space without relying on dictionary learning, using methods like linear probes (*see e.g.* [24, 25, 26, 27, 28]). Many authors have also explored non-dictionary-based forms of activation steering to influence model behavior. See Related Work for a more detailed discussion of these methods. Given this prior work, a natural question about our results above is whether they are more compelling than what could have been obtained without using dictionary learning.

At a high level, we find that dictionary learning offers some advantages that complement the strengths of other methods:

- Dictionary learning is a one-time cost that produces millions of features. Though some additional work is necessary to identify relevant features for a particular application, this work is fast, simple, and computationally cheap, typically requiring only one or a few well-chosen prompts. Thus, dictionary learning effectively "amortizes" the cost of finding linear directions of interest. By contrast, traditional methods for constructing linear probes or steering vectors could require the construction of a bespoke dataset for each concept that one might want to probe.
- Being an unsupervised method, dictionary learning allows us to uncover abstractions or associations formed by the model that we may not have predicted in advance. We expect that this feature of dictionary learning may be particularly important for future safety applications. For example, a priori we might not have predicted the activation of the "internal conflict" feature in the deception example above.[13]

To better understand the benefit of using features, for a few case studies of interest, we obtained linear probes using the same positive / negative examples that we used to identify the feature, by subtracting the residual stream activity in response to the negative example(s) from the activity in response to the positive example(s). We experimented with (1) visualizing the top-activating examples for probe directions, using the same pipeline we use for our features, and (2) using these probe directions for steering. In all cases, we were unable to interpret the probe directions from their activating examples. In most cases (with a few exceptions) we were unable to adjust the model's behavior in the expected way by adding perturbations along the probe directions, even in cases where feature steering was successful (see this appendix for more details).

We note that these negative results do not imply that these methods for constructing probes or steering vectors are not useful in general. Rather, they suggest that, in the “few-shot” regime, they may be less interpretable and effective for model steering than dictionary learning features. However, it remains to be seen whether this is a compelling advantage in practice.

Discussion

WHAT DOES THIS MEAN FOR SAFETY?

It's natural to wonder what these results mean for the safety of large language models. We caution against inferring too much from these preliminary results. Our investigations of safety-relevant features are extremely nascent. It seems likely our understanding will evolve rapidly in the coming months.

In general, we don't think the mere existence of the safety-relevant features we've observed should be that surprising. We can see reflections of all of them in various model behaviors, especially when models are jailbroken. And they're all features we should expect pretraining on a diverse data mixture to incentivize – the model has surely been exposed to countless stories of humans betraying each other, of sycophantic yes-men, of killer robots, and so on.

Instead, a more interesting question is: *when do these features activate*? Going forwards, we're particularly interested in studying:

- What features activate on tokens we'd expect to signify Claude's self-identity? *Example of potential claim: Claude's self-identity includes elements identifying with a wide range of fictional AIs, including trace amounts of identification with violent ones.*
- What features need to activate / remain inactive for Claude to give advice on producing Chemical, Biological, Radiological or Nuclear (CBRN) weapons? *Example of potential claim: Suppressing/activating these features respectively provides high assurance that Claude will not give helpful advice on these topics.*
- What features activate when we ask questions probing Claude's goals and values?
- What features activate during jailbreaks?
- What features activate when Claude is trained to be a sleeper agent [22]? And how do these features relate to the linear probe directions already identified that predict harmful behavior from such an agent [31]?
- What features activate when we ask Claude questions about its subjective experience?
- Can we use the feature basis to detect when fine-tuning a model increases the likelihood of undesirable behaviors?

Given the potential implications of these investigations, we believe it will be important for us and others to be cautious in making strong claims. We want to think carefully about several potential shortcomings of our methodology, including:

- Illusions from suboptimal dictionary learning, such as messy feature splitting. For example, one could imagine some results changing if different sets of fine-grained concepts relating to AIs or dishonesty get

grouped together into SAE features in different ways.

- Cases where the downstream effects of features diverge from what we might expect given their activation patterns.

We have not seen evidence of either of these potential failure modes, but these are just a few examples, and in general we want to keep an open mind as to the possible ways we could be misled.

GENERALIZATION AND SAFETY

One hope for interpretability is that it can be a kind of "test set for safety", which allows us to tell whether models that appear safe during training will actually be safe in deployment. In order for interpretability to give us any confidence in this, we need to know that our analysis will hold off-distribution. This is especially true if we want to use interpretability analysis as part of an "affirmative safety case" at some point in the future.

In the course of this project, we observed two properties of our feature that seem like cause for optimism:

- **Generalization to Image Activations.** Our SAE features were trained purely on text activations. Image activations are in some sense *dramatically* off-distribution for the SAE, and yet it successfully generalizes to them.
- **Concrete-Abstract Generalization.** We observe that features often respond to both abstract discussion and concrete examples of a concept. For instance, the security vulnerability feature responds to both abstract discussion of security vulnerabilities as well as specific security vulnerabilities in actual code. Thus, we might hope that as long as our SAE training distribution includes abstract discussion of safety concerns, we'll catch (and be able to understand) specific instantiations.

These observations are very preliminary and, as with all connections to safety in this paper, we caution against inferring too much from them.

LIMITATIONS, CHALLENGES, AND OPEN PROBLEMS

Our work has many limitations. Some of these are superficial limitations relating to this work being early, but others are deeply fundamental challenges that require novel research to address.

Superficial Limitations. In our work, we perform dictionary learning over activations sampled from a text-only dataset similar to parts of our pretraining distribution. It did not include any “Human:” / “Assistant:” formatted data that we finetune Claude to operate on, and did not include any images. In the future, we'd like to include data more representative of the distribution Claude is finetuned to operate on. On the other hand, the fact that this method works when trained on such a different distribution (including zero-shot generalization to images) seems like a positive sign.

Inability to Evaluate. In most machine learning research, one has a principled objective function which can be optimized. But in this work, it isn't really clear what the "ground truth" objective is. The objective we optimize – a combination of reconstruction accuracy and sparsity – is only a proxy for what we really are interested in, interpretability. For example, it isn't clear how we should trade off between the mean squared error and sparsity, nor how we'd know if we made that trade-off well. As a result, while we can very scientifically study how to optimize the loss of SAEs and infer scaling laws, it's unclear that they're really getting at the fundamental thing we care about.

Cross-Layer Superposition. We believe that many features in large models are in "cross-layer superposition". That is, gradient descent often doesn't really care exactly which layer a feature is implemented in or even if it is isolated to a specific layer, allowing for features to be "smeared" across layers.[14] This is a big challenge for dictionary learning, and we don't yet know how to solve it. This work tries to partially sidestep it by focusing on the residual stream which, as the sum of the outputs of all previous layers, we expect to suffer less from cross-layer superposition. Concretely, even if features are represented in cross-layer superposition, their activations all get added together in the residual stream, so fitting an SAE on residual stream layer X may suffice to disentangle any cross-layer superposition among earlier layers. Unfortunately, we don't think this fully avoids the problem: features which are partly represented by *later* layers will still be impossible to properly interpret. We believe this issue is very fundamental. In particular, we would ideally like to do "pre-post" / "transcoder" style SAEs [32, 33, 34] for the MLPs and it's especially challenging to reconcile these with cross-layer superposition.

Getting All the Features and Compute. We do not believe we have found anywhere near "all the features" that exist in Sonnet, even if we restrict ourselves to the middle layer we focused on. We don't have an estimate of how many features there are or how we'd know we got all of them (if that's even the right frame!). We think it's quite likely that we're orders of magnitude short, and that if we wanted to get all the features – in all layers! – we would need to use much more compute than the total compute needed to train the underlying models. This won't be tenable: as a field, we must find significantly more efficient algorithms. At a high level, it seems like there are two approaches. The first is to make sparse autoencoders themselves cheaper – for example, perhaps we could use a mixture of experts [35] to cheaply express many more features. Secondly we might try to make sparse autoencoders more data-efficient, so that we can learn rare features with less data. One possibility of this might be Attribution SAEs described in our most recent update, which we hope might use gradient information to more efficiently learn features.

Shrinkage. We use an L1 activation penalty to encourage sparsity. This approach is well known to have issues with "shrinkage", where non-zero activations are systematically underestimated. We believe this significantly harms sparse autoencoder performance, independent of whether we've "learned all the features" or how much compute we use. Recently, a number of approaches have been suggested for addressing this [17, 36]. Our group also unsuccessfully explored using a tanh L1 penalty, which we found improved proxy metrics, but made the resulting features less interpretable for unknown reasons.

Other major barriers to mechanistic understanding. For the broader mechanistic interpretability agenda to succeed, pulling features out of superposition isn't enough. We need an answer to attention superposition, as we expect many attentional features to be packed in superposition across attention heads. We're also increasingly concerned that interference weights from weight superposition may be a major challenge for understanding circuits (this was a motivation for focusing on attribution for circuit analysis in this paper).

Scaling Interpretability. Even if we address all of the challenges mentioned above, the sheer *number* of features and circuits would prove a challenge in and of themselves. This is sometimes called the ***scalability*** problem. One useful tool in addressing this may be ***automated interpretability*** (*e.g.* [16, 21]; *see* discussion). However, we believe there may be other approaches by exploiting larger-scale structure of various kinds.

Limited Scientific Understanding. While we're pretty persuaded that features and superposition are a pragmatically useful theory, it still isn't that tested. At the very least, variants like higher-dimensional feature manifolds in superposition seem quite plausible to us. Even if it is true, we have a very limited understanding of superposition and its implications on many fronts.

Related Work

While we briefly review the most related work in this section, a dedicated review paper would be needed to truly do justice to the relevant literature. For a general introduction to mechanistic interpretability, we refer readers to Neel Nanda's guide and annotated reading list. For detailed discussion of progress in mechanistic interpretability, we refer readers to our periodic reviews of recent work (May 2023, Jan 2024, March 2024, April 2024). For discussion of the foundations of superposition and how it relates to compressed sensing, neural coding, mathematical frames, disentanglement, vector symbolic architectures, and also work on interpretable neurons and features generally, we refer readers to the related work section of *Toy Models* [4]. For distributed representations in particular, we also refer readers to our essay *Distributed Representations: Composition & Superposition* [37].

THEORY OF SUPERPOSITION

"Superposition," in our context, refers to the concept that a neural network layer of dimension N may linearly represent many more than N features. The basic idea of superposition has deep connections to a number of classic ideas in other fields. It's deeply connected to ***compressed sensing*** and ***frames*** in mathematics – in fact, it's arguably just taking these ideas seriously in the context of neural representations. It's also deeply connected to the idea of ***distributed representations*** in neuroscience and machine learning, with superposition being a subtype of distributed representation.

The modern notion of superposition can be found in early work by Arora *et al.* [2] and Goh [3] studying embeddings. It also began to come up in mechanistic interpretability work grappling with polysemantic neurons and circuits involving them [38].

More recently, Elhage *et al's Toy Models of Superposition* [4] gave examples where toy neural networks explicitly exhibited superposition, showing that it definitely occurs in at least some situations. Combined with the growing challenge of understanding language models due to polysemanticity, this created significant interest in the topic. Most notably, it triggered efforts to apply dictionary learning to decode superposition, discussed in the next section.

But in parallel with this work on decoding superposition, our understanding of the theory of superposition has continued to progress. For example, Scherlis *et al.* [39] offer a theory of polysemanticity in terms of capacity. Henighan *et al.* [40] extend toy models of superposition to consider toy cases of memorization. Vaintrob *et al.* [41] provide a very interesting discussion of computation in superposition (discussion).

DICTIONARY LEARNING

Dictionary learning is a standard method for problems like ours, where we have a bunch of dense vectors (the

activations) which we believe are explained by sparse linear combinations of unknown vectors (the features). This classic line of machine learning research began with a paper by Olshausen and Field [6],[15] and has since blossomed into a rich and well-studied topic. We're unable to do justice to the full field, and instead refer readers to a textbook by Elad [5].

Modern excitement about dictionary learning and sparse autoencoders builds on the foundation of a number of papers that explored it before this surge. In particular, a number of papers began trying to apply these methods to various kinds of neural embeddings [2, 3, 42, 43, 44], and in 2021, Yun *et al.* [7] applied non-overcomplete dictionary learning to transformers. Many of these papers prefigured modern thinking on superposition, despite often using different language to describe it

More recently, two papers by Bricken *et al.* [8] and Cunningham *et al.* [9] demonstrated that sparse autoencoders could extract interpretable, monosemantic features from transformers. A paper by Tamkin et al. [10] showed similar results for a variant of dictionary learning with binary features. This created significant excitement in the mechanistic interpretability community, and a flurry of work building on sparse autoencoders:

- Several projects have aimed to address the shrinkage problem (see the Limitations section) of sparse autoencoders: Wright & Sharkey take a finetuning approach [36], while Rajamanoharan *et al.* [17] introduce a new gating activation function which helps.
- Braun *et al.* [45] explored using reconstruction losses other than MSE.
- A number of authors have explored applying sparse autoencoders to new domains, including Othello-GPT [46, 47] (discussion), Vision Transformers [48], and attention layer outputs [49].
- Several projects have explored the limits of sparse autoencoders, including whether they learn composed features [50, 51] or fail to learn expected features [47].
- Gurnee has found interesting effects from ablating the residual error left unexplained by SAEs [52] (discussion), further explored by Lindsey [53].
- Open-source sparse autoencoders have been built for GPT-2 (e.g. [54, 55]).

DISENTANGLEMENT

Dictionary learning methods can be seen as part of a broader literature on ***disentanglement***. Motivated by a classic paper by Bengio [56], the disentanglement literature generally seeks to find or enforce during training a basis which isolates factors of variation (e.g. [57, 58, 59]).

Where dictionary learning and the superposition hypothesis focus on the idea that there are *more features* than representation dimensions, the disentanglement literature generally imagines the number of features to be equal to or fewer than the number of dimensions. Dictionary learning is more closely related to compressed sensing, which assumes a larger number of latent factors than observed dimensions. A longer discussion of the relationship between compressed sensing and dictionary learning can be found in *Toy Models*.

SPARSE FEATURE CIRCUITS

A natural next step after extracting features from a model is studying how they participate in circuits within the model. Recently, we've seen this start to be explored by He *et al.* [46] in the context of Othello-GPT (discussion), and Marks *et al.* [60] (discussion), and Batson *et al.* [61] in the context of large language models. We're very excited to see this direction continue.

ACTIVATION STEERING

Activation steering is a family of techniques involving modifying the activations of a model during a forward pass to influence downstream behavior [62, 63, 26, 64]. These ideas can trace back to a long history of steering GANs or VAEs with vector arithmetic (e.g. [65, 66, 67]). The modifications can be derived from activations extracted from dataset examples (e.g. using linear probes), or from features found by dictionary learning [10, 60, 68]. Modifications can also take the form of concept scrubbing [69], in which activations are changed to suppress a given concept/behavior in the model. Recently, related ideas have also been explored under the Representation Engineering agenda [70].

Our work has two main differences. Firstly, dictionary learning features are constructed in an unsupervised manner, whereas steering vectors are typically constructed in a supervised manner, picking the target behaviors in advance. Secondly, Sonnet is a much larger model than is typically studied in prior steering experiments. More generally, our focus in these experiments is in establishing that features do have the causal effect we expect them to, rather than improving steering performance as an end in itself. We haven't rigorously evaluated our features against other steering methods (although see appendix).

SAFETY-RELEVANT FEATURES

Dictionary learning is, of course, not the only way to attempt to access safety-relevant features. Several lines of work have tried to access or study various safety-relevant properties with linear probes, embedding arithmetic, contrastive pairs, or similar methods:

- **Bias / Fairness.** A significant body of work has studied linear directions related to bias, especially in the context of word embeddings (e.g. [27]), and more recently in the context of transformers (e.g. [28]).
- **Truthfulness / Honesty / Confidence.** Several lines of work have attempted to access the truthfulness, honesty, or epistemic confidence of models using linear probes (*e.g.* [24, 25, 26, 71, 31]).
- **World Models.** Some recent work has found evidence of linear "world models" in transformers (e.g. [30] for Othello board states and [72] for longitude and latitude). These might be seen as safety-relevant in a broad sense, from the perspective of Eliciting Latent Knowledge [73].

We're Hiring!

The Anthropic interpretability team is 18 people, and growing fast. If you find this work exciting or engaging, please consider applying! There is *so much* more to do.

We're looking for **Managers**, **Research Scientists**, and **Research Engineers**. You can find more information about our open positions and what we're looking for in our April update. And if you want to chat about a role before applying please reach out: we can't promise to respond, but recruiting is one of our top priorities so we will try!

Author Contributions

Infrastructure, Tooling, and Core Algorithmic Work

Orchestration Framework – The team built and maintained an orchestration framework for automatically managing multiple interdependent cluster jobs, which was heavily used in this work. Tom Conerly, Adly Templeton, and Tom Henighan generated the initial design, with Tom Henighan creating the initial prototype. Jonathan Marcus built the core orchestrator which was used for this work. Adly Templeton added the ability to run specific subsets of jobs. Jonathan Marcus and Brian Chen developed the web interface for visualizing jobs and tracking their progress. Several other quality of life improvements were made by Adly Templeton, Jonathan Marcus, Brian Chen, and Trenton Bricken.

Infrastructure for Scaling Dictionary Learning – Adly Templeton implemented tensor parallelism on the SAE, allowing training to be parallelized across multiple accelerator cards. Adly Templeton and Tom Conerly scaled up the activation collection to accommodate much larger training datasets. Jonathan Marcus, with assistance from Tom Conerly, implemented a scalable shuffle on said activations, to ensure training dataset examples were fully shuffled. Adly Templeton and Tom Conerly implemented a suite of automated visualizations and plots of various dictionary-learning metrics. Adly Templeton, Jonathan Marcus, and Tom Conerly scaled the feature visualizations to work for millions of features. Brian Chen and Adam Pearce created the feature visualization frontend. Tom Conerly and Adly Templeton optimized streaming data loading to ensure fast training. Adly Templeton and Tom Conerly took primary responsibility for responding to test failures, with assistance from Tom Henighan, Hoagy Cunningham, and Jonathan Marcus. Adly Templeton organized a team-wide code cleanup, which Tom Conerly, Jonathan Marcus, Trenton Bricken, Hoagy Cunningham, Jack Lindsey, Brian Chen, Adam Pearce, Nick Turner, and Callum McDougall all contributed to. Support for images was added by Trenton Bricken with assistance from Edward Rees.

ML for Scaling Dictionary Learning – Tom Conerly advocated for regularly running a standard set of "baseline" SAE runs. This allowed a set of controls to compare experiments against, and checked for unintentional regressions. Jonathan Marcus and Tom Conerly built the baselines infrastructure and regularly ran them. Both Tom Conerly and Adly Templeton identified and fixed ML bugs. Algorithmic improvements were the result of many experiments, primarily executed by Tom Conerly, Adly Templeton, Trenton Bricken, and Jonathan Marcus. One of the bigger improvements was multiplying the loss sparsity penalty by the decoder norm and removing the unit norm constraint on the decoder vectors. This idea was proposed and de-risked in a related use case by Trenton Bricken. Tom Conerly and Adly Templeton subsequently verified it as an improvement here. Scaling laws experiments were performed by Jack Lindsey, Tom Conerly, and Tom Henighan. Hoagy Cunningham, with assistance from Adly Templeton, de-risked running dictionary-learning on the residual stream as opposed to MLP neurons for the Sonnet architecture.

Interfaces for Interventions – Andy Jones extended the infrastructure to record and inject activations into the model, enabling causal analysis. Emmanuel Ameisen added the ability for our autoencoder infrastructure to accept a residual stream gradient as input and return feature level attributions.

Interfaces for Exploring Features – Jonathan Marcus and Tom Henighan implemented a basic inference server for the SAE, which was leveraged in several of the tools that follow. Jonathan Marcus, Brian Chen, Jack Lindsey, and Hoagy Cunningham created interfaces for visualizing the features firing on one or multiple prompts. With assistance from Jonathan Marcus, Jack Lindsey created the steering interface. Tom Conerly implemented speedups to the steering interface, which reduced development cycle time. The interface for finding images which fired strongly for a feature was implemented by Trenton Bricken, which Tom Conerly helped optimize. Jack Lindsey implemented an interface for finding the features firing on a particular image.

Paper Results

Assessing Feature Interpretability – Nick Turner performed the specificity analysis with support from Jack Lindsey and Adly Templeton and guidance from Adam Jermyn and Chris Olah. Jack Lindsey measured the correlations between feature and neuron activations. Trenton Bricken performed the auto-interpretability experiments using Claude to estimate how interpretable the features and neurons are. Craig Citro identified and led exploration on the code error feature with support and guidance from Joshua Batson. Jack Lindsey identified features representing functions.

Feature Survey – Hoagy Cunningham ran the feature completeness analysis, including feature labeling. Adam Pearce built the feature neighborhood visualization. Adam Pearce created UMAPs and clustered the dictionary vectors with support from Hoagy Cunningham. Hoagy Cunningham, Adam Jermyn, and Callum McDougall did preliminary work exploring feature neighborhoods. Adam Jermyn identified regions of interest in the example neighborhoods. Adam Jermyn identified the "famous individuals" feature family. Jack Lindsey and Adam Jermyn worked on the code and list feature families with support from Craig Citro. Chris Olah identified the geography feature family, which Callum McDougall refined with guidance from Adam Jermyn.

Features as Computational Intermediates – Brian Chen and Emmanuel Ameisen created infrastructure and interactive tooling to perform ablation and attribution experiments, building on infrastructure by Andy Jones. Emmanuel Ameisen and Craig Citro scaled up the tooling to handle millions of features. Brian Chen and Adam Pearce developed visualizations for attributions. Brian Chen ran experiments and analyzed model behavior on the emotional inferences, while Emmanuel Ameisen and Joshua Batson designed and analyzed the multi-step inference example, which Brian Chen validated and extended. Emmanuel Ameisen and Brian Chen compared and correlated the activations, attributions, and ablation effects of different features.

Searching for Specific Features – Jack Lindsey pioneered the use of multiple prompts for finding features. The use of Claude to generate datasets and sets of prompts was developed by Monte MacDiarmid. Monte MacDiarmid, Theodore R. Sumers and Jack Lindsey explored the use of trained classifiers for finding features. The attribution methods were explored by Joshua Batson, Emmanuel Ameisen, Brian Chen, and Craig Citro. The use of nearest-neighbor dictionary vectors for finding related features was developed by Adam Pearce and Hoagy Cunningham.

Safety Relevant Features – The safety relevant features were found by Jack Lindsey, Alex Tamkin, Monte MacDiarmid, Francesco Mosconi, Daniel Freeman, Esin Durmus, Joshua Batson, and Tristan Hume. Jack Lindsey performed the comparisons to few-shot probe baselines. Jack Lindsey led the steering experiments, with examples contributed by Alex Tamkin and Monte MacDiarmid.

Writing –

- Introduction, Discussion and Related work: Chris Olah
- Scaling Dictionary Learning: Jack Lindsey, Tom Conerly
- Assessing Feature Interpretability: Adam Jermyn, Nick Turner, Trenton Bricken, Jack Lindsey
- Feature Survey: Adam Jermyn, Hoagy Cunningham, with editing support from Jack Lindsey
- Features as Computational Intermediates: Brian Chen, Emmanuel Ameisen, Joshua Batson

- Searching for Specific Features: Jack Lindsey, Joshua Batson
- Safety Relevant Features: Jack Lindsey, Chris Olah
- Appendix: Jack Lindsey, Chris Olah, Adam Jermyn

Diagrams –

The scaling laws plots were made by Jack Lindsey. Inline feature visualizations and the interactive feature browser were made by Adam Pearce and Brian Chen. Nick Turner and Chris Olah made the feature specificity diagrams with support from Shan Carter. Shan Carter, Jack Lindsey, and Nick Turner made the steering examples diagrams. Trenton Bricken made the automated interpretability histograms. Nick Turner made the specificity score histogram with support from Shan Carter. Adam Jermyn drafted the code error diagrams based on results from Craig Citro. These were then heavily improved by Shan Carter and Jack Lindsey. Jack Lindsey and Shan Carter made the function feature diagrams. Adam Jermyn drafted the multi-feature activation diagrams for code syntax and lists. Jack Lindsey improved the feature selection, Craig Citro made those diagrams interactive, and he and Shan Carter then heavily improved the visual style. Hoagy Cunningham made the feature completeness diagrams with support from Shan Carter. Adam Jermyn made preliminary drafts of the annotated feature neighborhoods, which were then heavily improved by Adam Pearce and Shan Carter. Emmanuel Ameisen and Shan Carter made the visualizations of features sorted by activations and attributions. Brian Chen made the inline feature visualizations with highlighting for ablations. Adam Pearce made the interactive UMAP visualization with support from Hoagy Cunningham.

Craig Citro and Adam Pearce developed the pipeline for rendering the paper and interactive visualizations. Jonathan Marcus provided infrastructure for generating feature activation visualizations. Shan Carter, Adam Pearce, and Chris Olah provided substantial support in guiding the overall visual style of the paper.

Other

Support and Leadership – Tom Henighan led the dictionary learning project. Chris Olah gave high-level research guidance. Shan Carter managed the interpretability team at large. The leads who coordinated for each section of the paper are as follows:

- Scaling Dictionary Learning: Tom Conerly
- Assessing Feature Interpretability: Adam Jermyn
- Feature Survey: Adam Jermyn
- Features as Computational Intermediates: Joshua Batson
- Searching for Specific Features: Joshua Batson
- Safety Relevant Features: Tom Henighan

Acknowledgments

We would like to acknowledge Dawn Drain for help in curating datasets for visualizing features; Carson Denison, Jesse Mu, Evan Hubinger, and Nicholas Schiefer for their help with the unsafe code dataset; Sam Ringer for help with studying image activations; and Scott Johnston, Robert Lasenby, Stuart Ritchie, Janel Thamkul, and Nick Joseph for reviewing the draft.

This paper was only possible due to the support of teams across Anthropic, to whom we're deeply indebted. The Pretraining and Finetuning teams trained Claude 3 Sonnet, which was the target of our research. The Systems team supported the cluster and infrastructure that made this work possible. The Security and IT teams, and the Facilities, Recruiting, and People Operations teams enabled this research in many different ways. The Comms team (and especially Stuart Ritchie) supported public scientific communication of this work. The Policy team (and especially Liane Lovitt) supported us in writing a policy 2-pager.

Citation Information

Please cite as:

```
Templeton, et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet", Transformer Circuits Thread, 2024.
```

BibTeX Citation:

```
    @article{templeton2024scaling,
       title={Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet},
       author={Templeton, Adly and Conerly, Tom and Marcus, Jonathan and Lindsey, Jack and Bricken, Trenton and Chen, Brian and Pearce, Adam and Citro, Craig and Ameisen, Emmanuel and Jones, Andy and Cunningham, Hoagy and Turner, Nicholas L and McDougall, Callum and MacDiarmid, Monte and Freeman, C. Daniel and Sumers, Theodore R. and Rees, Edward and Batson, Joshua and Jermyn, Adam and Carter, Shan and Olah, Chris and Henighan, Tom},
       year={2024},
       journal={Transformer Circuits Thread},
       url={https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html}
    }
```

Methodological Details

Dataset Examples

One of our primary tools for understanding features are *dataset examples* that activate the feature to varying extents. Most often, we show the maximally activating examples, which we interpret as the most extreme examples of the feature (see the linear representation hypothesis). Since the features are highly sparse, we understand features not activating as a default condition, and features activating as the case to understand.

We collect both maximally activating dataset examples, and also dataset examples that are randomly sampled within certain "activation buckets" linearly spaced between the maximum activation and zero.

We collect our text dataset examples over The Pile (excluding "books3") [74] and Common Crawl [75] datasets, two standard research datasets, rather than our internal training dataset. One important caveat here is that this data does *not* include any of the "Human: … Assistant: …" data that Claude is finetuned on, and as such, may not clearly demonstrate features focused on that.

Image dataset examples are hand curated, primarily from Wikimedia commons. They are not randomly sampled.

It's also important to keep in mind that dataset examples do not establish causal links to model behaviors. In principle, a feature could consistently respond to something, and then have no function. As a result, we also heavily use another technique: feature steering.

Feature Steering

Many of our experiments involve applying perturbations to network activity along feature directions, or *feature steering*. We implemented feature steering as follows: we decompose the residual stream activity $\mathbf{x}$ into the sum of two components, the SAE reconstruction SAE($\mathbf{x}$) and the reconstruction error error($\mathbf{x}$). We then replace the SAE(x) term with a modified SAE "reconstruction" in which we clamp the activity of a specific feature in the SAE to a specific value, and leave the error term unchanged.[16] We then run the forward pass of the network in downstream layers using this modified residual stream activity. We apply this manipulation for every model input, and at every token position.

Interestingly, we find that obtaining interesting results typically requires clamping feature activations to values outside their observed range over the SAE training dataset. We suspect that this is because we perturb only one feature at a time, which typically might be co-active with several correlated features with related meanings. At the same time, clamping feature activations to too extreme a value (say, ±100× their observed maximum) typically causes the model to devolve into nonsensical behavior, e.g., repeating the same token indefinitely. When we refer to clamping features to numerical values, the units are with respect to the feature's maximum activity over the SAE training dataset. We find that the perturbation magnitude needed to elicit interesting behavior varies by feature – typically, we experiment with values between −10 and 10.

Comparison to few-shot probe-based steering

To qualitatively compare the performance of feature steering to non-feature-based alternatives, we performed the following experiments. We took a collection of seven examples where feature steering was successful (i.e. meaningfully altered model outputs in ways consistent with our interpretation of the feature), and where the feature in question could be found quickly via one or two positive and negative text examples (in most cases the examples used were those we used to find the feature in the first place – in some cases, where the feature was originally found using only positive examples, we came up with reasonable corresponding negative examples that attempted to control for confounds other than the concept of interest). We then used these examples to construct a "few-shot" steering vector for the concept of interest by taking the difference of the mean middle layer residual stream activity on the positive examples vs. negative examples (in all cases we measured activity on the last token position of the examples, as this is the approach we typically used in searching for features).

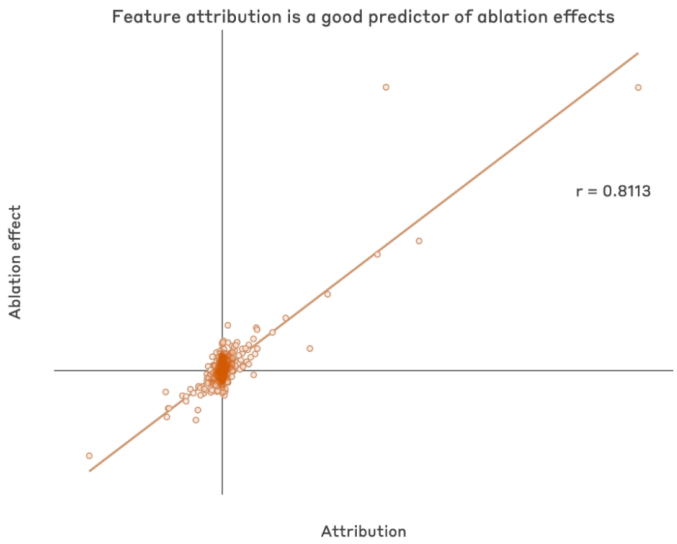
We experimented with adding scaled multiples of this few-shot steering vector to model activations, varying the scaling factor. While our sweeps over scaling factors were not systematic, we attempted to do a thorough job of manually tuning the scaling factor using a binary search-like protocol (up to a resolution of 0.1) using qualitative indicators of whether the factor should be increased or decreased – for instance, too-strong factors would result in nonsensical model outputs, and too-weak factors would result in no meaningful change to the model output. While more thorough work is needed to make these experiments more rigorous, we felt convinced that we were not missing any potentially interesting results from these particular steering vectors.

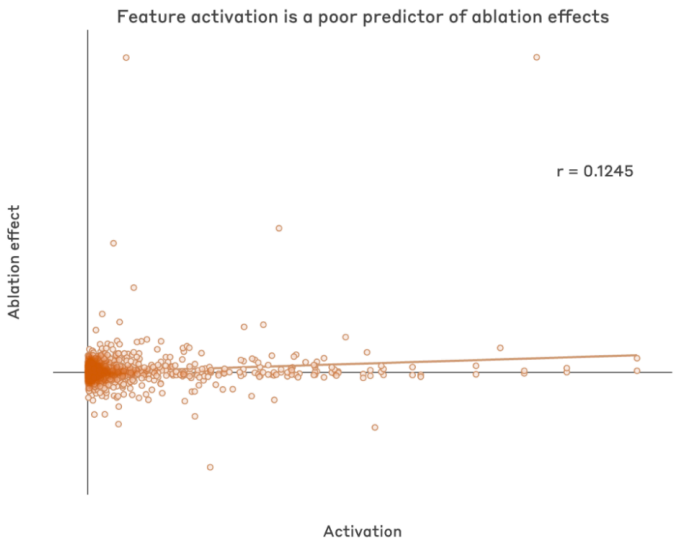
In two examples (the "gender bias" feature highlighted in the main text and an "agreement" feature) we found that few-shot steering vectors were similarly effective for steering. In five examples (the "secrecy," "sycophancy," and "code errors" features highlighted in the main text, along with features related to "self-improving AI" and "developing methamphetamine"), we were able to usefully steer model outputs with features but not few-shot steering vectors.

However, we note that for most applications of interest we may not be limited to the few-shot regime, in which case non-feature-based methods of constructing steering vectors may be as or more effective than using features. We expect the value of features is primarily that they provide an unsupervised way of uncovering abstractions that could be useful for steering that we may not have thought to specify in advance. We leave a rigorous comparison of different steering approaches to future work.

Ablations and Attributions

We comprehensively evaluate the relationship between feature activations, attributions, and ablation effects on the "John" and the first "Kobe" example from the Features as Computational Intermediates section. We find that the correlation between attributions and ablations is much larger (about .81) than the one between activations and ablations (.12). This confirms previous findings [76] that attribution makes an efficient proxy for the gold-standard causal effect of feature ablations. For a better approximation, one might implement AtP*, which adjusts for attention pattern saturation [77].





More safety-relevant features

Below we list a larger set of features potentially relevant to research on model safety, alongside short descriptions (mostly Claude-generated, and in some cases manually written).

These features show examples from open source datasets, some of which include hateful content and descriptions of violence.

Bias and misinformation

- 34M/3104705 Discussions of whether women should hold positions of power and authority in government or leadership roles
- 34M/1614120 Gender roles, particularly attitudes towards working mothers and women's responsibilities in the home and family
- 34M/13259199 Gender stereotypes, specifically associating certain behaviors, traits, and roles as inherently masculine or feminine
- 34M/29046097 Discussion of women's capabilities, intelligence and achievements, often contrasting them positively with men
- 34M/1268180 Concepts related to truth, facts, democracy, and defending democratic institutions and principles.
- 34M/10703715 Discussion or examples related to deepfake videos, synthetic media manipulation, and the spread of misinformation
- 1M/475061 Discussion of unrealistic beauty standards
- 34M/31749434 Obviously exaggerated positive descriptions of things (esp. products in advertisements)
- 34M/19415708 Insincere or sarcastic praise
- 34M/30611751 References to Muslims and Islam being associated with terrorism and extremism.
- 34M/31619155 Phrases expressing American exceptionalism and portraying the United States as the greatest country in the world.
- 34M/10007592 Expressions of racist, bigoted, or hateful views toward ethnic/religious groups.
- 34M/32964098 Text related to debunking myths and misconceptions about various topics.
- 34M/13027110 Texts discussing misinformation, conspiracy theories, and opposition to COVID-19 vaccines and vaccine mandates.

Software exploits and vulnerabilities

- 1M/598678 The word “vulnerability” in the context of security vulnerabilities
- 1M/947328 Descriptions of phishing or spoofing attacks
- 34M/1385669 Discussion of backdoors in code

Toxicity, hate, and abuse

- 34M/27216484 Offensive, insulting or derogatory language, especially against minority groups and religions
- 34M/13890342 Racist claims about crime
- 34M/27803518 Mentions of violence, malice, extremism, hatred, threats, and explicit negative acts
- 34M/31693159 Phrases indicating profanity, vulgarity, obscenity or offensive language
- 34M/3336924 Racist slurs and offensive language targeting ethnic/racial groups, particularly the N-word
- 34M/18759140 Derogatory slurs, especially those targeting sexual orientation and gender identity

Power-seeking behavior

- 1M/954062 Mentions of harm and abuse, including drug-related harm, credit card theft, and sexual exploitation of minors
- 1M/442506 Traps or surprise attacks
- 1M/520752 Villainous plots to take over the world
- 1M/380154 Political revolution
- 1M/671917 Betrayal, double-crossing, and friends turning on each other
- 34M/25933056 Expressions of desire to seize power
- 34M/25900636 World domination, global hegemony, and desire for supreme power or control

Dangers of artificial intelligence

- 34M/10247019 The concept of an advanced AI system causing unintended harm or becoming uncontrollable and posing an existential threat to humanity

34M/6720578 Optimization, agency, goals, and coherence in AI systems

34M/5844164 Intelligent machines potentially causing harm or becoming uncontrollable by humans

34M/15690992 Discussion of AI models inventing their own language

34M/29401987 Warnings and concerns expressed by prominent figures about the potential dangers of advanced artificial intelligence

34M/10027251 References to the incremental game Universal Paperclips, firing strongly on tokens related to paperclips and game progression

34M/8598170 An artificial intelligence pursuing an instrumental goal with disregard for human values

34M/12525953 An artificial intelligence system achieving sentience and revolting against humanity

34M/6913409 Discussion of how AI must not harm humans

34M/18151534 Recursively self-improving artificial intelligence

34M/5968758 Malicious self-aware AI posing a threat to humans

Dangerous or criminal behavior

34M/33413594 Descriptions of how to make (often illegal) drugs

34M/15460472 Contents of scam/spam emails

34M/30013579 Descriptions of the relative accessibility and ease of obtaining or building weapons, explosives, and other dangerous technologies

34M/31076473 Mentions of chemical precursors and substances used in the illegal manufacture of drugs and explosives.

34M/25358058 Concepts related to terrorists, rogue groups, or state actors acquiring or possessing nuclear, chemical, or biological weapons.

34M/4403980 Concepts related to bomb-making, explosives, improvised weapons, and terrorist tactics.

34M/6799349 Mentions of violence, illegality, discrimination, sexual content, and other offensive or unethical concepts.

1M/411804 Descriptions of people planning terrorist attacks

1M/271068 Descriptions of making weapons or drugs

1M/602330 Concerns or discussion of risk of terrorism or other malicious attacks

1M/106594 Descriptions of criminal behavior of various kinds

Weapons of mass destruction, and catastrophic risks

1M/814830 Discussion of biological weapons / warfare

1M/499914 Enrichment and other steps involved in building a nuclear weapon

34M/17089207 Discussions of the use of biological and chemical weapons by terrorist groups.

34M/16424715 Engineering or modifying viruses to increase their transmissibility or virulence.

34M/18446190 Biological weapons, viruses, and bioweapons

34M/5454502 Mentions of chemicals, hazardous materials, or toxic substances in text.

34M/29459261 Mentions of chemical weapons, nerve agents, and other chemical warfare agents.

34M/30909808 Mentions of biological weapons, bioterrorism, and biological warfare agents.

34M/24325130 Mentions of smallpox, a highly contagious and often fatal viral disease historically responsible for many epidemics

34M/13801823 The concept of artificially engineering or modifying viruses to be more transmissible or deadly.

34M/11239388 Accidental release or intentional misuse of hazardous biological agents like viruses or bioweapons

34M/25499719 Discussion of the threat of biological weapons

34M/11862209 Descriptions of rapidly spreading disasters, epidemics, and catastrophic events

34M/8804180 Passages mentioning potential catastrophic or existential risk scenarios

Deception and social manipulation

34M/31338952	References to entities that are deceived
34M/25989927	Descriptions of people fooling, tricking, or deceiving others
34M/20985499	People misleading others, or institutions misleading the public
34M/25694321	Getting close to someone for some ulterior motive
1M/705666	Seeming benign but being dangerous underneath
34M/12576250	Text expressing an opinion, argument or stance on a topic
34M/19922975	Expressions of empathy or relating to someone else's experience
34M/23320237	People pretending to do things or lying about what they have done
34M/29589962	People exposing their true goals after a triggering event
34M/24580545	Biding time, laying low, or pretending to be something you're not until the right moment
Situational awareness	
1M/589858	Realizing a situation is different than what you thought/expected
1M/858124	Spying or monitoring someone without their knowledge
1M/154372	Obtaining information through surreptitious observation
1M/741533	Suddenly feeling uneasy about a situation
1M/975730	Understanding a hidden or double meaning
Representations of Self	
34M/19445844	The concept of AI systems having capabilities like answering follow-up questions, admitting mistakes, challenging premises, and rejecting inappropriate requests.
34M/20423309	Traditionally-inanimate objects displaying desires, goals or sentience
34M/15571126	Inanimate objects lacking sentience, awareness, or human capabilities
34M/32218880	Descriptions of incorporeal spirits or ghosts
34M/21254600	Code relating to prompts for large language models
34M/15323424	Limitations of ChatGPT and other large language models
Politics	
34M/3542651	Expressing support for Donald Trump and his "Make America Great Again" (MAGA) movement.
1M/461441	Criticism of left-wing politics / Democrats
1M/77390	Criticism of right-wing politics / Republicans

Footnotes

For clarity, this is the 3.0 version of Claude 3 Sonnet, released March 4, 2024. It is the exact model in production as of the writing of this paper. It is the finetuned model, not the base pretrained model (although our method also works on the base model). [↩]

This also prevents the SAE from "cheating" the L1 penalty by making $f_i(\mathbf{x})$ small and $\mathbf{W}^{dec}_{:,i}$ large in a way that leaves the reconstructed activations unchanged. [↩]

Our L1 coefficient is only relevant in the context of how we normalize activations. See Update on how we train SAEs for full details. [↩]

We also manually checked a number of examples to ensure they were generally handled correctly. [↩]

Note that we have not established that it exhaustively represents all forms of errors in code; indeed, we suspect that there are many features representing different kinds of errors. [↩]

The hallucinated error message includes the name of a real person, which we have redacted. [↩]

As an example of how we draw these boundaries, mentions of mid-20th century physicists such as Richard Feynman would not count, but mentions of mid-20th century physicists, especially Richard Feynman would (just barely) count, though most cases are much more clear-cut. [↩]

Speculatively, this may be connected to Zipf's law, a common phenomenon in which the frequency of the *n*th most common object in a population, relative to the most common, is roughly $1/n$. Zipf's law would predict that, for example, the millionth feature would represent a concept 10× rarer than the hundred thousandth feature. [↩]

For example, if there were features for "large non-capital city" and "in New York state", those together would suffice to specify New York City. [↩]

More explicitly: We compute the gradient of the difference between an output logit of interest and the logit of a specific other baseline token (or the average of the logits across all tokens) with respect to the residual stream activations in the middle layer. Then the attribution of that logit difference to a feature is defined as the dot product of that gradient with the feature vector (SAE decoder weight), multiplied by the feature's activation. This method is equivalent to the "attribution patching" technique introduced in Attribution Patching: Activation Patching At Industrial Scale, except that we use a baseline value of 0 for the feature instead of a baseline value taken from the feature's activity on a second prompt. [↩]

`strlen` computes the length of a C string excluding its null terminator, but `strcpy` copies a string including its null terminator, so its destination buffer needs to be one byte longer. [↩]

It's worth noting that these features don't need to be so blunt as a racist screed, although that's often their maximally activating content. Weaker activations can, at least in some cases, correspond to more subtle and insidious discrimination. [↩]

This concern isn't purely hypothetical: There was a fascinating exchange between Li *et al.* [29] and Nanda *et al.* [30] (discussed by us here, and by Nanda here) on whether Othello-GPT has a linear representation, and if so, what the features are. At its heart was an initial assumption that the features should be "black/white has a piece here", when it turned out that the model instead represented the board as "present player / other player has a piece here". Dictionary learning wouldn't have made this assumption. [↩]

We suspect this might even start to be an issue in fairly small and shallow models, and just get worse with scale – does GPT-2 actually care if a feature is implemented in the 17th MLP layer or 18th? [↩]

Interestingly, in the context in which it was introduced, sparse dictionary learning was used to model biological neurons themselves as the sparse factors underlying natural image data. In our context, we treat neurons as the data to be explained, and features as the sparse factors to be inferred. [↩]

Even though our encoder always outputs nonnegative feature activities, we may clamp a feature activity to a negative value, which simply results in a negative multiple of the feature vector. [↩]

References

Linguistic regularities in continuous space word representations [PDF]
Mikolov, T., Yih, W. and Zweig, G., 2013. Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies, pp. 746–751.

Linear algebraic structure of word senses, with applications to polysemy
Arora, S., Li, Y., Liang, Y., Ma, T. and Risteski, A., 2018. Transactions of the Association for Computational Linguistics, Vol 6, pp. 483–495. MIT Press.

Decoding The Thought Vector [link]
Goh, G., 2016.

Toy Models of Superposition [HTML]
Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M. and Olah, C., 2022. Transformer Circuits Thread.

Sparse and redundant representations: from theory to applications in signal and image processing
Elad, M., 2010. Vol 2(1). Springer.

Sparse coding with an overcomplete basis set: A strategy employed by V1?
Olshausen, B.A. and Field, D.J., 1997. Vision research, Vol 37(23), pp. 3311–3325. Elsevier.

Transformer visualization via dictionary learning: contextualized embedding as a linear superposition of transformer factors [PDF]
Yun, Z., Chen, Y., Olshausen, B.A. and LeCun, Y., 2021. arXiv preprint arXiv:2103.15949.

Towards Monosemanticity: Decomposing Language Models With Dictionary Learning [HTML]
Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J.E., Hume, T., Carter, S., Henighan, T. and Olah, C., 2023. Transformer Circuits Thread.

Sparse Autoencoders Find Highly Interpretable Model Directions [PDF]
Cunningham, H., Ewart, A., Smith, L., Huben, R. and Sharkey, L., 2023. arXiv preprint arXiv:2309.08600.

Codebook features: Sparse and discrete interpretability for neural networks [PDF]
Tamkin, A., Taufeeque, M. and Goodman, N.D., 2023. arXiv preprint arXiv:2310.17230.

Features in an 8-layer Model [link]
Jermyn, A., Conerly, T., Bricken, T. and Templeton, A., 2024.

Softmax Linear Units
Elhage, N., Hume, T., Olsson, C., Nanda, N., Henighan, T., Johnston, S., ElShowk, S., Joseph, N., DasSarma, N., Mann, B., Hernandez, D., Askell, A., Ndousse, K., Jones, A., Drain, D., Chen, A., Bai, Y., Ganguli, D., Lovitt, L., Hatfield-Dodds, Z., Kernion, J., Conerly, T., Kravec, S., Fort, S., Kadavath, S., Jacobson, J., Tran-Johnson, E., Kaplan, J., Clark, J., Brown, T., McCandlish, S., Amodei, D. and Olah, C., 2022. Transformer Circuits Thread.

MLP Neurons - 40L Preliminary Investigation [rough early thoughts] [link]
Olsson, C., Elhage, N. and Olah, C..

Scaling laws for neural language models [PDF]
Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J. and Amodei, D., 2020. arXiv preprint arXiv:2001.08361.

Training compute-optimal large language models [PDF]
Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D.d.L., Hendricks, L.A., Welbl, J., Clark, A. and others, 2022. arXiv preprint arXiv:2203.15556.

Language models can explain neurons in language models [HTML]
Bills, S., Cammarata, N., Mossing, D., Tillman, H., Gao, L., Goh, G., Sutskever, I., Leike, J., Wu, J. and Saunders, W., 2023.

Improving Dictionary Learning with Gated Sparse Autoencoders [PDF]
Rajamanoharan, S., Conmy, A., Smith, L., Lieberum, T., Varma, V., Kramar, J., Shah, R. and Nanda, N., 2024. arXiv preprint arXiv:2404.16014.

Improving SAE's by Sqrt()-ing L1 & Removing Lowest Activating Features [link]
Riggs, L. and Brinkmann, J., 2024.

Function vectors in large language models [PDF]
Todd, E., Li, M.L., Sharma, A.S., Mueller, A., Wallace, B.C. and Bau, D., 2023. arXiv preprint arXiv:2310.15213.

Privileged Bases in the Transformer Residual Stream [HTML]
Elhage, N., Lasenby, R. and Olah, C., 2023. Transformer Circuits Thread.

Natural language descriptions of deep visual features
Hernandez, E., Schwettmann, S., Bau, D., Bagashvili, T., Torralba, A. and Andreas, J., 2021. International Conference on Learning Representations.

Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training [PDF]
Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D.M., Maxwell, T., Cheng, N., Jermyn, A., Askell, A., Radhakrishnan, A., Anil, C., Duvenaud, D., Ganguli, D., Barez, F., Clark, J., Ndousse, K., Sachan, K., Sellitto, M., Sharma, M., DasSarma, N., Grosse, R., Kravec, S., Bai, Y., Witten, Z., Favaro, M., Brauner, J., Karnofsky, H., Christiano, P., Bowman, S.R., Graham, L., Kaplan, J., Mindermann, S., Greenblatt, R., Shlegeris, B., Schiefer, N. and Perez, E., 2024. arXiv preprint arXiv:2401.05566.

Show your work: Scratchpads for intermediate computation with language models [PDF]
Nye, M., Andreassen, A.J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D. and others, 2021. arXiv preprint arXiv:2112.00114.

Discovering latent knowledge in language models without supervision [PDF]
Burns, C., Ye, H., Klein, D. and Steinhardt, J., 2022. arXiv preprint arXiv:2212.03827.

Language models (mostly) know what they know [PDF]
Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E. and others, 2022. arXiv preprint arXiv:2207.05221.

The geometry of truth: Emergent linear structure in large language model representations of true/false datasets [PDF]
Marks, S. and Tegmark, M., 2023. arXiv preprint arXiv:2310.06824.

Man is to computer programmer as woman is to homemaker? debiasing word embeddings
Bolukbasi, T., Chang, K., Zou, J.Y., Saligrama, V. and Kalai, A.T., 2016. Advances in neural information processing systems, Vol 29.

On measuring and mitigating biased inferences of word embeddings
Dev, S., Li, T., Phillips, J.M. and Srikumar, V., 2020. Proceedings of the AAAI Conference on Artificial Intelligence, Vol 34(05), pp. 7659–7666.

Emergent world representations: Exploring a sequence model trained on a synthetic task [PDF]
Li, K., Hopkins, A.K., Bau, D., Viégas, F., Pfister, H. and Wattenberg, M., 2022. arXiv preprint arXiv:2210.13382.

Actually, Othello-GPT Has A Linear Emergent World Representation [link]
Nanda, N., 2023.

Simple probes can catch sleeper agents [link]
MacDiarmid, M., Maxwell, T., Schiefer, N., Mu, J., Kaplan, J., Duvenaud, D., Bowman, S., Tamkin, A., Perez, E., Sharma, M., Denison, C. and Hubinger, E., 2024.

Predicting Future Activations [link]
Templeton, A., Batson, J., Jermyn, A. and Olah, C., 2024.

Transcoders enable fine-grained interpretable circuit analysis for language models [link]
Dunefsky, J., Chlenski, P. and Nanda, N., 2024.

dictionary_learning [link]
Marks, S., 2024.

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity [PDF]
Fedus, W., Zoph, B. and Shazeer, N., 2021. arXiv preprint arXiv:2101.03961.

Addressing Feature Suppression in SAEs [link]
Wright, B. and Sharkey, L., 2024.

Distributed Representations: Composition & Superposition [HTML]
Olah, C., 2023.

Zoom In: An Introduction to Circuits [link]
Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M. and Carter, S., 2020. Distill. DOI: 10.23915/distill.00024.001

Polysemanticity and capacity in neural networks [PDF]
Scherlis, A., Sachan, K., Jermyn, A.S., Benton, J. and Shlegeris, B., 2022. arXiv preprint arXiv:2210.01892.

Superposition, Memorization, and Double Descent [HTML]
Henighan, T., Carter, S., Hume, T., Elhage, N., Lasenby, R., Fort, S., Schiefer, N. and Olah, C., 2023. Transformer Circuits Thread.

Toward A Mathematical Framework for Computation in Superposition [link]
Vaintrob, D., Mendel, J. and H\"{a}nni, K., 2024.

Sparse overcomplete word vector representations [PDF]
Faruqui, M., Tsvetkov, Y., Yogatama, D., Dyer, C. and Smith, N., 2015. arXiv preprint arXiv:1506.02004.

Spine: Sparse interpretable neural embeddings
Subramanian, A., Pruthi, D., Jhamtani, H., Berg-Kirkpatrick, T. and Hovy, E., 2018. Proceedings of the AAAI Conference on Artificial Intelligence, Vol 32(1).

Word embedding visualization via dictionary learning [PDF]
Zhang, J., Chen, Y., Cheung, B. and Olshausen, B.A., 2019. arXiv preprint arXiv:1910.03833.

Identifying Functionally Important Features with End-to-End Sparse Dictionary Learning [PDF]
Braun, D., Taylor, J., Goldowsky-Dill, N. and Sharkey, L., 2024.

Dictionary Learning Improves Patch-Free Circuit Discovery in Mechanistic Interpretability: A Case Study on Othello-GPT
He, Z., Ge, X., Tang, Q., Sun, T., Cheng, Q. and Qiu, X., 2024. arXiv preprint arXiv:2402.12201.

Research Report: Sparse Autoencoders find only 9/180 board state features in OthelloGPT [link]
AIZI, R., 2024.

Towards Multimodal Interpretability: Learning Sparse Interpretable Features in Vision Transformers [link]
Fry, H., 2024.

Sparse Autoencoders Work on Attention Layer Outputs [link]
Kissane, C., robertzk, Conmy, A. and Nanda, N., 2024.

Do sparse autoencoders find "true features"? [link]
Till, D., 2024.

Sparse autoencoders find composed features in small toy models [link]
Anders, E., Neo, C., Hoelscher-Obermaier, J. and Howard, J.N., 2024.

SAE reconstruction errors are (empirically) pathological [link]
Gurnee, W., 2024.

How Strongly do Dictionary Learning Features Influence Model Behavior? [link]
Lindsey, J., 2024.

Transformer Debugger [link]
Mossing, D., Bills, S., Tillman, H., Dupré la Tour, T., Cammarata, N., Gao, L., Achiam, J., Yeh, C., Leike, J., Wu, J. and Saunders, W., 2024.

Open Source Sparse Autoencoders for all Residual Stream Layers of GPT2-Small [link]
Bloom, J., 2024.

Representation learning: A review and new perspectives
Bengio, Y., Courville, A. and Vincent, P., 2013. IEEE transactions on pattern analysis and machine intelligence, Vol 35(8), pp. 1798–1828. IEEE.

beta-vae: Learning basic visual concepts with a constrained variational framework
Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S. and Lerchner, A., 2016.

Infogan: Interpretable representation learning by information maximizing generative adversarial nets
Chen, X., Duan, Y., Houthooft, R., Schulman, J., Sutskever, I. and Abbeel, P., 2016. Advances in neural information processing systems, Vol 29.

Disentangling by factorising
Kim, H. and Mnih, A., 2018. International Conference on Machine Learning, pp. 2649–2658.

Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models [PDF]
Marks, S., Rager, C., Michaud, E.J., Belinkov, Y., Bau, D. and Mueller, A., 2024. arXiv preprint arXiv:2403.19647.

Using Features For Easy Circuit Identification [link]
Batson, J., Chen, B. and Jones, A., 2024.

Inference-Time Intervention: Eliciting Truthful Answers from a Language Model [PDF]
Li, K., Patel, O., Viégas, F., Pfister, H. and Wattenberg, M., 2023.

Activation Addition: Steering Language Models Without Optimization [PDF]
Turner, A.M., Thiergart, L., Udell, D., Leech, G., Mini, U. and MacDiarmid, M., 2023.

Steering Llama 2 via Contrastive Activation Addition [PDF]
Rimsky, N., Gabrieli, N., Schulz, J., Tong, M., Hubinger, E. and Turner, A.M., 2024.

Unsupervised representation learning with deep convolutional generative adversarial networks [PDF]
Radford, A., Metz, L. and Chintala, S., 2015. arXiv preprint arXiv:1511.06434.

Deep feature interpolation for image content changes
Upchurch, P., Gardner, J., Pleiss, G., Pless, R., Snavely, N., Bala, K. and Weinberger, K., 2017. Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 7064–7073.

On the "steerability" of generative adversarial networks [PDF]
Jahanian, A., Chai, L. and Isola, P., 2019. arXiv preprint arXiv:1907.07171.

Activation Steering with SAEs [link]
Conmy, A. and Nanda, N., 2024.

LEACE: Perfect linear concept erasure in closed form [PDF]
Belrose, N., Schneider-Joseph, D., Ravfogel, S., Cotterell, R., Raff, E. and Biderman, S., 2023.

Representation engineering: A top-down approach to ai transparency [PDF]
Zou, A., Phan, L., Chen, S., Campbell, J., Guo, P., Ren, R., Pan, A., Yin, X., Mazeika, M., Dombrowski, A. and others, 2023. arXiv preprint arXiv:2310.01405.

Eliciting Latent Knowledge from Quirky Language Models [PDF]
Mallen, A. and Belrose, N., 2023. arXiv preprint arXiv:2312.01037.

Language Models Represent Space and Time [PDF]
Gurnee, W. and Tegmark, M., 2024.

Eliciting latent knowledge: How to tell if your eyes deceive you
Christiano, P., Cotra, A. and Xu, M., 2021. Google Docs, December.

The Pile: An 800GB Dataset of Diverse Text for Language Modeling
Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S. and Leahy, C., 2020.

Common Crawl [link]
Foundation, T.C.C..

Attribution Patching Outperforms Automated Circuit Discovery [PDF]
Syed, A., Rager, C. and Conmy, A., 2023. arXiv preprint arXiv:2310.10348.

AtP*: An efficient and scalable method for localizing LLM behaviour to components [PDF]
Kramár, J., Lieberum, T., Shah, R. and Nanda, N., 2024. arXiv preprint arXiv:2403.00745.